Guía de programación (defensa 6)

> **Defensa 6.** Esta guía es una copia de `Defensa 5/Correcciones finales/Guia-programacion-final.md` con el cambio de la defensa 6: **el ataque por contacto lo resuelve la estrategia de ataque**. El Visitor detecta el choque y `Enemigo.colisionarCon(Link)` llama a `ataque.ataqueAlContactar(this, l)`; `AtaquePorContacto` guarda su `danio` y lo aplica; `Link.colisionarCon(Enemigo)` queda con implementación vacía; sale `danioContacto` de `Enemigo*`; y el MasterStalfos alterna `AtaqueEspada` y `AtaquePorContacto`. Ver Defensa 6: ataque por contacto. Las partes afectadas están actualizadas y dicen "defensa 6".

Esta guía junta en un solo lugar **todo lo que hace falta para programar el juego a partir de los dos diagramas finales**:

- `Diagrama-reducido-defensa-6.drawio`: clases y relaciones (igual al de las correcciones finales).
- `Diagrama-extendido-defensa-6.drawio`: atributos y métodos.

Los dos están en esta misma carpeta, `Defensa 6/`. Cada clase tiene su ficha con:

- su utilidad;
- cada atributo, con su tipo y para qué sirve;
- cada método, con qué hace y quién lo llama;
- sus relaciones del reducido y cómo se escriben en Java;
- un esqueleto de código cuando ayuda;
- las consideraciones que hay que tener en cuenta al programarla.

Al final hay flujos completos con diagramas de secuencia, la concurrencia entre hilos, los valores del juego y una lista de decisiones que el diagrama no fija.

Cómo se relaciona con los otros documentos:

- `Explicacion-diagrama-reducido-defensa-6.md` y `Explicacion-diagrama-extendido-defensa-6.md` explican el *porqué* del diseño (para la defensa).
- `Defensa 5/Correcciones finales/Cambios-correcciones-finales.md` explica qué cambió en las correcciones finales.
- Esta guía explica el *cómo*: qué escribir en cada clase. Si algo de un explicativo no coincide con esta guía, manda el **extendido**, y en segundo lugar esta guía, que está armada a partir de él.

Marcas que se usan:

Marca	Significado
(extendido)	Está tal cual en el diagrama extendido: nombre, parámetros y tipo de retorno exactos.
(propuesta)	El diagrama no lo fija; es una forma razonable de programarlo que conviene confirmar en el equipo.
(a confirmar)	Depende del enunciado o del ayudante; está en Decisiones abiertas.
⚠️	Algo que, si se programa sin cuidado, produce un error difícil de encontrar.

Índice

- Defensa 6: ataque por contacto
- 1. Cómo leer los diagramas
  - 1.1 Notación del extendido
  - 1.2 Notación del reducido
  - 1.3 Unidades que usa todo el código
- 2. Reglas que no se pueden romper
- 3. Capas y paquetes sugeridos
- 4. Orden sugerido para programar
- 5. Valores del juego
  - 5.1 Link
  - 5.2 Enemigos y jefes
  - 5.3 Power-ups
  - 5.4 Hilos
- 6. Todas las relaciones del reducido
  - 6.1 Herencias (`extends`)
  - 6.2 Realizaciones (`implements`)
  - 6.3 Composiciones (◆)
  - 6.4 Agregaciones (◇)
  - 6.5 Asociaciones (▶)
  - 6.6 Dependencias (no dibujadas)
- 7. Utilidades: Direccion, Posicion, Hitbox y Sprite
  - 7.1 Direccion (enumerado)
  - 7.2 Posicion
  - 7.3 Hitbox
  - 7.4 Sprite
- 8. La base de las entidades: Colisionable, Colisionador, EntidadLogica y Entidad*
  - 8.1 Colisionable (interfaz)
  - 8.2 Colisionador (interfaz)
  - 8.3 EntidadLogica (interfaz)
  - 8.4 Entidad* (abstracta)

- 9. Obstáculos
  - 9.1 Obstaculo* (abstracta)
  - 9.2 Las cinco subclases
- 10. Power-ups
  - 10.1 PowerUP* (abstracta)
  - 10.2 Las cuatro subclases
- 11. Objetos: Objeto*, Espada y Escudo
  - 11.1 Objeto* (abstracta)
  - 11.2 Espada y Escudo
- 12. EntidadAnimada* y AccionProgramada
  - 12.1 EntidadAnimada* (abstracta)
  - 12.2 AccionProgramada
- 13. Link y sus interfaces
  - 13.1 Link
  - 13.2 ControlJugador (interfaz)
  - 13.3 EntidadJugador (interfaz)
- 14. Proyectiles, enemigos y jefes
  - 14.1 Proyectil
  - 14.2 Enemigo* (abstracta)
  - 14.3 EnemigoConDrop* y GeneradorDrop
  - 14.4 GeneradorProyectil (interfaz)
  - 14.5 Los seis enemigos comunes
  - 14.6 Jefe*, MasterStalfos y StoneHinox
- 15. Comportamientos (patrón Strategy)
  - 15.1 Las tres interfaces
  - 15.2 Las tres estrategias de movimiento
  - 15.3 Las cuatro estrategias de ataque
  - 15.4 Las dos estrategias de defensa
- 16. Mapa y niveles
  - 16.1 Sector
  - 16.2 Escena
  - 16.3 Nivel
  - 16.4 GeneradorNivel y el formato del archivo
  - 16.5 AdministradorColisiones
- 17. Fábricas (Abstract Factory)
  - 17.1 FabricaEntidades
  - 17.2 FabricaSprites*, FabricaSpritesOriginal y FabricaSpritesAlternativa
- 18. Núcleo del juego
  - 18.1 ControladorJuego (interfaz)
  - 18.2 ControladorGrafico (interfaz)
  - 18.3 Juego
  - 18.4 ModoJuego*, Clasico, Contrarreloj y NoHit
  - 18.5 Ranking, EntradaRanking y PersistenciaRanking
- 19. Hilos
  - 19.1 Runnable (de Java)
  - 19.2 HiloJuego* (abstracta, Template Method)
  - 19.3 HiloJugador, HiloMundo y Reloj
- 20. Vista
  - 20.1 GUI
  - 20.2 Ventana
  - 20.3 PanelVista* y los tres paneles
  - 20.4 ControladorDeVistas (interfaz)
  - 20.5 Observer, CeldaGrafica y CeldaJugador
  - 20.6 AnimadorTransicion
- 21. Flujos completos (diagramas de secuencia)
  - 21.1 Arranque de la aplicación
  - 21.2 Del menú al ranking
  - 21.3 Iniciar una partida
  - 21.4 Cargar un nivel y activar el sector inicial
  - 21.5 Una vuelta de HiloJugador
  - 21.6 Link mata a un Moblin y aparece un drop
  - 21.7 El Octorok dispara y la roca alcanza a Link quieto
  - 21.8 Cambio de sector
  - 21.9 Suelo quebradizo y caída al vacío
  - 21.10 Daño, vidas y fin por vidas
  - 21.11 Aparece una bellota
  - 21.12 Jefe derrotado, fin de nivel y ranking
  - 21.13 Contrarreloj: un segundo
  - 21.14 NoHit: el primer daño termina la partida
- 22. Concurrencia: qué se sincroniza y por qué
  - 22.1 Los hilos y qué toca cada uno
  - 22.2 La regla de las colisiones
  - 22.3 Qué se protege
  - 22.4 ⚠️ El abrazo mortal que hay que evitar

    - 22.5 Lo que corre en el EDT
- 23. Correspondencia 1 a 1 con el extendido
- 24. Decisiones abiertas y cosas que el diagrama no fija
- 25. Checklist antes de entregar
- 26. Glosario

Defensa 6: ataque por contacto

Idea: el Visitor **detecta** el choque y la **estrategia de ataque** del enemigo **decide** qué pasa.

Clase	Cambio
`ComportamientoAtaque`	+ `ataqueAlContactar(atacante : EntidadAnimada, tocado : EntidadAnimada)`
`AtaquePorContacto`	+ `# danio : int`, constructor `(danio : int)`; `ataqueAlContactar` aplica `danio`; `atacar` sigue vacío
`AtaqueEspada`	+ `ataqueAlContactar` vacío (lo hereda `AtaqueRecargado`)
`AtaquePorProyectil`	+ `ataqueAlContactar` vacío (lo hereda `AtaqueAvalancha`)
`Enemigo*`	− `# danioContacto`, − `getDanioContacto()`, constructor `(sprite, x, y, vida)`; + `colisionarCon(l : Link)`
`EnemigoConDrop*`, `Jefe*`	Constructor sin `danioContacto`
`Link`	`colisionarCon(e : Enemigo)` se mantiene (lo define `Colisionador`) con implementación vacía
`MasterStalfos`	+ `- espada`, `- contacto`, `- DURACION_FASE : long = 2000`, `- faseEspada()`, `- faseContacto()`

Recorrido (un Moblin toca a Link): `HiloMundo` → `c.verificarColisiones(moblin)` → `link.aceptarColision(moblin)` → `moblin.colisionarCon(link)` → `ataque.ataqueAlContactar(moblin, link)` → `link.recibirDanio(1, direccion)`.

Quién lastima al tocar: Moblin, MoblinConEspadaYEscudo, ErizoDeMar y TrampaDeCuchillas (`AtaquePorContacto(1)`), StoneHinox al embestir (`AtaquePorContacto(2)`) y MasterStalfos en su fase de contacto (`AtaquePorContacto(1)`). El Octorok y el Wizzrobe no: solo lastiman sus proyectiles (decisión del equipo).

Hilos: el choque Link–enemigo ahora lo resuelve `HiloMundo` (el del enemigo). `recibirDanio` no cambia, pero se llama desde `HiloMundo`, así que tiene que ser `synchronized` (ya lo era). Ver 22.2.

1. Cómo leer los diagramas

1.1 Notación del extendido

Notación	Significado	En Java
`Nombre*` (en cursiva)	Clase abstracta	`public abstract class Nombre`
`«interface»`	Interfaz	`public interface Nombre`
`«enumeration»`	Enumerado	`public enum Nombre`
`+`	Público	`public`
`#`	Protegido: lo ven las subclases (y el paquete)	`protected`
`-`	Privado	`private`
`«constructores»`	Constructores	`public Nombre(...)` o `protected Nombre(...)` si es `#`
`«comandos»`	Métodos que cambian el estado (casi siempre `void`)	
`«consultas»`	Métodos que devuelven algo y no cambian el estado	
`ATRIBUTO : int = 48` en mayúsculas	Constante	`static final`
Nombre de método igual al de la superclase	Redefinición	`@Override`

1.2 Notación del reducido

Dibujo	Relación	Cómo se escribe en Java
Línea continua con triángulo vacío	Herencia	`class B extends A`
Línea punteada con triángulo vacío	Realización	`class B implements I`
Rombo lleno ◆ del lado del todo	Composición: el todo crea la parte y la parte muere con él	Atributo del todo; la parte se crea **dentro** del todo (o se le entrega recién creada) y no se comparte
Rombo vacío ◇ del lado del todo	Agregación: el todo usa la parte, que puede existir aparte o cambiarse	Atributo del todo, con un *setter* o recibido por parámetro
Flecha negra pintada ▶	Asociación navegable	Atributo de la clase de origen, del tipo de la de destino
`#nombre mult` junto a la flecha	Rol: es **el nombre del atributo** y su multiplicidad	`protected Tipo nombre;` (`1` o `0..1`) o `protected List<Tipo> nombre;` (`0..*`, `1..*`)

Regla práctica: cada rol `#algo` del reducido es un atributo `# algo : Tipo` del extendido. Si en el código aparece un atributo que no está en el diagrama, o falta uno que sí está, hay que revisar.

1.3 Unidades que usa todo el código

Magnitud	Unidad	Ejemplo
Posiciones y tamaños	**Píxeles**, relativos a la esquina superior izquierda del **sector**	Link en `(96, 240)`
Tamaño de un sector	`Sector.ANCHO = 768`, `Sector.ALTO = 528` (16 × 11 celdas de 48 px)	
Celdas	**Solo** al leer el archivo del nivel (`GeneradorNivel.CELDA = 48`)	La celda `(2, 5)` es el píxel `(96, 240)`
Vida de Link y daño hacia Link	**Medios corazones**	½ corazón = `1`; 1 corazón = `2`
Vida de los enemigos y daño hacia ellos	**Golpes**	Moblin: 2 golpes; la espada normal saca `1`
Tiempos	Milisegundos (`long`), con `System.currentTimeMillis()`	Invulnerabilidad de Link: `1000`
Velocidad	Píxeles por vuelta del hilo (una vuelta ≈ 30 ms)	Link camina `4` px por vuelta ≈ 133 px/s

---

2. Reglas que no se pueden romper

Salen del enunciado y de la sección 8 de `CONTEXTO-proyecto-zelda.md`. Romper cualquiera desaprueba o contradice el diseño defendido.

Regla	Qué significa al programar	Cómo lo resuelve el diseño
Prohibido `instanceof` (y también `getClass()` para decidir)	Nunca preguntar el tipo de un objeto para decidir qué hacer	Doble despacho (`Colisionable` / `Colisionador`), ganchos vacíos en las superclases y Strategy
Solo Java + Swing, sin librerías externas	Nada de JavaFX, ni librerías de juego, ni JSON	Lectura de archivos con `java.io`
Movimientos y animaciones con `Thread`, **nunca** `javax.swing.Timer`	Ningún `Timer` ni `ScheduledExecutorService` para mover cosas	`HiloJugador`, `HiloMundo`, `Reloj` y `AnimadorTransicion`
Al menos 2 hilos a la vez		Hay 3 fijos más el de la transición
Swing solo en la capa de vista	Ninguna clase de la lógica importa `javax.swing` ni `java.awt`	La lógica solo conoce `Observer`, `EntidadLogica`, `EntidadJugador`, `ControladorGrafico` y `ControlJugador`
Swing solo desde su hilo (EDT)	Todo cambio en un componente va dentro de `SwingUtilities.invokeLater`	Lo hacen `CeldaGrafica`, `PanelPantallaNivel` y `GUI`
Nombres exactos del enunciado	Las clases se llaman igual en los dos dominios gráficos	Los dominios solo cambian las imágenes (`FabricaSprites*`)
Patrones usados de forma clara	Strategy, Observer, Abstract Factory (pedidos) y los que ya están en el diagrama	No agregar patrones nuevos sin acordarlo
Commits repartidos	Cada integrante con commits reales	Ver el orden de trabajo de la sección 4
Código igual al extendido (1 a 1)	La defensa 6 lo evalúa: cada atributo y cada método del código tiene que estar en el extendido, con el mismo nombre, y al revés	Lo marcado **(propuesta)** en esta guía evita agregar miembros nuevos. Si igual hace falta uno (por ejemplo, un método privado auxiliar), se agrega también al extendido
Clean Code	Indentación pareja, nombres claros, cada método hace una sola cosa	Los esquemas de esta guía ya están cortados en métodos chicos con los nombres del diagrama

---

3. Capas y paquetes sugeridos

El contexto del proyecto dice que la estructura de paquetes **todavía no está definida** y que hay que acordarla antes de crear archivos. Esta es una **propuesta** que respeta las capas del diagrama:

Paquete	Clases
`vista`	`GUI`, `Ventana`, `PanelVista*`, `PanelPantallaPrincipal`, `PanelPantallaRanking`, `PanelPantallaNivel`, `CeldaGrafica`, `CeldaJugador`, `AnimadorTransicion`, `ControladorDeVistas`
`logica` (contratos con la vista)	`ControladorJuego`, `ControladorGrafico`, `ControlJugador`, `Observer`, `EntidadLogica`, `EntidadJugador`
`logica.juego`	`Juego`, `ModoJuego*`, `Clasico`, `Contrarreloj`, `NoHit`, `Ranking`, `EntradaRanking`, `PersistenciaRanking`
`logica.hilos`	`HiloJuego*`, `HiloJugador`, `HiloMundo`, `Reloj`, `AccionProgramada`
`logica.mapa`	`Nivel`, `Escena`, `Sector`, `GeneradorNivel` (y el archivo `Formateo-de-niveles.pdf`, que la defensa 6 pide junto a la clase que parsea)
`logica.fabricas`	`FabricaEntidades`, `FabricaSprites*`, `FabricaSpritesOriginal`, `FabricaSpritesAlternativa`, `Sprite`
`logica.entidades`	`Entidad*`, `EntidadAnimada*`, `Colisionable`, `Colisionador`, `Link`, `Proyectil`, `AdministradorColisiones` (ver ⚠️ abajo)
`logica.entidades.enemigos`	`Enemigo*`, `EnemigoConDrop*`, `GeneradorDrop`, `GeneradorProyectil`, los seis enemigos, `Jefe*`, `MasterStalfos`, `StoneHinox`
`logica.entidades.fijas`	`Obstaculo*` y sus cinco subclases, `PowerUP*` y sus cuatro, `Objeto*`, `Espada`, `Escudo`
`logica.comportamientos`	Las tres interfaces de Strategy y sus siete implementaciones
`logica.util`	`Direccion`, `Posicion`, `Hitbox`
`recursos` (no es código)	`niveles/nivel1.txt`, `niveles/nivel2.txt`, `imagenes/original/…`, `imagenes/alternativa/…`, archivos de ranking

⚠️ **`protected` y los paquetes.** En Java, `protected` deja ver el atributo a las subclases **y a todo el paquete**. Si las estrategias quedan en otro paquete que las entidades, no pueden tocar atributos `#` de `EntidadAnimada`: solo sus métodos públicos. El diseño ya está pensado así (las estrategias llaman `setDireccion`, `avanzar`, `getPosicion`…), pero conviene recordarlo al programar.

⚠️ **`AdministradorColisiones` va en el paquete de las entidades.** Su método `mover` llama a dos ganchos protegidos de `EntidadAnimada*`: `# alChocarConBloqueo()` y `# alSalirDelSector(d)`. Desde otro paquete eso **no compila**, porque `AdministradorColisiones` no es subclase de `EntidadAnimada*`. Ponerlo en `logica.entidades` lo resuelve sin tocar el diagrama. Si el equipo prefiere dejarlo en `logica.mapa`, hay que pasar los dos ganchos a públicos (`+`) en el extendido y en el código.

La clase con `main` no está en el diagrama (no hace falta). Alcanza con una clase chica, por ejemplo `Principal` en el paquete raíz, que haga lo que se muestra en 21.1.

---

4. Orden sugerido para programar

Cada paso deja algo que se puede probar. Los pasos de una misma fila se pueden repartir entre integrantes.

Paso	Qué programar	Cómo se prueba
1	`Direccion`, `Posicion`, `Hitbox`, `Sprite`	Pruebas chicas en un `main`: `desplazada`, `intersecta`, `adelante`, `direccionHacia`
2	Interfaces de contrato: `Observer`, `EntidadLogica`, `EntidadJugador`, `Colisionable`, `Colisionador`, `ControlJugador`, `ControladorJuego`, `ControladorGrafico`, `ControladorDeVistas`	Compilan
3	`Entidad*`, `Obstaculo*` y sus subclases, `PowerUP*`, `Objeto*` (sin efectos todavía)	Se crean con un sprite de prueba
4	`Sector`, `Escena`, `Nivel`, `GeneradorNivel`, `FabricaSprites*` y concretas, `FabricaEntidades` (solo fijas)	Cargar `nivel1.txt` e imprimir cuántas entidades hay por sector (sector (0,0): 62)
5	Vista mínima: `Ventana`, `PanelPantallaNivel`, `CeldaGrafica`, `GUI` con `mostrarEntidad`	Se ve el sector (0,0) quieto
6	`HiloJuego*`, `AccionProgramada`, `EntidadAnimada*`, `AdministradorColisiones`, `MovimientoInducido`, `Link` caminando, `HiloJugador`, `CeldaJugador`	Link camina con las flechas y las rocas lo frenan
7	`Juego` (iniciar, cargarNivel, activarSector, agregarEntidad), `Reloj`, HUD	Partida que arranca desde el menú; el tiempo corre
8	Estrategias restantes, `Enemigo*`, `EnemigoConDrop*`, enemigos concretos, `Proyectil`, `HiloMundo`	Los enemigos se mueven, disparan y lastiman
9	Espada, escudo, power-ups, obstáculos con efecto, cambio de sector con `AnimadorTransicion`	Recorrer el nivel 1
10	Jefes, fin de nivel, modos, ranking y persistencia, menú y pantalla de ranking, sonidos	Partida completa en los tres modos

---

5. Valores del juego

Los que salen del enunciado están marcados **(enunciado)**. El resto son **(propuesta)**: valores que usan los esquemas de esta guía y que se pueden ajustar al probar.

5.1 Link

Dato	Valor	Origen
Vidas al empezar	3	(enunciado)
Corazones al empezar	3 corazones = `6` medios (`vida = corazonesMaximos = 6`)	(propuesta, como el juego original)
Velocidad	4 px por vuelta; 2 px sobre pasto	(propuesta)
Espada	Daño 1 golpe, alcance 32 px	(propuesta)
Ataque recargado	Mantener 3 s (`TIEMPO_CARGA = 3000`) y soltar: daño ×2, alcance ×2	(enunciado: 3 s)
Invulnerabilidad después de un golpe	1000 ms	(propuesta)
Suelo quebradizo	Se rompe 500 ms después de pisarlo	(propuesta)
Fragmentos de corazón	Cada 4, +1 corazón máximo (`+2` medios)	(enunciado)

5.2 Enemigos y jefes

Clase	Golpes (`vida`)	Contacto (medios)	Ataque	Puntos	Drop	Movimiento
`Moblin`	2 (enunciado)	1 (enunciado)	Contacto	100 (enunciado)	20% (enunciado)	Aleatorio, 2 px, cambia cada 1500 ms
`MoblinConEspadaYEscudo`	2 (enunciado)	1 (enunciado)	Contacto, se abalanza	300 (enunciado)	50% (enunciado)	Aleatorio → inducido a 6 px
`ErizoDeMar`	1 (enunciado)	1	Contacto	50 (enunciado)	20% (enunciado)	Quieto
`Octorok`	1 (enunciado)	0 (defensa 6, decisión del equipo)	Proyectil de 1 medio cada 2000 ms	150 (enunciado)	40% (enunciado)	Aleatorio, 2 px
`Wizzrobe`	2 (enunciado)	0 (defensa 6, decisión del equipo)	Proyectil cada 3000 ms	500 (enunciado)	100% (enunciado)	Teletransporte cada 3000 ms
`TrampaDeCuchillas`	Indestructible (enunciado)	1	Contacto	0	No suelta (enunciado)	Quieta → 8 px hacia Link

Clase	Golpes (`vida`)	Contacto (medios)	Ataque	Puntos	Drop	Movimiento
`MasterStalfos`	5 (enunciado: ≥ 5)	1 (enunciado), solo en su fase de contacto (defensa 6)	Espada de 2 medios = 1 corazón (enunciado); alterna espada y contacto cada 2 s (defensa 6)	5000 (enunciado)	No suelta (enunciado)	Aleatorio, 3 px
`StoneHinox`	10 (propuesta: "el que más")	2 (1 corazón, enunciado)	Embestida, 3 rocas, avalancha de 5 a 10 rocas (enunciado)	5000 (enunciado)	No suelta (enunciado)	Según el ataque
`Proyectil` común	Indestructible	—	1 medio	—	—	Recto, 6 px
Roca grande (`crearRocaGrande`)	Indestructible	—	2 medios (1 corazón, enunciado)	—	—	Recto o hacia su destino, 6 px

5.3 Power-ups

Clase	Efecto	Puntos
`Rupia`	Suma entre 1 y 5 rupias al azar (enunciado)	10 por rupia (enunciado)
`Bellota`	Los próximos 3 golpes hacen como mucho ½ corazón (enunciado)	300 (enunciado)
`Corazon`	+1 corazón = `curar(2)` (enunciado)	300 (enunciado)
`FragmentoDeCorazon`	Suma un fragmento; cada 4, +1 corazón máximo (enunciado)	300 (enunciado)

5.4 Hilos

Hilo	Período
`HiloJugador`	30 ms
`HiloMundo`	30 ms
`Reloj`	1000 ms
`AnimadorTransicion`	Unos 30 pasos de 15 ms (propuesta)

---

6. Todas las relaciones del reducido

Esta tabla es la lista de control al programar: cada asociación, agregación o composición tiene que aparecer como atributo, y cada herencia o realización como `extends` o `implements`.

6.1 Herencias (`extends`)

Subclase	Superclase
`Obstaculo*`, `PowerUP*`, `Objeto*`, `EntidadAnimada*`	`Entidad*`
`Roca`, `Agua`, `Pasto`, `CaidasAlVacio`, `SuelosQuebradizo`	`Obstaculo*`
`Rupia`, `Bellota`, `Corazon`, `FragmentoDeCorazon`	`PowerUP*`
`Espada`, `Escudo`	`Objeto*`
`Link`, `Proyectil`, `Enemigo*`	`EntidadAnimada*`
`EnemigoConDrop*`, `TrampaDeCuchillas`, `Jefe*`	`Enemigo*`
`Moblin`, `ErizoDeMar`, `Octorok`, `Wizzrobe`	`EnemigoConDrop*`
`MoblinConEspadaYEscudo`	`Moblin`
`MasterStalfos`, `StoneHinox`	`Jefe*`
`AtaqueRecargado`	`AtaqueEspada`
`AtaqueAvalancha`	`AtaquePorProyectil`
`Clasico`, `Contrarreloj`, `NoHit`	`ModoJuego*`
`FabricaSpritesOriginal`, `FabricaSpritesAlternativa`	`FabricaSprites*`
`HiloJugador`, `HiloMundo`, `Reloj`	`HiloJuego*`
`PanelPantallaPrincipal`, `PanelPantallaRanking`, `PanelPantallaNivel`	`PanelVista*`
`PanelVista*`	`JPanel` (Swing)
`CeldaGrafica`	`JLabel` (Swing)
`CeldaJugador`	`CeldaGrafica`
`EntidadJugador` (interfaz)	`EntidadLogica` (interfaz que extiende otra: `interface EntidadJugador extends EntidadLogica`)

6.2 Realizaciones (`implements`)

Clase	Interfaz
`Entidad*`	`Colisionable`, `EntidadLogica`
`EntidadAnimada*`	`Colisionador`

Clase	Interfaz
`Link`	`EntidadJugador`, `ControlJugador`
`EnemigoConDrop*`	`GeneradorDrop`
`Octorok`, `Wizzrobe`, `StoneHinox`	`GeneradorProyectil`
`MovimientoAleatorio`, `MovimientoInducido`, `MovimientoTeletransporte`	`ComportamientoMovimiento`
`AtaquePorContacto`, `AtaqueEspada`, `AtaquePorProyectil`	`ComportamientoAtaque`
`DefensaLimitada`, `DefensaPorEscudo`	`ComportamientoDefensa`
`Juego`	`ControladorJuego`
`GUI`	`ControladorGrafico`, `ControladorDeVistas`
`CeldaGrafica`	`Observer`
`HiloJuego*`, `AnimadorTransicion`	`Runnable` (de `java.lang`)

Las subclases heredan la realización: `AtaqueRecargado` también es un `ComportamientoAtaque` y no hace falta volver a escribir `implements`.

6.3 Composiciones (◆)

Todo	Parte	Rol (atributo del todo)	Quién crea la parte
`Juego`	`Reloj`	`#reloj 1`	`Juego.iniciar`
`Juego`	`HiloJugador`	`#hiloJugador 1`	`Juego.iniciar`
`Juego`	`HiloMundo`	`#hiloMundo 1`	`Juego.iniciar`
`Juego`	`AdministradorColisiones`	`#colisiones 1`	Constructor de `Juego`
`Juego`	`Ranking`	`#rankings 3`	Constructor de `Juego` (los carga `PersistenciaRanking`)
`Ranking`	`EntradaRanking`	`#entradas 0..5`	`Juego.registrarNombre` crea la entrada y el ranking la guarda
`Nivel`	`Escena`	`#mapa 1`	`GeneradorNivel` la arma y se la entrega al nivel
`Escena`	`Sector`	`#sectores 1..*`	`GeneradorNivel`
`Entidad*`	`Hitbox`	`#hitbox 1`	Constructor de `Entidad`
`EntidadAnimada*`	`AccionProgramada`	`#pendientes 0..*`	`EntidadAnimada.programar`
`PanelPantallaNivel`	`CeldaGrafica`	`#celdas 0..*`	`GUI.mostrarEntidad` / `mostrarJugador`
`GUI`	`Ventana`	`#ventana 1`	Constructor de `GUI`

6.4 Agregaciones (◇)

Todo	Parte	Rol	Por qué no es composición
`Juego`	`ModoJuego*`	`#modoJuego 1`	Lo crea el menú y cambia en cada partida
`Juego`	`Nivel`	`#nivel 1`	Lo crea `GeneradorNivel` y se reemplaza al pasar o reiniciar el nivel
`EntidadAnimada*`	`ComportamientoMovimiento`	`#movimiento 1`	Se cambia en tiempo de ejecución
`EntidadAnimada*`	`ComportamientoAtaque`	`#ataque 1`	Ídem
`EntidadAnimada*`	`ComportamientoDefensa`	`#defensa 1`	Ídem
`Link`	`Objeto*`	`#inventario 1..2`	La espada existe en el mapa antes de que Link la tenga

6.5 Asociaciones (▶)

Origen	Destino	Rol (atributo del origen)
`Entidad*`	`Sprite`	`#sprite 1`
`Entidad*`	`Observer`	`#observadores 0..*`
`Entidad*`	`Juego`	`#miJuego 1`
`Entidad*`	`Sector`	`#miSector 1` (cambio 4)
`Hitbox`	`Posicion`	`#posicion 1`
`EntidadAnimada*`	`Direccion`	`#direccion 1`
`Link`	`Objeto*`	`#objetoEquipado 0..1`
`Proyectil`	`Posicion`	`#destino 0..1`
`TrampaDeCuchillas`	`Posicion`	`#posicionOriginal 1`
`MovimientoInducido`	`Direccion`	`#direccion 0..1`
`MovimientoInducido`	`Posicion`	`#objetivo 0..1`
`AtaquePorProyectil`	`GeneradorProyectil`	`#generadorProyectil 1`
`DefensaPorEscudo`	`ComportamientoDefensa`	`#sinEscudo 1`
`Sector`	`Entidad*`	`#entidades 0..*`
`Nivel`	`Jefe*`	`#jefes 1..*`

Origen	Destino	Rol (atributo del origen)
`Nivel`	`Posicion`	`#posicionInicial 1`
`AdministradorColisiones`	`Sector`	`#sector 1`
`Juego`	`Link`	`#jugador 1`
`Juego`	`ControladorGrafico`	`#controladorGrafico 1`
`Juego`	`GeneradorNivel`	`#generadorNivel 1`
`Juego`	`FabricaEntidades`	`#fabricaEntidades 1`
`Juego`	`PersistenciaRanking`	`#persistencia 1`
`FabricaEntidades`	`FabricaSprites*`	`#fabricaSprites 1`
`Reloj`	`Juego`	`#juego 1`
`HiloJugador`	`Link`	`#jugador 1`
`HiloJugador`	`AdministradorColisiones`	`#colisiones 1`
`HiloMundo`	`AdministradorColisiones`	`#colisiones 1`
`GUI`	`ControladorJuego`	`#controladorJuego 1`
`GUI`	`ControlJugador`	`#controlJugador 1`
`GUI`	`PanelPantallaPrincipal`, `PanelPantallaRanking`, `PanelPantallaNivel`	`#panelPrincipal 1`, `#panelRanking 1`, `#panelNivel 1`
`PanelVista*`	`ControladorDeVistas`	`#controladorVistas 1`
`CeldaGrafica`	`EntidadLogica`	`#entidadObservada 1`
`CeldaJugador`	`EntidadJugador`	`#jugadorObservado 1`
`AnimadorTransicion`	`PanelPantallaNivel`	`#panel 1`
`AnimadorTransicion`	`ControladorJuego`	`#controlador 1`

Atributos del extendido sin flecha en el reducido (no es un error: son tipos de Java, de Swing o datos simples): `FabricaEntidades.juego`, `CeldaGrafica.panelNivel`, `PanelPantallaPrincipal.dominio` y `modo`, los `JLabel` y la `JTable` de los paneles, `Link.caminar`, `recargado` y `defensaBase` (son las mismas estrategias que agrega `EntidadAnimada*`, guardadas con su tipo concreto), `StoneHinox.embestida`, `lanzarRocas`, `avalancha` y `caminar`, `AnimadorTransicion.direccion` y `HiloJuego.hilo`.

6.6 Dependencias (no dibujadas)

El equipo decidió no dibujarlas. Son usos de paso (parámetro, retorno o `new`), **no** atributos:

Origen	Destino	Estereotipo	Dónde aparece
`FabricaSprites*`	`Sprite`	«crea»	`crearSpritesX()`
`GeneradorNivel`	`Nivel`	«crea»	`generar(...)`
`GeneradorDrop`	`Rupia`, `Corazon`	«crea»	`generarDrop()`
`GeneradorProyectil`	`Proyectil`	«crea»	`generarProyectil()`
`PersistenciaRanking`	`Ranking`	«carga/guarda»	`cargar` y `guardar`
`Posicion`	`Direccion`	«usa»	`desplazada(d, pasos)`
`GeneradorNivel`	`FabricaEntidades`	«usa»	Parámetro de `generar`
Estrategias, `EntidadAnimada*`	`AdministradorColisiones`	«usa»	Parámetro `c` de `actualizarEstado`, `mover`, `atacar`
`GUI`	`AnimadorTransicion`	«crea»	`transicion(d)`
`FabricaEntidades`	Todas las entidades	«crea»	`crearX(...)`

---

7. Utilidades: Direccion, Posicion, Hitbox y Sprite

Son las clases más chicas y las que usan todas las demás. Conviene programarlas primero y probarlas solas.

7.1 Direccion (enumerado)

Utilidad: las cuatro direcciones del juego. La usan Link, los enemigos, los proyectiles, la espada, el escudo y el cambio de sector.

Elemento	Qué es	Para qué
`ARRIBA`, `ABAJO`, `IZQUIERDA`, `DERECHA`	Valores	Hacia dónde mira o se mueve una entidad
`+ opuesta() : Direccion`	Consulta	La contraria. La usan el escudo del Moblin (¿el golpe viene de frente?) y el cambio de sector
`+ getDx() : int`	Consulta	`-1`, `0` o `1` en x (`IZQUIERDA` = -1, `DERECHA` = 1)
`+ getDy() : int`	Consulta	`-1`, `0` o `1` en y (`ARRIBA` = -1, `ABAJO` = 1). La y crece hacia abajo, como en Swing

Relaciones: la referencian `EntidadAnimada*` (`#direccion 1`) y `MovimientoInducido` (`#direccion 0..1`). La usan por parámetro `Posicion.desplazada`, `Hitbox.desplazada`, `Hitbox.adelante`, `Escena.getVecino` y casi todos los métodos de movimiento.

```
        public enum Direccion {
            ARRIBA(0, -1), ABAJO(0, 1), IZQUIERDA(-1, 0), DERECHA(1, 0);
```

```
            private final int dx;
            private final int dy;

            Direccion(int dx, int dy) {
                this.dx = dx;
                this.dy = dy;
            }

            public Direccion opuesta() {
                switch (this) {
                    case ARRIBA:    return ABAJO;
                    case ABAJO:     return ARRIBA;
                    case IZQUIERDA: return DERECHA;
                    default:        return IZQUIERDA;
                }
            }

            public int getDx() { return dx; }
            public int getDy() { return dy; }
        }
```

Consideraciones:

- `dx` y `dy` no están dibujados en el extendido como atributos, pero son la forma natural de tener `getDx()` y `getDy()` en un enumerado. Si el ayudante pide correspondencia estricta, se pueden escribir `getDx()` y `getDy()` con un `switch`, igual que `opuesta()`, y no hace falta ningún atributo. **(a confirmar)**
- Un `switch` sobre un enumerado no es `instanceof`: pregunta por un **valor**, no por el tipo de un objeto.

7.2 Posicion

Utilidad: un punto `(x, y)` en píxeles, relativo a la esquina superior izquierda del sector. Es **inmutable**: los métodos que "mueven" devuelven una posición nueva.

Elemento	Qué es	Para qué
`- x : int`, `- y : int`	Atributos privados	Coordenadas en píxeles
`+ Posicion(x, y)`	Constructor	
`+ getX()`, `+ getY()`	Consultas	
`+ desplazada(d, pasos) : Posicion`	Consulta	La posición corrida `pasos` píxeles en la dirección `d`. La usa `EntidadAnimada.avanzar`
`+ distancia(otra) : double`	Consulta	Distancia en línea recta. La usan `MasterStalfos.quiereAtacar` y `AtaqueAvalancha.puedeAtacar`
`+ direccionHacia(otra) : Direccion`	Consulta	La dirección (de las cuatro) que más acerca a `otra`. La usan los enemigos para mirar a Link

Relaciones: la referencian `Hitbox` (`#posicion 1`), `Proyectil` (`#destino 0..1`), `TrampaDeCuchillas` (`#posicionOriginal 1`), `MovimientoInducido` (`#objetivo 0..1`) y `Nivel` (`#posicionInicial 1`). Usa `Direccion` por parámetro (dependencia no dibujada).

```
        public class Posicion {
            private int x;
            private int y;

            public Posicion(int x, int y) {
                this.x = x;
                this.y = y;
            }

            public int getX() { return x; }
            public int getY() { return y; }

            public Posicion desplazada(Direccion d, int pasos) {
                return new Posicion(x + d.getDx() * pasos, y + d.getDy() * pasos);
            }

            public double distancia(Posicion otra) {
                return Math.hypot(otra.x - x, otra.y - y);
            }

            public Direccion direccionHacia(Posicion otra) {
                int dx = otra.x - x;
                int dy = otra.y - y;
                if (Math.abs(dx) >= Math.abs(dy))
                    return dx >= 0 ? Direccion.DERECHA : Direccion.IZQUIERDA;
                return dy >= 0 ? Direccion.ABAJO : Direccion.ARRIBA;
            }
        }
```

Ejemplo: `new Posicion(96, 240).desplazada(Direccion.DERECHA, 4)` da `(100, 240)`. Desde `(384, 240)`, `direccionHacia(new Posicion(96, 260))` da `IZQUIERDA` (la diferencia en x, 288, es mayor que en y, 20).

7.3 Hitbox

Utilidad: el rectángulo que ocupa una entidad, en píxeles. Todas las colisiones comparan rectángulos.

Elemento	Qué es	Para qué
`# posicion : Posicion`	Atributo	Esquina superior izquierda
`# ancho : int`, `# alto : int`	Atributos	Tamaño en píxeles (48 × 48 para casi todo)
`+ Hitbox(x, y, ancho, alto)`	Constructor	Lo crea el constructor de `Entidad*`
`+ setPosicion(p)`	Comando	La entidad se movió
`+ getPosicion()`, `getX()`, `getY()`, `getAncho()`, `getAlto()`	Consultas	
`+ intersecta(otra) : boolean`	Consulta	¿Se superponen los dos rectángulos? Base de `verificarColisiones` y `puedeMoverA`
`+ desplazada(d, pasos) : Hitbox`	Consulta	Un rectángulo nuevo, corrido. `puedeMoverA` lo usa para ver si el lugar al que se quiere ir está libre
`+ adelante(d, alcance) : Hitbox`	Consulta	El rectángulo que queda **delante** en la dirección `d`, de profundidad `alcance`. Es la zona que corta la espada
`+ estaDentroDe(ancho, alto) : boolean`	Consulta	¿Está todo el rectángulo dentro de `0..ancho` × `0..alto`? `AdministradorColisiones.estaFuera` lo usa con 768 × 528
`+ estaEnLineaCon(otra) : boolean`	Consulta	¿Comparten una franja horizontal o vertical? La usan la trampa, el Octorok y `linkEstaEnfrente()`

Relaciones: es parte de `Entidad*` (◆ `#hitbox 1`) y referencia a `Posicion` (`#posicion 1`).

```
        public class Hitbox {
            protected Posicion posicion;
            protected int ancho;
            protected int alto;

            public Hitbox(int x, int y, int ancho, int alto) {
                this.posicion = new Posicion(x, y);
                this.ancho = ancho;
                this.alto = alto;
            }

            public void setPosicion(Posicion p) { posicion = p; }

            public Posicion getPosicion() { return posicion; }
            public int getX() { return posicion.getX(); }
            public int getY() { return posicion.getY(); }
            public int getAncho() { return ancho; }
            public int getAlto() { return alto; }

            public boolean intersecta(Hitbox otra) {
                return getX() < otra.getX() + otra.ancho && otra.getX() < getX() + ancho
                    && getY() < otra.getY() + otra.alto && otra.getY() < getY() + alto;
            }

            public Hitbox desplazada(Direccion d, int pasos) {
                Posicion p = posicion.desplazada(d, pasos);
                return new Hitbox(p.getX(), p.getY(), ancho, alto);
            }

            public Hitbox adelante(Direccion d, int alcance) {
                switch (d) {
                    case DERECHA:   return new Hitbox(getX() + ancho, getY(), alcance, alto);
                    case IZQUIERDA: return new Hitbox(getX() - alcance, getY(), alcance, alto);
                    case ABAJO:     return new Hitbox(getX(), getY() + alto, ancho, alcance);
                    default:        return new Hitbox(getX(), getY() - alcance, ancho, alcance);
                }
            }

            public boolean estaDentroDe(int anchoZona, int altoZona) {
                return getX() >= 0 && getY() >= 0 && getX() + ancho <= anchoZona && getY() + alto <= altoZona;
            }

            public boolean estaEnLineaCon(Hitbox otra) {
                boolean mismaFila = getY() < otra.getY() + otra.alto && otra.getY() < getY() + alto;
                boolean mismaColumna = getX() < otra.getX() + otra.ancho && otra.getX() < getX() + ancho;
                return mismaFila || mismaColumna;
            }
        }
```

Ejemplo de la espada: Link en `(96, 240)`, de 48 × 48, mira a la `DERECHA`. `adelante(DERECHA, 32)` es el rectángulo `(144, 240)` de 32 × 48. Un Moblin en `(160, 250)` lo intersecta: recibe el golpe. Con el ataque recargado el alcance es 64 y el rectángulo llega hasta `x = 208`.

Consideraciones:

- ⚠️ `intersecta` usa `<` y no `<=`: dos rectángulos que **solo se tocan** en el borde no chocan. Si se usara `<=`, Link pegado a una roca quedaría "chocando" siempre y no podría despegarse.
- ⚠️ **Caídas al vacío con 1 píxel.** Con el rectángulo completo de Link, alcanza con rozar el borde de una caída para caer. Se siente injusto. Dos arreglos posibles, sin tocar el diagrama: hacer las caídas con un hitbox más chico (por ejemplo 24 × 24 centrado en la celda, en el constructor de `CaidasAlVacio`) o que `CaidasAlVacio.afectar(l)` solo haga caer a Link si el centro de Link está dentro de la caída. **(propuesta)**

7.4 Sprite

Utilidad: el conjunto de imágenes de una entidad, una por **estado** (`"abajo"`, `"arriba"`, `"ataque-derecha"`...), y cuál se está mostrando. Guarda **rutas** (texto), no imágenes: así la lógica no depende de Swing.

Elemento	Qué es	Para qué
`# rutas : Map<String, String>`	Atributo	Estado → ruta de la imagen (por ejemplo `"abajo"` → `"/imagenes/original/link-abajo.png"`)
`# estadoActual : String`	Atributo	El estado que se ve ahora
`+ Sprite()`	Constructor	Mapa vacío; lo llena `FabricaSprites*` con `agregar`
`+ agregar(estado, ruta)`	Comando	Lo llama `FabricaSprites.armar(...)`. El primer estado agregado queda como actual
`+ setEstado(estado)`	Comando	Cambia la imagen. Si el estado no existe, no cambia (evita una imagen vacía)
`+ getRutaActual() : String`	Consulta	Lo que devuelve `Entidad.getRutaImagen()` y dibuja `CeldaGrafica`
`+ getRuta(estado)`, `+ tieneEstado(estado)`	Consultas	

Relaciones: la referencia `Entidad*` (`#sprite 1`). La crea `FabricaSprites*` (dependencia «crea»).

```
        public class Sprite {
            protected Map<String, String> rutas;
            protected String estadoActual;

            public Sprite() { rutas = new HashMap<>(); }

            public void agregar(String estado, String ruta) {
                rutas.put(estado, ruta);
                if (estadoActual == null) estadoActual = estado;
            }

            public void setEstado(String estado) {
                if (tieneEstado(estado)) estadoActual = estado;
            }

            public String getRutaActual() { return rutas.get(estadoActual); }
            public String getRuta(String estado) { return rutas.get(estado); }
            public boolean tieneEstado(String estado) { return rutas.containsKey(estado); }
        }
```

Convención de nombres de estados (propuesta): las cuatro direcciones en minúscula (`"arriba"`, `"abajo"`, `"izquierda"`, `"derecha"`), que es lo que pone `EntidadAnimada.setDireccion(d)` con `d.name().toLowerCase()`, y para Link además `"ataque-" + dirección` y `"escudo-" + dirección`. Las entidades fijas tienen un solo estado, `"normal"`.

---

8. La base de las entidades: Colisionable, Colisionador, EntidadLogica y Entidad*

8.1 Colisionable (interfaz)

Utilidad: "algo con lo que se puede chocar". Es la mitad del **doble despacho** (Visitor) de las colisiones.

Elemento	Para qué
`+ aceptarColision(c : Colisionador)`	El primer despacho. Cada clase concreta lo implementa con una sola línea: `c.colisionarCon(this)`

La realiza `Entidad*`; la implementan de verdad `Obstaculo*`, `PowerUP*`, `Objeto*`, `Link`, `Enemigo*` y `Proyectil`.

8.2 Colisionador (interfaz)

Utilidad: "algo que choca y reacciona". La otra mitad del doble despacho: tiene un `colisionarCon` por cada tipo con el que se puede chocar.

Elemento	Para qué
`+ colisionarCon(l : Link)`	Chocó con Link
`+ colisionarCon(e : Enemigo)`	Chocó con un enemigo (incluye jefes)
`+ colisionarCon(p : Proyectil)`	Chocó con un proyectil
`+ colisionarCon(o : Obstaculo)`	Chocó con un obstáculo
`+ colisionarCon(p : PowerUP)`	Chocó con un power-up
`+ colisionarCon(o : Objeto)`	Chocó con la espada o el escudo tirados en el mapa

La realiza `EntidadAnimada*`, con los seis métodos **vacíos**. Solo `Link` redefine los suyos (regla de la sección 22).

Cómo funciona el doble despacho, con un ejemplo: Link toca una rupia.

```
        // AdministradorColisiones.verificarColisiones(link)
        rupia.aceptarColision(link);            // 1er despacho: Java elige el aceptarColision de PowerUP
        // PowerUP.aceptarColision(c)
        c.colisionarCon(this);                  // 2do despacho: this es un PowerUP, Java elige colisionarCon(PowerUP)
        // Link.colisionarCon(PowerUP p)
        p.serRecogidoPor(this);                 // efecto de la rupia
```

Nadie pregunta "¿esto es una rupia?". El tipo lo resuelve Java al elegir el método sobrecargado.

8.3 EntidadLogica (interfaz)

Utilidad: lo **único** que la vista puede ver de una entidad. Separa las capas.

Elemento	Para qué
`+ getX() : int`, `+ getY() : int`	Dónde dibujar la celda
`+ getRutaImagen() : String`	Qué imagen mostrar
`+ estaEliminada() : boolean`	Si es `true`, la celda se saca del panel

La realiza `Entidad*`. La referencia `CeldaGrafica` (`#entidadObservada 1`). `EntidadJugador` la extiende.

8.4 Entidad* (abstracta)

Utilidad: todo lo que aparece en un sector: obstáculos, power-ups, objetos, Link, enemigos y proyectiles. Tiene lo común: rectángulo, imagen, observadores, a qué juego y a qué sector pertenece, y si fue eliminada.

Hereda / realiza: realiza `Colisionable` y `EntidadLogica`. Subclases directas: `Obstaculo*`, `PowerUP*`, `Objeto*`, `EntidadAnimada*`.

Atributo	Tipo	Para qué	Relación
`# hitbox`	`Hitbox`	Rectángulo en píxeles	◆ `#hitbox 1`
`# sprite`	`Sprite`	Imágenes	→ `#sprite 1`
`# observadores`	`List<Observer>`	Las celdas gráficas que la dibujan	→ `#observadores 0..*`
`# miJuego`	`Juego`	A quién avisar sucesos (puntos, drops, daño)	→ `#miJuego 1`
`# miSector`	`Sector`	En qué sector está. Lo usa `eliminar()` para sacarse solo (cambio 4)	→ `#miSector 1`
`# eliminada`	`boolean`	Ya no está en el juego	

Método	Qué hace	Quién lo llama
`# Entidad(sprite, x, y)`	Guarda el sprite y crea el hitbox de 48 × 48 en `(x, y)`	Constructores de las subclases
`+ setJuego(j)`	Guarda `miJuego`	`FabricaEntidades`, al crear cada entidad
`+ setSector(s)`	Guarda `miSector`	`Sector.agregar(e)` (siempre, sin excepción)
`+ setPosicion(p)`	Mueve el hitbox y notifica	`avanzar`, teletransporte, cambio de sector, trampa que vuelve
`+ registrarObserver(o)`	Agrega una celda	`Juego.activarSector` y `Juego.agregarEntidad`
`+ eliminar()`	`eliminada = true`, se saca de su sector y notifica	Power-ups recogidos, enemigos muertos, proyectiles, suelo roto, pasto cortado
`+ recibirGolpeDeEspada(atacante, danio)`	**Gancho vacío**: por defecto la espada no hace nada	`AdministradorColisiones.golpearEn`. Lo redefinen `Pasto`, `Enemigo*` y `Link`
`+ aceptarColision(c)`	**Abstracto** en `Entidad*` (lo declara `Colisionable`)	`AdministradorColisiones.verificarColisiones`
`+ actualizarPorMundo(c)`	**Gancho vacío**: las entidades fijas no hacen nada en la vuelta de `HiloMundo`	`Sector.actualizarEntidades`. Lo redefine `EntidadAnimada*` y lo vuelve a vaciar `Link`
`# notificarObservers()`	`o.actualizar()` para cada observador	`setPosicion`, `eliminar`, `setDireccion`, cambios de vida
`+ getX()`, `getY()`, `getPosicion()`, `getHitbox()`, `getAncho()`, `getAlto()`	Delegan en el hitbox	Todos
`+ getJuego()`, `+ getSector()`	Devuelven `miJuego` y `miSector`	`AtaquePorProyectil`, `Juego.cargarNivel`, `Link.romperSuelo`
`+ getRutaImagen()`	`sprite.getRutaActual()`	`CeldaGrafica`
`+ estaEliminada()`	`eliminada`	Hilos, colisiones y celdas
`+ getPuntos()`	`0` por defecto	`Juego` (lo redefinen enemigos y power-ups)
`+ bloqueaPaso()`	`false` por defecto	`AdministradorColisiones.puedeMoverA` (lo redefinen `Roca` y `Agua`)
`+ intersecta(otra)`	`hitbox.intersecta(otra.getHitbox())`	

```
        public abstract class Entidad implements Colisionable, EntidadLogica {
            protected Hitbox hitbox;
            protected Sprite sprite;
            protected List<Observer> observadores;
            protected Juego miJuego;
            protected Sector miSector;
            protected boolean eliminada;

            protected Entidad(Sprite sprite, int x, int y) {
                this.sprite = sprite;
                this.hitbox = new Hitbox(x, y, 48, 48);
                this.observadores = new CopyOnWriteArrayList<>();
            }
```

```
    public void setJuego(Juego j) { miJuego = j; }
    public void setSector(Sector s) { miSector = s; }

    public void setPosicion(Posicion p) {
        hitbox.setPosicion(p);
        notificarObservers();
    }

    public void registrarObserver(Observer o) { observadores.add(o); }

    public void eliminar() {                    // cambio 4: se saca sola, sin pasar por Link ni por Juego
        eliminada = true;
        if (miSector != null) miSector.quitar(this);
        notificarObservers();                   // la celda ve estaEliminada() y se borra
    }

    public void recibirGolpeDeEspada(EntidadAnimada atacante, int danio) { }   // gancho vacío
    public abstract void aceptarColision(Colisionador c);
    public void actualizarPorMundo(AdministradorColisiones c) { }              // gancho vacío

    protected void notificarObservers() {
        for (Observer o : observadores) o.actualizar();
    }

    public int getX() { return hitbox.getX(); }
    public int getY() { return hitbox.getY(); }
    public Posicion getPosicion() { return hitbox.getPosicion(); }
    public Hitbox getHitbox() { return hitbox; }
    public Juego getJuego() { return miJuego; }
    public Sector getSector() { return miSector; }
    public int getAncho() { return hitbox.getAncho(); }
    public int getAlto() { return hitbox.getAlto(); }
    public String getRutaImagen() { return sprite.getRutaActual(); }
    public boolean estaEliminada() { return eliminada; }
    public int getPuntos() { return 0; }
    public boolean bloqueaPaso() { return false; }
    public boolean intersecta(Entidad otra) { return hitbox.intersecta(otra.getHitbox()); }
}
```

Consideraciones:

- **Cambio 4 en la práctica.** Una rupia se recoge y hace `eliminar()`: se saca de su propio sector con `miSector`, sin pedirle nada a Link ni a `Juego`. Lo mismo un enemigo que muere, un proyectil que choca o un suelo que se rompe. Para **agregar** algo nuevo (un drop, un proyectil) se sigue usando `miJuego.agregarEntidad(e)`, porque además hay que crearle la celda en la vista.
- `observadores` es una `CopyOnWriteArrayList` (de `java.util.concurrent`, parte de Java): la recorren varios hilos mientras `Juego` agrega celdas, y así no tira `ConcurrentModificationException`. Sigue siendo `List<Observer>`, como dice el extendido.
- `eliminar()` puede llamarse dos veces (por ejemplo, un proyectil que toca a Link y a la vez sale del sector). No pasa nada: `quitar` de algo que ya no está no hace nada. Si se quiere, se puede empezar con `if (eliminada) return;`.
- **Tamaño del hitbox.** El constructor del extendido no recibe tamaño, así que todas nacen de 48 × 48. Si alguna entidad necesita otro tamaño (un proyectil de 24 × 24 o StoneHinox de 96 × 96), lo reemplaza en su constructor: `hitbox = new Hitbox(x, y, 96, 96);`. **(propuesta)**

---

9. Obstáculos

9.1 Obstaculo* (abstracta)

Utilidad: lo que está fijo en el mapa y afecta a Link o le bloquea el paso.

Elemento	Qué hace
`# Obstaculo(sprite, x, y)`	`super(sprite, x, y)`
`+ aceptarColision(c)`	`c.colisionarCon(this)`
`+ afectar(l : Link)`	**Gancho**: qué le hace a Link cuando lo toca. Vacío por defecto

Relaciones: hereda de `Entidad*`; subclases `Roca`, `Agua`, `Pasto`, `CaidasAlVacio`, `SuelosQuebradizo`.

Cómo llega el efecto: `Link.colisionarCon(Obstaculo o)` hace `o.afectar(this)`. Así Link no sabe qué obstáculo es.

9.2 Las cinco subclases

Clase	Símbolo	`bloqueaPaso()`	`afectar(l)`	Otros
`Roca`	`R`	`true`	Nada	Frena a Link, enemigos y proyectiles (los proyectiles desaparecen)
`Agua`	`A`	`true`	Nada	Igual que la roca, con otra imagen
`Pasto`	`P`	`false`	`l.ralentizar()`	`recibirGolpeDeEspada(...)`: `eliminar()` (la espada corta el pasto)
`CaidasAlVacio`	`V`	`false`	`l.caerAlVacio()`	

Clase	Símbolo	`bloqueaPaso()`	`afectar(l)`	Otros
`SuelosQuebradizo`	`Q`	`false`	Si no estaba pisado: `pisado = true` y `l.romperSuelo(this)`	`estaPisado()`

```
public class Roca extends Obstaculo {
    public Roca(Sprite sprite, int x, int y) { super(sprite, x, y); }
    public void afectar(Link l) { }
    public boolean bloqueaPaso() { return true; }
}

public class Pasto extends Obstaculo {
    public Pasto(Sprite sprite, int x, int y) { super(sprite, x, y); }
    public void afectar(Link l) { l.ralentizar(); }
    public void recibirGolpeDeEspada(EntidadAnimada atacante, int danio) { eliminar(); }
}

public class SuelosQuebradizo extends Obstaculo {
    protected boolean pisado;

    public SuelosQuebradizo(Sprite sprite, int x, int y) { super(sprite, x, y); }

    public void afectar(Link l) {
        if (!pisado) {
            pisado = true;
            l.romperSuelo(this);     // Link programa la rotura para dentro de 500 ms
        }
    }

    public boolean estaPisado() { return pisado; }
}
```

Consideraciones:

- `Roca.afectar` y `Agua.afectar` están en el extendido aunque no hagan nada: Link nunca llega a superponerse con ellas porque `puedeMoverA` lo frena antes. Igual se escriben (vacíos) para respetar el diagrama 1 a 1.
- El pasto ralentiza **mientras** Link está encima: cada vuelta de `HiloJugador` que Link toca pasto se llama `ralentizar()`, y el próximo `avanzar` usa la mitad de pasos (ver Link).
- El suelo quebradizo cambia de imagen al pisarse si su sprite tiene el estado `"pisado"` (opcional): `sprite.setEstado("pisado"); notificarObservers();`.

---

10. Power-ups

10.1 PowerUP* (abstracta)

Utilidad: lo que Link recoge al tocarlo y le da un efecto y puntos.

Elemento	Qué hace
`# PowerUP(sprite, x, y)`	`super(sprite, x, y)`
`+ aceptarColision(c)`	`c.colisionarCon(this)`
`+ serRecogidoPor(l : Link)`	En la base: los pasos comunes, `l.powerUpRecogido(this)` y `eliminar()`. Cada subclase lo redefine: aplica **su** efecto y llama `super.serRecogidoPor(l)`
`+ getPuntos() : int`	`300` (lo redefine `Rupia`)

Relaciones: hereda de `Entidad*`; subclases `Rupia`, `Bellota`, `Corazon`, `FragmentoDeCorazon`. `GeneradorDrop` crea `Rupia` y `Corazon` (dependencia «crea»).

```
public abstract class PowerUP extends Entidad {
    protected PowerUP(Sprite sprite, int x, int y) { super(sprite, x, y); }

    public void aceptarColision(Colisionador c) { c.colisionarCon(this); }

    public void serRecogidoPor(Link l) {
        l.powerUpRecogido(this);     // Link avisa a Juego, que le suma getPuntos()
        eliminar();                  // cambio 4: se saca sola de su sector
    }

    public int getPuntos() { return 300; }
}
```

10.2 Las cuatro subclases

Clase	Atributos	`serRecogidoPor(l)`	`getPuntos()`
`Rupia`	`# cantidad : int` (1 a 5, al azar en el constructor)	`l.sumarRupias(cantidad)` y `super`	`10 * cantidad`
`Bellota`	—	`l.setDefensaBase(new DefensaLimitada(1, 3))` y `super`	300 (heredado)
`Corazon`	—	`l.curar(2)` y `super`	300

Clase	Atributos	`serRecogidoPor(l)`	`getPuntos()`
`FragmentoDeCorazon`	—	`l.sumarFragmento()` y `super`	300

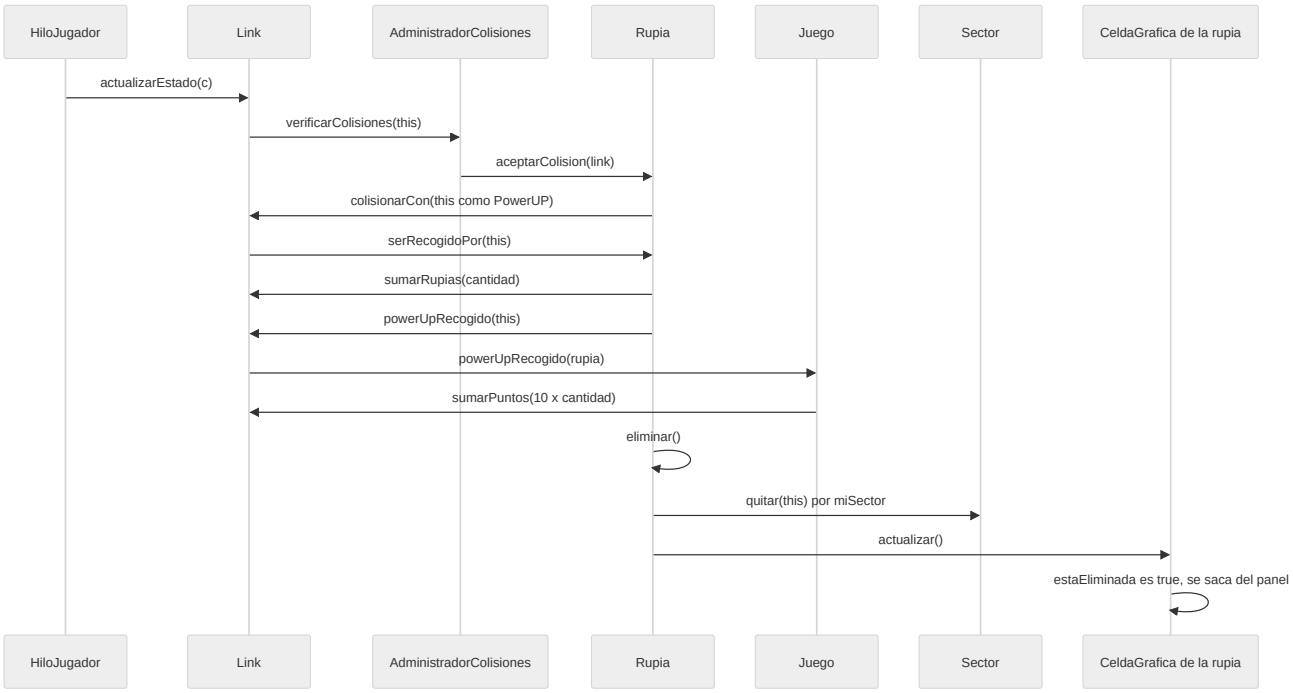
```java
public class Rupia extends PowerUP {
    protected int cantidad;

    public Rupia(Sprite sprite, int x, int y) {
        super(sprite, x, y);
        cantidad = 1 + new Random().nextInt(5);
    }

    public void serRecogidoPor(Link l) {
        l.sumarRupias(cantidad);
        super.serRecogidoPor(l);
    }

    public int getPuntos() { return 10 * cantidad; }
    public int getCantidad() { return cantidad; }
}
```

Diagrama de secuencia: Link junta una rupia (todo pasa en `HiloJugador`):



Consideraciones:

- El orden importa: primero el efecto y los puntos, **después** `eliminar()`. Si se elimina antes, la celda se borra igual, pero conviene que el HUD ya muestre los puntos nuevos cuando desaparece la imagen.
- Los fragmentos los ubica `GeneradorNivel` (4 por nivel, como mucho 1 por sector) y la bellota la crea `Juego.aparecerBellota` (racha de 4 enemigos sin daño). No están en el archivo del nivel.

---

11. Objetos: Objeto*, Espada y Escudo

11.1 Objeto* (abstracta)

Utilidad: lo que Link lleva en el inventario y usa con las teclas. Cada objeto decide qué hace cada tecla: así Link no pregunta "¿tengo la espada?".

Elemento	Qué hace
`# Objeto(sprite, x, y)`	`super(sprite, x, y)`
`+ aceptarColision(c)`	`c.colisionarCon(this)`
`+ serRecogidoPor(l)`	`l.agregarAlInventario(this)` y `eliminar()` (sale del mapa)
`+ atacarCon(l)`, `+ soltarAtaqueCon(l)`	Ganchos vacíos: tecla de ataque apretada y soltada
`+ protegerCon(l)`, `+ soltarProteccionCon(l)`	Ganchos vacíos: tecla del escudo apretada y soltada
`+ getNombre() : String`	Abstracto: lo muestra el HUD

Relaciones: hereda de `Entidad*`; lo agrega `Link` (◇ `#inventario 1..2`) y lo referencia (→ `#objetoEquipado 0..1`).

11.2 Espada y Escudo

Clase	Redefine	Efecto
`Espada` (`X` en el mapa)	`atacarCon(l)` → `l.cargarAtaque()`; `soltarAtaqueCon(l)` → `l.liberarAtaque()`; `getNombre()` → `"Espada"`	Apretar empieza la carga; soltar dispara el golpe (normal o recargado)
`Escudo` (Link empieza con él)	`protegerCon(l)` → `l.activarEscudo()`; `soltarProteccionCon(l)` → `l.desactivarEscudo()`; `getNombre()` → `"Escudo"`	Mientras se mantiene la tecla, Link no recibe daño

```
public class Espada extends Objeto {
    public Espada(Sprite sprite, int x, int y) { super(sprite, x, y); }
    public void atacarCon(Link l) { l.cargarAtaque(); }
    public void soltarAtaqueCon(Link l) { l.liberarAtaque(); }
    public String getNombre() { return "Espada"; }
}
```

Consideraciones:

- Con el escudo equipado, la tecla de ataque no hace nada (el gancho de `Escudo` es vacío), y con la espada equipada la del escudo tampoco. Se cambia de objeto con la tecla de `cambiarObjeto()`. **(a confirmar** con el enunciado: si el escudo se usa siempre, sin equiparlo, `Link.usarEscudo()` puede llamar directamente `activarEscudo()` cuando el escudo está en el inventario.**)**
- El escudo de Link nunca estuvo en un sector, así que su `miSector` es `null`: por eso `eliminar()` pregunta `if (miSector != null)`.

---

12. EntidadAnimada* y AccionProgramada

12.1 EntidadAnimada* (abstracta)

Utilidad: toda entidad que se mueve, ataca o recibe daño: Link, enemigos, jefes y proyectiles. Delega **cómo** se mueve, ataca y se defiende en tres estrategias (Strategy) y ofrece **ganchos** que las subclases redefinen.

Hereda / realiza: hereda de `Entidad*`, realiza `Colisionador`. Subclases: `Link`, `Proyectil`, `Enemigo*`.

Atributo	Tipo	Para qué	Relación
`# movimiento`	`ComportamientoMovimiento`	Cómo se mueve	◇ `#movimiento 1`
`# ataque`	`ComportamientoAtaque`	Cómo ataca	◇ `#ataque 1`
`# defensa`	`ComportamientoDefensa`	Cuánto daño deja pasar	◇ `#defensa 1`
`# direccion`	`Direccion`	Hacia dónde mira	→ `#direccion 1`
`# vida`	`int`	Medios corazones (Link) o golpes (enemigos)	
`# pendientes`	`List<AccionProgramada>`	Acciones para dentro de un rato	◆ `#pendientes 0..*`

Métodos, agrupados:

Grupo	Métodos	Qué hacen
Constructor	`# EntidadAnimada(sprite, x, y, vida)`	`super(sprite, x, y)`, guarda la vida, `direccion = ABAJO` y estrategias por defecto (ver código)
Strategy	`setMovimiento`, `setAtaque`, `setDefensa`	Cambian la estrategia en tiempo de ejecución
Dirección	`setDireccion(d)`, `getDireccion()`	Cambia hacia dónde mira, el estado del sprite y notifica
La vuelta	`actualizarEstado(c)`	Una vuelta completa (ver código). La llaman `HiloJugador` (Link) o `actualizarPorMundo`
	`actualizarPorMundo(c)`	Redefine el gancho de `Entidad*`: `actualizarEstado(c)`
Movimiento	`avanzar(d, pasos)`, `avanzarHacia(destino, pasos)`	Cambian la posición. **No** controlan bloqueos: eso lo hace `AdministradorColisiones.mover` antes
Ganchos de choque	`# alChocarConBloqueo()`, `# alSalirDelSector(d)`	Vacíos. Los llama `AdministradorColisiones.mover`
Gancho de vigilancia	`# vigilarJugador()`	Vacío. Primer paso de la vuelta
Daño	`recibirDanio(danio, origen)`	`synchronized`. Pasa por la defensa y, si el daño pasó, `alSerGolpeado()`
	`aplicarDanio(danio)`	`synchronized`. Resta vida, notifica y, si llega a 0, `morir()`. Lo llama la defensa
	`# morir()`, `# alSerGolpeado()`	Ganchos vacíos
Ganchos de estrategias	`ataqueRealizado()`, `restaurarDefensa()`, `teletransportarse()`, `ataqueTerminado()`	Vacíos. Los llaman las estrategias y los redefine quien los necesita
Acciones programadas	`# programar(ms, accion)`, `- ejecutarPendientes()`, `- sacarVencidas(ahora)`	Esperas sin hilos extra
Visitor	Los seis `colisionarCon(...)`	Vacíos
Consultas	`getVida()`, `quiereAtacar()` (`false`), `soltoElAtaque()` (`false`), `linkEstaEnfrente()`, `direccionHaciaJugador()`	

```
public abstract class EntidadAnimada extends Entidad implements Colisionador {
    protected ComportamientoMovimiento movimiento;
```

```java
protected ComportamientoAtaque ataque;
protected ComportamientoDefensa defensa;
protected Direccion direccion;
protected int vida;
protected List<AccionProgramada> pendientes;

protected EntidadAnimada(Sprite sprite, int x, int y, int vida) {
    super(sprite, x, y);
    this.vida = vida;
    this.direccion = Direccion.ABAJO;
    this.pendientes = new ArrayList<>();
    this.movimiento = new MovimientoInducido((Direccion) null, 0);    // quieta
    this.ataque = new AtaquePorContacto(0);                            // defensa 6: no lastima al tocar
    this.defensa = new DefensaLimitada(DefensaLimitada.SIN_LIMITE, DefensaLimitada.SIEMPRE);
}

public void setMovimiento(ComportamientoMovimiento m) { movimiento = m; }
public void setAtaque(ComportamientoAtaque a) { ataque = a; }
public void setDefensa(ComportamientoDefensa d) { defensa = d; }

public void setDireccion(Direccion d) {
    direccion = d;
    sprite.setEstado(d.name().toLowerCase());
    notificarObservers();
}

public void actualizarEstado(AdministradorColisiones c) {       // UNA vuelta
    if (estaEliminada()) return;
    vigilarJugador();
    movimiento.mover(this, c);
    ataque.atacar(this, c);
    if (!estaEliminada()) c.verificarColisiones(this);
    ejecutarPendientes();
}

public void actualizarPorMundo(AdministradorColisiones c) { actualizarEstado(c); }

public void avanzar(Direccion d, int pasos) {
    setPosicion(getPosicion().desplazada(d, pasos));
}

public void avanzarHacia(Posicion destino, int pasos) {
    int dx = Math.max(-pasos, Math.min(pasos, destino.getX() - getX()));
    int dy = Math.max(-pasos, Math.min(pasos, destino.getY() - getY()));
    setPosicion(new Posicion(getX() + dx, getY() + dy));
}

protected void alChocarConBloqueo() { }
protected void alSalirDelSector(Direccion d) { }
protected void vigilarJugador() { }

public synchronized void recibirDanio(int danio, Direccion origen) {
    if (defensa.recibirDanio(this, danio, origen)) alSerGolpeado();
}

public synchronized void aplicarDanio(int danio) {
    vida -= danio;
    notificarObservers();
    if (vida <= 0) morir();
}

protected void morir() { }
protected void alSerGolpeado() { }
public void ataqueRealizado() { }
public void restaurarDefensa() { }
public void teletransportarse() { }
public void ataqueTerminado() { }

protected synchronized void programar(long ms, Runnable accion) {
    pendientes.add(new AccionProgramada(System.currentTimeMillis() + ms, accion));
}

private void ejecutarPendientes() {
    for (AccionProgramada a : sacarVencidas(System.currentTimeMillis()))
        a.ejecutar();                                  // fuera del lock: la acción puede volver a programar
}

private synchronized List<AccionProgramada> sacarVencidas(long ahora) {
    List<AccionProgramada> vencidas = new ArrayList<>();
    for (AccionProgramada a : pendientes)
        if (a.estaVencida(ahora)) vencidas.add(a);
    pendientes.removeAll(vencidas);
    return vencidas;
}

public void colisionarCon(Link l) { }
public void colisionarCon(Enemigo e) { }
```

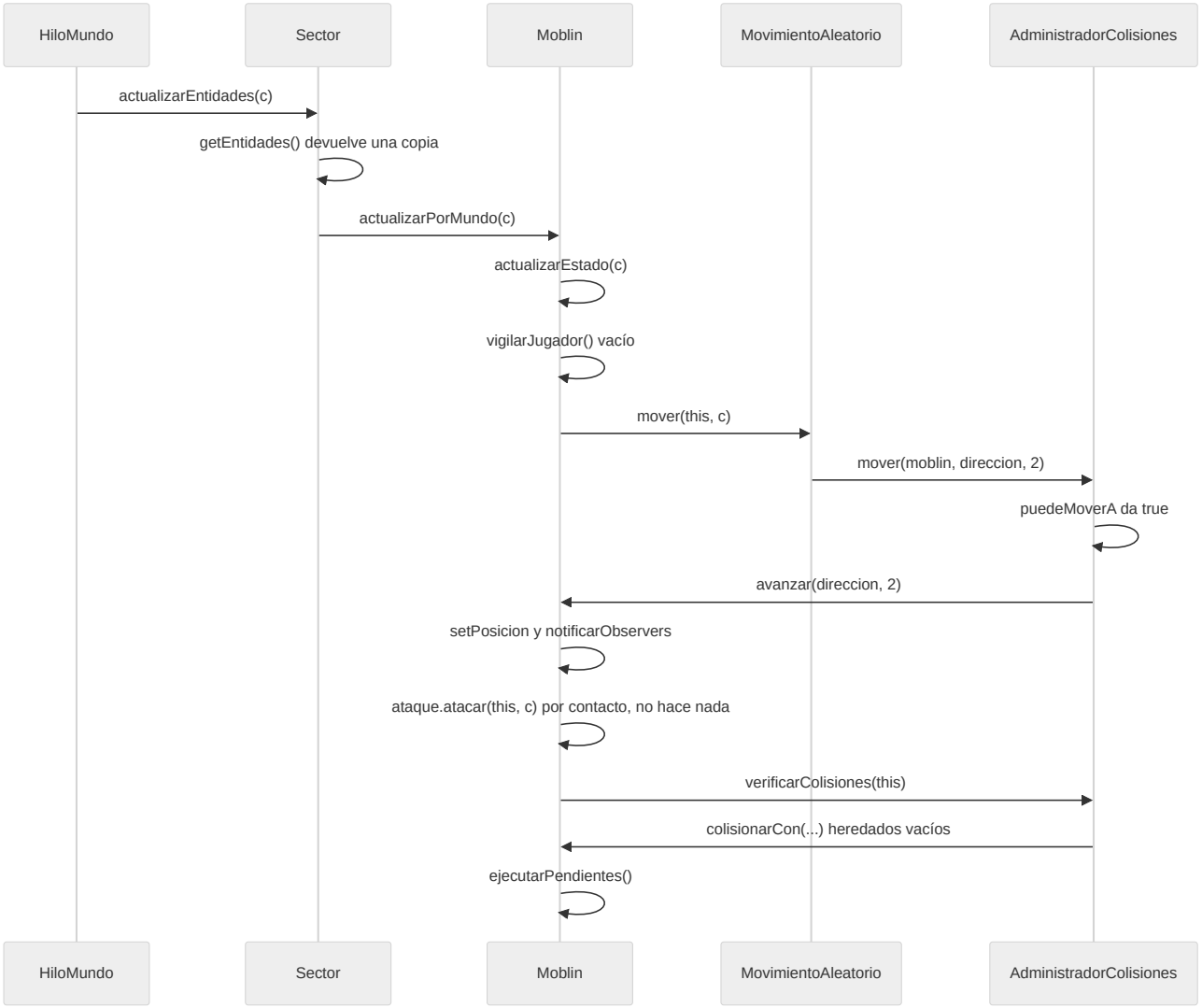
```
        public void colisionarCon(Proyectil p) { }
        public void colisionarCon(Obstaculo o) { }
        public void colisionarCon(PowerUP p) { }
        public void colisionarCon(Objeto o) { }

        public Direccion getDireccion() { return direccion; }
        public int getVida() { return vida; }
        public boolean quiereAtacar() { return false; }
        public boolean soltoElAtaque() { return false; }

        public boolean linkEstaEnfrente() {
            Link link = miJuego.getJugador();
            return getHitbox().estaEnLineaCon(link.getHitbox()) && direccionHaciaJugador() == direccion;
        }

        public Direccion direccionHaciaJugador() {
            return getPosicion().direccionHacia(miJuego.getJugador().getPosicion());
        }
    }
```

Diagrama de secuencia: una vuelta de un Moblin (en `HiloMundo`):



Consideraciones:

- ⚠️ `new MovimientoInducido(null, 0)` **no compila**: hay dos constructores (`Direccion` y `Posicion`) y `null` sirve para los dos. Hay que escribir `(Direccion) null`.
- ⚠️ `ejecutarPendientes` ejecuta las acciones **fuera** del `synchronized`. Si se ejecutaran adentro, una acción que llama a `programar` (StoneHinox lo hace) o que toca otra entidad podría trabar el juego.
- Las estrategias por defecto del constructor son una **(propuesta)** para que ninguna entidad tenga una estrategia `null`. Cada subclase pone las suyas en su constructor.
- `setDireccion` usa `d.name().toLowerCase()` para el estado del sprite. Si una entidad no tiene ese estado (un erizo con una sola imagen), `Sprite.setEstado` lo ignora.
- `avanzarHacia` avanza en x y en y a la vez, sin pasarse del destino. Lo usa `MovimientoInducido` cuando tiene objetivo (rocas de la avalancha, embestida y StoneHinox yendo al centro). **No** pasa por el control de bloqueos: es a propósito, para que las rocas que caen atraviesen las rocas del borde superior.

12.2 AccionProgramada

Utilidad: "hacé esto dentro de `ms` milisegundos" sin crear un hilo. Es una alarma anotada: la cumple el mismo hilo que actualiza a la entidad, cuando revisa sus pendientes.

Elemento	Qué hace
`# instante : long`	Cuándo vence (`System.currentTimeMillis()` + espera)
`# accion : Runnable`	Qué hacer. Casi siempre una *lambda*: `() -> restaurarDefensa()` o `this::volver`
`+ AccionProgramada(instante, accion)`	Constructor. La crea `EntidadAnimada.programar`
`+ estaVencida(ahora) : boolean`	`ahora >= instante`
`+ ejecutar()`	`accion.run()`. **No** crea un `Thread`: es un llamado normal

Relaciones: es parte de `EntidadAnimada*` (◆ `#pendientes 0..*`). No realiza `Runnable`: lo **tiene** como atributo.

Dónde se usa:

Quién	Espera	Para qué
`Link`	1000 ms	Termina la invulnerabilidad después de un golpe (`restaurarDefensa`)
`Link`	500 ms	El suelo quebradizo se rompe y deja una caída
`MoblinConEspadaYEscudo`	800 ms	Vuelve a caminar al azar después de abalanzarse
`MasterStalfos`	800 ms	Termina su invulnerabilidad
`MasterStalfos`	2000 ms	Defensa 6: cambia de fase (espada ↔ contacto)
`StoneHinox`	1500 a 3000 ms	Fin de la embestida, fin de la vulnerabilidad, siguiente ataque

Consideraciones:

- Si la entidad muere, sus pendientes **no se ejecutan** (`actualizarEstado` sale enseguida si está eliminada). Es lo correcto: un Moblin muerto no tiene que volver a caminar.
- Durante la transición de sector los hilos se pausan, así que las acciones se cumplen un poco más tarde. Para un juego no es un problema.

---

13. Link y sus interfaces

13.1 Link

Utilidad: el personaje del jugador. Guarda el estado de la partida que se ve en el HUD (vidas, corazones, puntaje, rupias, objeto) y reacciona a todo lo que toca. Lo mueve **solo** `HiloJugador`.

Hereda / realiza: hereda de `EntidadAnimada*`; realiza `EntidadJugador` (lo que ve el HUD) y `ControlJugador` (lo que manda el teclado).

Atributo	Tipo	Para qué	Relación
`# vidas`	`int`	Vidas que le quedan (empieza con 3)	
`# corazonesMaximos`	`int`	Tope de `vida`, en medios corazones (empieza en 6)	
`# puntaje`	`int`	Puntos de la partida	
`# rupias`	`int`	Rupias juntadas	
`# fragmentos`	`int`	Fragmentos de corazón juntados	
`# racha`	`int`	Enemigos seguidos derrotados sin recibir daño (para la bellota)	
`# inventario`	`List<Objeto>`	Espada y escudo	◇ `#inventario 1..2`
`# objetoEquipado`	`Objeto`	El que usan las teclas	→ `#objetoEquipado 0..1`
`# caminar`	`MovimientoInducido`	Su estrategia de movimiento, con el tipo concreto para cambiarle la dirección sin *cast*	(es `#movimiento`)
`# recargado`	`AtaqueRecargado`	Su estrategia de ataque, con tipo concreto para `cargar()`	(es `#ataque`)
`# defensaBase`	`ComportamientoDefensa`	La defensa a la que vuelve después del escudo o la invulnerabilidad	(es `#defensa`)
`# escudoActivo`	`boolean`	Está usando el escudo	
`# atacando`	`boolean`	Apretó la tecla de ataque (está cargando)	
`# soltoAtaque`	`boolean`	Soltó la tecla: el próximo `atacar` de la estrategia golpea	
`# ralentizado`	`boolean`	Tocó pasto en la vuelta anterior	

⚠️ `atacando`, `soltoAtaque` y `escudoActivo` los escribe el EDT (teclado) y los lee `HiloJugador`: declararlos `volatile`. `MovimientoInducido.direccion` también (ver 15.2).

Métodos, agrupados por quién los llama:

Quién llama	Métodos	Qué hacen
Teclado (`ControlJugador`, en el EDT)	`mover(d)`	`caminar.setDireccion(d)` y `setDireccion(d)`
	`detenerMovimiento()`	`caminar.setDireccion(null)`
	`atacar()`, `soltarAtaque()`	`objetoEquipado.atacarCon(this)` / `soltarAtaqueCon(this)`, si hay objeto equipado
	`usarEscudo()`, `soltarEscudo()`	`objetoEquipado.protegerCon(this)` / `soltarProteccionCon(this)`
	`cambiarObjeto()`	Equipa el siguiente objeto del inventario
Objetos	`cargarAtaque()`	Si no estaba atacando: `atacando = true`, `soltoAtaque = false`, `recargado.cargar()`
	`liberarAtaque()`	`soltoAtaque = true`
	`activarEscudo()`, `desactivarEscudo()`	Cambia a `DefensaPorEscudo(false)` y vuelve a `defensaBase`
	`agregarAlInventario(o)`	Lo agrega y lo equipa
Power-ups	`sumarRupias(n)`, `curar(medios)`, `sumarFragmento()`, `setDefensaBase(d)`	Efectos
	`powerUpRecogido(p)`	`miJuego.powerUpRecogido(p)`
`Juego`	`sumarPuntos(p)`, `reiniciarPuntaje()`, `incrementarRacha()`, `reiniciarRacha()`, `reiniciarVidas()`	Reglas de la partida
	`entrarPorBordeOpuesto(d)`	Lo ubica en el borde por el que entra al sector nuevo
Obstáculos	`ralentizar()`, `caerAlVacio()`, `romperSuelo(s)`	Efectos del pasto, la caída y el suelo
Estrategias	`ataqueRealizado()`	`atacando = false`, `soltoAtaque = false`
	`restaurarDefensa()`	Si no usa el escudo: vuelve a `defensaBase`
	`quiereAtacar()`, `soltoElAtaque()`	Devuelven `atacando` y `soltoAtaque`
`AdministradorColisiones`	`avanzar(d, pasos)`	Redefinido: si está ralentizado avanza la mitad
	`alSalirDelSector(d)`	`miJuego.cambiarDeSector(d)`
	`aceptarColision(c)`	`c.colisionarCon(this)`
	Los cinco `colisionarCon(...)` (todos menos el de `Link`)	Las reacciones de Link (tabla de abajo)
	`recibirGolpeDeEspada(atacante, danio)`	`recibirDanio(danio, atacante.getDireccion())` (espada de MasterStalfos)
Daño	`recibirDanio(danio, origen)`	Redefinido: si el daño pasa, corta la racha, avisa `jugadorDaniado()` y `alSerGolpeado()`
	`# alSerGolpeado()`	Invulnerable 1 s
	`# morir()`	Pierde una vida
`HiloMundo`	`actualizarPorMundo(c)`	**Vacío**: a Link lo mueve solo `HiloJugador`
HUD (`EntidadJugador`)	`getVidas()`, `getCorazones()`, `getCorazonesMaximos()`, `getPuntaje()`, `getRupias()`, `getNombreObjetoEquipado()`	
Otros	`getRacha()`, `getInventario()`, `tieneEscudoActivo()`	

Qué hace Link con cada choque:

Choca con	Método	Efecto
Enemigo o jefe	`colisionarCon(Enemigo e)`	Defensa 6: **vacío**. Lo resuelve el enemigo: `Enemigo.colisionarCon(Link)` → `ataque.ataqueAlContactar(this, l)`
Proyectil	`colisionarCon(Proyectil p)`	`recibirDanio(p.getDanio(), p.getDireccion())` y `p.eliminar()` (el proyectil desaparece aunque Link tenga el escudo)
Obstáculo	`colisionarCon(Obstaculo o)`	`o.afectar(this)`
Power-up	`colisionarCon(PowerUP p)`	`p.serRecogidoPor(this)`
Objeto	`colisionarCon(Objeto o)`	`o.serRecogidoPor(this)`

```
        public class Link extends EntidadAnimada implements EntidadJugador, ControlJugador {
            protected int vidas;
            protected int corazonesMaximos;
            protected int puntaje;
            protected int rupias;
            protected int fragmentos;
            protected int racha;
            protected List<Objeto> inventario;
            protected Objeto objetoEquipado;
            protected MovimientoInducido caminar;
            protected AtaqueRecargado recargado;
            protected ComportamientoDefensa defensaBase;
            protected volatile boolean escudoActivo;
            protected volatile boolean atacando;
```

```
    protected volatile boolean soltoAtaque;
    protected boolean ralentizado;

    public Link(Sprite sprite, int x, int y) {
        super(sprite, x, y, 6);
        vidas = 3;
        corazonesMaximos = 6;
        inventario = new ArrayList<>();
        caminar = new MovimientoInducido((Direccion) null, 4);
        recargado = new AtaqueRecargado(1, 32);
        defensaBase = new DefensaLimitada(DefensaLimitada.SIN_LIMITE, DefensaLimitada.SIEMPRE);
        setMovimiento(caminar);
        setAtaque(recargado);
        setDefensa(defensaBase);
    }

    // ---- ControlJugador (lo llama la GUI, en el EDT) ----
    public void mover(Direccion d) {
        caminar.setDireccion(d);
        setDireccion(d);
    }
    public void detenerMovimiento() { caminar.setDireccion(null); }
    public void atacar() { if (objetoEquipado != null) objetoEquipado.atacarCon(this); }
    public void soltarAtaque() { if (objetoEquipado != null) objetoEquipado.soltarAtaqueCon(this); }
    public void usarEscudo() { if (objetoEquipado != null) objetoEquipado.protegerCon(this); }
    public void soltarEscudo() { if (objetoEquipado != null) objetoEquipado.soltarProteccionCon(this); }
    public void cambiarObjeto() {
        if (inventario.isEmpty() || atacando) return;
        int siguiente = (inventario.indexOf(objetoEquipado) + 1) % inventario.size();
        objetoEquipado = inventario.get(siguiente);
        notificarObservers();                       // el HUD muestra el nombre nuevo
    }

    // ---- Espada y escudo ----
    public void cargarAtaque() {
        if (atacando) return;                       // la tecla repetida no reinicia la carga
        atacando = true;
        soltoAtaque = false;
        recargado.cargar();
    }
    public void liberarAtaque() { soltoAtaque = true; }
    public void ataqueRealizado() {
        atacando = false;
        soltoAtaque = false;
    }
    public boolean quiereAtacar() { return atacando; }
    public boolean soltoElAtaque() { return soltoAtaque; }

    public void activarEscudo() {
        escudoActivo = true;
        setDefensa(new DefensaPorEscudo(false));
    }
    public void desactivarEscudo() {
        escudoActivo = false;
        setDefensa(defensaBase);
    }
    public void setDefensaBase(ComportamientoDefensa d) {
        defensaBase = d;
        if (!escudoActivo) setDefensa(d);
    }
    public void restaurarDefensa() {
        if (!escudoActivo) setDefensa(defensaBase);
    }

    // ---- Movimiento ----
    public void avanzar(Direccion d, int pasos) {
        int reales = ralentizado ? pasos / 2 : pasos;
        ralentizado = false;
        super.avanzar(d, reales);
    }
    public void ralentizar() { ralentizado = true; }
    protected void alSalirDelSector(Direccion d) { miJuego.cambiarDeSector(d); }
    public void actualizarPorMundo(AdministradorColisiones c) { }          // lo mueve HiloJugador

    public void entrarPorBordeOpuesto(Direccion d) {    // d: hacia dónde salió del sector viejo
        switch (d) {
            case DERECHA:   setPosicion(new Posicion(0, getY())); break;
            case IZQUIERDA: setPosicion(new Posicion(Sector.ANCHO - getAncho(), getY())); break;
            case ABAJO:     setPosicion(new Posicion(getX(), 0)); break;
            default:        setPosicion(new Posicion(getX(), Sector.ALTO - getAlto())); break;
        }
    }

    // ---- Colisiones (Visitor) ----
    public void aceptarColision(Colisionador c) { c.colisionarCon(this); }
    public void colisionarCon(Enemigo e) { }          // defensa 6: vacío; el contacto lo resuelve el enemigo (su estrategia)
    public void colisionarCon(Proyectil p) {
```

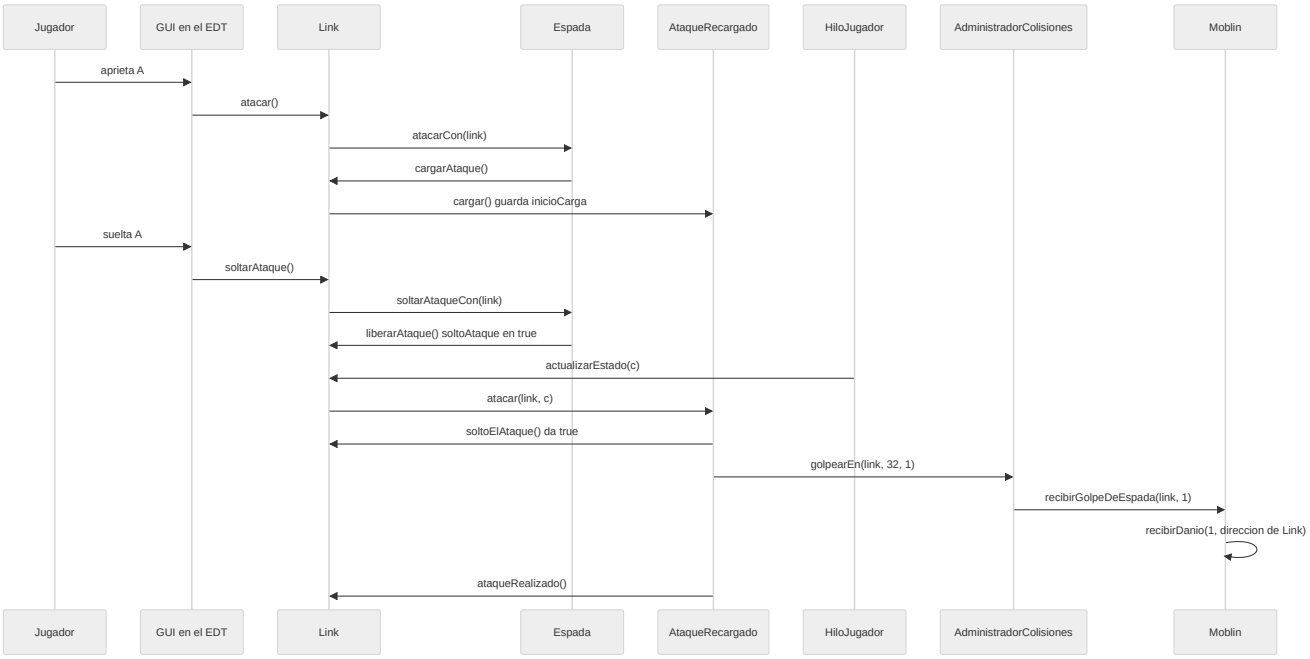
```
            recibirDanio(p.getDanio(), p.getDireccion());
            p.eliminar();
        }
        public void colisionarCon(Obstaculo o) { o.afectar(this); }
        public void colisionarCon(PowerUP p) { p.serRecogidoPor(this); }
        public void colisionarCon(Objeto o) { o.serRecogidoPor(this); }
        public void recibirGolpeDeEspada(EntidadAnimada atacante, int danio) {
            recibirDanio(danio, atacante.getDireccion());
        }

        // ---- Daño, vidas y obstáculos ----
        public synchronized void recibirDanio(int danio, Direccion origen) {
            if (defensa.recibirDanio(this, danio, origen)) {
                reiniciarRacha();
                miJuego.jugadorDaniado();                   // NoHit termina la partida acá
                alSerGolpeado();
            }
        }
        protected void alSerGolpeado() {
            setDefensa(new DefensaLimitada(0, DefensaLimitada.SIEMPRE));   // invulnerable
            programar(1000, this::restaurarDefensa);
        }
        protected void morir() {
            vidas--;
            if (vidas <= 0) {
                miJuego.jugadorSinVidas();
            } else {
                vida = corazonesMaximos;
                notificarObservers();
            }
        }
        public void caerAlVacio() {
            vidas--;
            vida = corazonesMaximos;
            reiniciarRacha();
            if (vidas <= 0) miJuego.jugadorSinVidas();
            else miJuego.jugadorCayoAlVacio();              // Juego reinicia el nivel
        }
        public void romperSuelo(SuelosQuebradizo s) {
            programar(500, () -> {
                Entidad caida = miJuego.getFabricaEntidades().crearCaidasAlVacio(s.getX(), s.getY());
                s.eliminar();
                miJuego.agregarEntidad(caida);
            });
        }

        // ---- Estado de la partida ----
        public void agregarAlInventario(Objeto o) {
            inventario.add(o);
            objetoEquipado = o;
            notificarObservers();
        }
        public void sumarPuntos(int p) { puntaje += p; notificarObservers(); }
        public void reiniciarPuntaje() { puntaje = 0; }
        public void sumarRupias(int n) { rupias += n; notificarObservers(); }
        public void curar(int medios) { vida = Math.min(vida + medios, corazonesMaximos); notificarObservers(); }
        public void sumarFragmento() {
            fragmentos++;
            if (fragmentos % 4 == 0) {                      // cada 4: +1 corazón máximo
                corazonesMaximos += 2;
                vida = corazonesMaximos;
            }
            notificarObservers();
        }
        public void incrementarRacha() { racha++; }
        public void reiniciarRracha() { racha = 0; }
        public void reiniciarVidas() { vidas = 3; vida = corazonesMaximos; }
        public void powerUpRecogido(PowerUP p) { miJuego.powerUpRecogido(p); }

        // ---- Consultas (EntidadJugador y otras) ----
        public int getVidas() { return vidas; }
        public int getCorazones() { return vida; }                  // en medios corazones
        public int getCorazonesMaximos() { return corazonesMaximos; }
        public int getPuntaje() { return puntaje; }
        public int getRupias() { return rupias; }
        public int getRacha() { return racha; }
        public String getNombreObjetoEquipado() {
            return objetoEquipado == null ? "Ninguno" : objetoEquipado.getNombre();
        }
        public List<Objeto> getInventario() { return inventario; }
        public boolean tieneEscudoActivo() { return escudoActivo; }
    }
```

Diagrama de secuencia: Link ataca con la espada (la tecla en el EDT, el golpe en `HiloJugador`):



Consideraciones:

- **Vida en medios corazones.** `vida = 6` son 3 corazones. El HUD muestra `getCorazones() / 2.0` o dibuja corazones enteros y medios.
- **`morir()` no termina el juego si quedan vidas:** Link sigue donde está con los corazones llenos. Si el equipo prefiere que al perder una vida se reinicie el nivel (como en la caída), `morir()` llama `miJuego.jugadorCayoAlVacio()` en lugar de rellenar la vida. **(a confirmar)**
- **Invulnerabilidad y escudo.** Si Link levanta y baja el escudo durante el segundo de invulnerabilidad, vuelve a la defensa base antes de tiempo. Es un detalle menor y se acepta.
- **Bellota.** Después de la bellota, `defensaBase` es la `DefensaLimitada(1, 3)`. Cuando gasta los 3 golpes, esa misma defensa pasa sola a "sin límite" (lo hace `DefensaLimitada`), así que Link no tiene que hacer nada.
- ⚠️ **Suelo quebradizo y cambio de sector.** Si Link sale del sector antes de los 500 ms, `agregarEntidad` pondría la caída en el sector **nuevo**. Para evitarlo sin tocar el diagrama, la acción puede preguntar con el cambio 4: `if (s.getSector() == getSector()) miJuego.agregarEntidad(caida); else s.getSector().agregar(caida);` (la caída se ve cuando Link vuelve a entrar, porque `activarSector` crea las celdas de todo el sector).
- **La caída al vacío corre en `HiloJugador`**, en medio de `verificarColisiones`. `Juego.jugadorCayoAlVacio()` recarga el nivel en ese mismo llamado. Para que el resto del recorrido no compare a Link con entidades del sector viejo, `verificarColisiones` corta el recorrido si Link cambió de sector (ver 16.5).
- **`cambiarObjeto()` sin espada:** al empezar el inventario solo tiene el escudo, así que el cambio no hace nada.
- **`reiniciarPuntaje()` y `reiniciarVidas()`** sirven si se reutiliza el mismo Link en otra partida. Con el diseño actual `Juego.iniciar` crea un Link nuevo, pero el extendido los tiene y se escriben.

13.2 ControlJugador (interfaz)

Utilidad: las siete acciones del teclado. La `GUI` las manda directamente a Link sin conocer la clase `Link`.

Método	Tecla sugerida (propuesta)
`+ mover(d : Direccion)`	Flechas (al apretar)
`+ detenerMovimiento()`	Flechas (al soltar)
`+ atacar()` / `+ soltarAtaque()`	A (apretar / soltar)
`+ usarEscudo()` / `+ soltarEscudo()`	S (apretar / soltar)
`+ cambiarObjeto()`	D o Espacio

Relaciones: la realiza `Link`; la referencia `GUI` (`#controlJugador 1`). La devuelve `ControladorJuego.getControlJugador()`.

⚠️ Los métodos corren en el **EDT**: tienen que ser cortos y **no** `synchronized`. Solo anotan la intención (`volatile`); el trabajo lo hace `HiloJugador` en su próxima vuelta.

13.3 EntidadJugador (interfaz)

Utilidad: lo que el HUD puede leer de Link. Extiende `EntidadLogica` (posición, imagen, eliminada).

Método	Qué muestra el HUD
`+ getVidas()`	Vidas
`+ getCorazones()` / `+ getCorazonesMaximos()`	Corazones actuales y máximos (en medios)
`+ getPuntaje()`	Puntaje
`+ getRupias()`	Rupias
`+ getNombreObjetoEquipado()`	`"Espada"`, `"Escudo"` o `"Ninguno"`

Relaciones: la realiza `Link`; la referencia `CeldaJugador` (`#jugadorObservado 1`). Es lo que cumple el requisito de la defensa 6 "que el estado del personaje principal se vea reflejado en todo momento".

---

14. Proyectiles, enemigos y jefes

14.1 Proyectil

Utilidad: las rocas del Octorok, la magia del Wizzrobe y las rocas grandes de StoneHinox. Es una entidad animada más: la mueve `HiloMundo`.

Elemento	Qué hace
`# danio : int`	Daño a Link, en medios (1 o 2)
`# destino : Posicion`	Solo para las rocas de la avalancha: dónde caen. `null` en los disparos rectos
`+ Proyectil(sprite, x, y, d, danio)`	Vida 1, `setDireccion(d)`, `MovimientoInducido(d, 6)`, defensa con tope 0
`+ setDestino(p)`	Guarda el destino y cambia a `MovimientoInducido(p, 6)`
`+ aceptarColision(c)`	`c.colisionarCon(this)`
`# alChocarConBloqueo()`	Chocó con roca o agua: `desaparecer()`
`# alSalirDelSector(d)`	Salió de la pantalla: `desaparecer()`
`# morir()`	`desaparecer()`
`- desaparecer()`	`eliminar()`
`+ getDanio()`	

```
        public class Proyectil extends EntidadAnimada {
            protected int danio;
            protected Posicion destino;

            public Proyectil(Sprite sprite, int x, int y, Direccion d, int danio) {
                super(sprite, x, y, 1);
                this.danio = danio;
                setDireccion(d);
                setMovimiento(new MovimientoInducido(d, 6));
                setDefensa(new DefensaLimitada(0, DefensaLimitada.SIEMPRE));
            }

            public void setDestino(Posicion p) {
                destino = p;
                setMovimiento(new MovimientoInducido(p, 6));
            }

            public void aceptarColision(Colisionador c) { c.colisionarCon(this); }
            protected void alChocarConBloqueo() { desaparecer(); }
            protected void alSalirDelSector(Direccion d) { desaparecer(); }
            protected void morir() { desaparecer(); }
            private void desaparecer() { eliminar(); }
            public int getDanio() { return danio; }
        }
```

Consideraciones:

- **Quién saca el proyectil cuando toca a Link:** Link, con `p.eliminar()` (`desaparecer()` es privado). El proyectil no redefine ningún `colisionarCon`: atraviesa a los enemigos sin hacerles nada.
- ⚠️ **Las rocas de la avalancha y su destino.** `MovimientoInducido` con objetivo usa `avanzarHacia`, que no pasa por `AdministradorColisiones.mover`, así que nadie llama `alChocarConBloqueo()` al llegar. Para que la roca desaparezca al tocar el piso hay dos caminos: (a) que `MovimientoInducido`, al llegar al objetivo, deje de mover y la roca quede quieta hasta que la saque Link al tocarla o el próximo ataque; (b) que `Proyectil` redefina `avanzarHacia(destino, pasos)`: `super.avanzarHacia(...)` y, si llegó, `desaparecer()`. La (b) es la más clara, pero agrega una redefinición que hay que dibujar en el extendido. **(a confirmar)**
- El `danio` de las rocas grandes es 2 (1 corazón) y el de las comunes 1. Lo decide `FabricaEntidades`.

14.2 Enemigo* (abstracta)

Utilidad: lo común a todos los enemigos y jefes: qué pasa cuando tocan a Link, puntos, cómo recibe la espada y qué pasa al morir.

Elemento	Qué hace
`# Enemigo(sprite, x, y, vida)`	Defensa 6: sin `danioContacto` (el daño al tocar lo guarda la estrategia)
`+ aceptarColision(c)`	`c.colisionarCon(this)`
`+ colisionarCon(l : Link)`	Defensa 6: `ataque.ataqueAlContactar(this, l)`. La estrategia decide qué pasa
`+ recibirGolpeDeEspada(atacante, danio)`	`recibirDanio(danio, atacante.getDireccion())` (pasa por su defensa)
`# morir()`	`eliminar()` y `miJuego.enemigoDerrotado(this)` (sin drop)
`# vigilarJugador()`	Gancho vacío. Lo redefinen la trampa y el Moblin con escudo **(propuesta)**
`+ getPuntos()`	Abstracto o `0`: cada enemigo devuelve sus puntos

```
public abstract class Enemigo extends EntidadAnimada {
    protected Enemigo(Sprite sprite, int x, int y, int vida) {
        super(sprite, x, y, vida);
    }

    public void aceptarColision(Colisionador c) { c.colisionarCon(this); }

    public void colisionarCon(Link l) { ataque.ataqueAlContactar(this, l); }   // defensa 6

    public void recibirGolpeDeEspada(EntidadAnimada atacante, int danio) {
        recibirDanio(danio, atacante.getDireccion());
    }

    protected void morir() {
        eliminar();
        miJuego.enemigoDerrotado(this);
    }

    protected void vigilarJugador() { }
    public abstract int getPuntos();
}
```

Consideraciones:

- **Defensa 6:** `Enemigo*` **redefine** `colisionarCon(Link)` **y** `Link.colisionarCon(Enemigo)` **queda vacío** (se mantiene porque lo define `Colisionador`). El choque lo resuelve `HiloMundo` en la vuelta del enemigo, que llama a `verificarColisiones` aunque el enemigo no se mueva (sirve para el erizo y la trampa quietos). Como del lado de Link no se hace nada, el daño se aplica una sola vez.
- `vigilarJugador()` **en el extendido.** Está en `EntidadAnimada*` y en `Enemigo*`, pero **no** en `TrampaDeCuchillas` ni en `MoblinConEspadaYEscudo`, que son los que lo usan. La propuesta de esta guía es que esas dos clases lo redefinan, y agregarlo en el extendido de las dos. La alternativa sin tocar el diagrama es escribir en `Enemigo*` una versión común, pero termina preguntando qué enemigo es. **(a confirmar)**
- ⚠️ La espada de MasterStalfos llama `golpearEn`, que golpea a **todo** lo que está adelante, también a otros enemigos. En el sector del jefe no hay otros, así que no pasa nada, pero conviene saberlo.

14.3 EnemigoConDrop* y GeneradorDrop

Utilidad: enemigos que, al morir, pueden soltar una rupia o un corazón.

Elemento	Qué hace
`# EnemigoConDrop(sprite, x, y, vida)`	Defensa 6: sin `danioContacto`
`# morir()`	`super.morir()` y después `soltarDrop()`
`# soltarDrop()`	Con probabilidad `getProbabilidadDrop()`: `miJuego.agregarEntidad(generarDrop())`
`+ generarDrop() : PowerUP`	De `GeneradorDrop`: 50% rupia, 50% corazón, en la posición del enemigo **(propuesta)**
`# getProbabilidadDrop() : double`	Abstracto: cada enemigo devuelve la suya (0.2, 0.4, 0.5, 1.0)

`GeneradorDrop` **(interfaz):** `+ generarDrop() : PowerUP`. La realiza `EnemigoConDrop*` (Factory Method). Crea `Rupia` y `Corazon` (dependencia «crea»).

```
public abstract class EnemigoConDrop extends Enemigo implements GeneradorDrop {
    protected EnemigoConDrop(Sprite sprite, int x, int y, int vida) {
        super(sprite, x, y, vida);
    }

    protected void morir() {
        super.morir();
        soltarDrop();
    }

    protected void soltarDrop() {
        if (Math.random() < getProbabilidadDrop())
            miJuego.agregarEntidad(generarDrop());
    }

    public PowerUP generarDrop() {
        FabricaEntidades f = miJuego.getFabricaEntidades();
        return Math.random() < 0.5 ? f.crearRupia(getX(), getY()) : f.crearCorazon(getX(), getY());
    }

    protected abstract double getProbabilidadDrop();
}
```

14.4 GeneradorProyectil (interfaz)

`+ generarProyectil() : Proyectil`. La realizan `Octorok`, `Wizzrobe` y `StoneHinox`. La usa `AtaquePorProyectil` (`#generadorProyectil 1`): la estrategia sabe **cuándo** disparar y el enemigo decide **qué** sale.

Quién	Qué devuelve
`Octorok`	`fabrica.crearProyectil(getX(), getY(), getDireccion())`: recto hacia donde mira
`Wizzrobe`	`fabrica.crearProyectil(getX(), getY(), direccionHaciaJugador())`: hacia Link

Quién	Qué devuelve
`StoneHinox`	`fabrica.crearRocaGrande(getX(), getY(), direccionHaciaJugador())`: roca de 1 corazón

14.5 Los seis enemigos comunes

Clase	Constructor (`super(...)`)	Movimiento	Ataque	Defensa	Redefine
`Moblin`	`(sprite, x, y, 2)`	`MovimientoAleatorio(2, 1500)`	`AtaquePorContacto(1)`	Sin límite	`getPuntos()` 100, `getProbabilidadDro` 0.2
`MoblinConEspadaYEscudo`	`super(sprite, x, y)` (de `Moblin`)	Hereda el aleatorio; se abalanza con `MovimientoInducido(d, 6)`	Hereda	`DefensaPorEscudo(true)`	300, 0.5, `vigilarJugador()` **(propuesta)**
`ErizoDeMar`	`(sprite, x, y, 1)`	`MovimientoInducido((Direccion) null, 0)` (quieto)	`AtaquePorContacto(1)`	Sin límite	50, 0.2
`Octorok`	`(sprite, x, y, 1)`	`MovimientoAleatorio(2, 1500)`	`AtaquePorProyectil(this, 1, 2000)`	Sin límite	150, 0.4, `generarProyectil()`
`Wizzrobe`	`(sprite, x, y, 2)`	`MovimientoTeletransporte(3000)`	`AtaquePorProyectil(this, 1, 3000)`	Sin límite, con tope 0 después de un golpe	500, 1.0, `generarProyectil()` `teletransportarse()` `alSerGolpeado()`
`TrampaDeCuchillas`	`Enemigo(sprite, x, y, 1)`	`MovimientoInducido((Direccion) null, 8)` (quieta)	`AtaquePorContacto(1)`	Tope 0 (indestructible)	`getPuntos()` 0, `alChocarConBloqueo` `volver()`, `vigilarJugador()` **(propuesta)**

```
public class Moblin extends EnemigoConDrop {
    public Moblin(Sprite sprite, int x, int y) {
        super(sprite, x, y, 2);
        setMovimiento(new MovimientoAleatorio(2, 1500));
        setAtaque(new AtaquePorContacto(1));          // defensa 6: ½ corazón al tocar
    }
    public int getPuntos() { return 100; }
    protected double getProbabilidadDrop() { return 0.2; }
}

public class MoblinConEspadaYEscudo extends Moblin {
    public MoblinConEspadaYEscudo(Sprite sprite, int x, int y) {
        super(sprite, x, y);
        setDefensa(new DefensaPorEscudo(true));       // bloquea la espada de frente
    }
    protected void vigilarJugador() {                  // (propuesta) se abalanza si ve a Link de frente
        if (linkEstaEnfrente()) {
            setMovimiento(new MovimientoInducido(getDireccion(), 6));
            programar(800, () -> setMovimiento(new MovimientoAleatorio(2, 1500)));
        }
    }
    public int getPuntos() { return 300; }
    protected double getProbabilidadDrop() { return 0.5; }
}

public class Wizzrobe extends EnemigoConDrop implements GeneradorProyectil {
    public Wizzrobe(Sprite sprite, int x, int y) {
        super(sprite, x, y, 2);
        setMovimiento(new MovimientoTeletransporte(3000));
        setAtaque(new AtaquePorProyectil(this, 1, 3000));
    }
    public void teletransportarse() {                  // lo llama MovimientoTeletransporte
        setDefensa(new DefensaLimitada(DefensaLimitada.SIN_LIMITE, DefensaLimitada.SIEMPRE));
        setDireccion(direccionHaciaJugador());         // aparece mirando a Link
    }
    protected void alSerGolpeado() {                   // invulnerable hasta el próximo salto
        setDefensa(new DefensaLimitada(0, DefensaLimitada.SIEMPRE));
    }
    public Proyectil generarProyectil() {
        return miJuego.getFabricaEntidades().crearProyectil(getX(), getY(), direccionHaciaJugador());
    }
    public int getPuntos() { return 500; }
    protected double getProbabilidadDrop() { return 1.0; }
}

public class TrampaDeCuchillas extends Enemigo {
    protected Posicion posicionOriginal;

    public TrampaDeCuchillas(Sprite sprite, int x, int y) {
        super(sprite, x, y, 1);
        posicionOriginal = new Posicion(x, y);
        setMovimiento(new MovimientoInducido((Direccion) null, 8));
        setAtaque(new AtaquePorContacto(1));
```

```
            setDefensa(new DefensaLimitada(0, DefensaLimitada.SIEMPRE));
        }
        protected void vigilarJugador() {                  // (propuesta)
            boolean enReposo = getX() == posicionOriginal.getX() && getY() == posicionOriginal.getY();
            if (enReposo && getHitbox().estaEnLineaCon(miJuego.getJugador().getHitbox())) {
                Direccion d = direccionHaciaJugador();
                setDireccion(d);
                setMovimiento(new MovimientoInducido(d, 8));      // se abalanza
            }
        }
        protected void alChocarConBloqueo() { volver(); }
        private void volver() {
            setPosicion(posicionOriginal);
            setMovimiento(new MovimientoInducido((Direccion) null, 8));   // quieta otra vez
        }
        public int getPuntos() { return 0; }
    }
```

Consideraciones:

- **Moblin con escudo:** mientras Link sigue enfrente, cada vuelta vuelve a crear el movimiento y a programar la vuelta al aleatorio. Funciona (vuelve a caminar 800 ms después de que Link sale de la línea) y no necesita atributos nuevos; el costo es que agrega una acción pendiente por vuelta mientras ve a Link, que se van cumpliendo.
- **Trampa:** solo se dispara si está en su lugar (`enReposo`). Si no, una trampa que ya se abalanzó cambiaría de rumbo en el aire. ⚠️ Si apunta a un **paso** del borde (un `.` en el borde), puede salir del sector sin chocar: hay que ubicarlas en el archivo donde su línea termine en una roca.
- **Octorok** dispara cuando Link está enfrente (lo controla `AtaquePorProyectil.puedeAtacar`). Como su movimiento es al azar, a veces mira a Link y dispara; es el comportamiento esperado.
- **Wizzrobe:** después de un golpe queda invulnerable hasta su próximo teletransporte (3 s). Con 2 golpes de vida, hay que pegarle en dos apariciones distintas.

14.6 Jefe*, MasterStalfos y StoneHinox

*Jefe (abstracta):**

Elemento	Qué hace
`# Jefe(sprite, x, y, vida)`	`super(sprite, x, y, vida)` (defensa 6: el daño al tocar lo pone la estrategia)
`# morir()`	`eliminar()` y `miJuego.jefeDerrotado(this)`. **No** llama `super.morir()` (eso avisaría `enemigoDerrotado`)

Relaciones: hereda de `Enemigo*`; lo referencia `Nivel` (`#jefes 1..*`). No suelta drop.

MasterStalfos (nivel 1, `S`):

Elemento	Valor o efecto
`- espada : ComportamientoAtaque`	Defensa 6: `AtaqueEspada(2, 40)`. Espadazo de 1 corazón; no lastima al tocar
`- contacto : ComportamientoAtaque`	Defensa 6: `AtaquePorContacto(1)`. ½ corazón al tocarlo; no da espadazos
`- DURACION_FASE : long = 2000`	Defensa 6: milisegundos de cada fase (estática)
Constructor	`super(sprite, x, y, 5)`; `MovimientoAleatorio(3, 1200)`; crea `espada` y `contacto`; defensa sin límite; `faseEspada()`
`- faseEspada()`	Defensa 6: `setAtaque(espada)` y `programar(DURACION_FASE, this::faseContacto)`
`- faseContacto()`	Defensa 6: `setAtaque(contacto)` y `programar(DURACION_FASE, this::faseEspada)`
`+ quiereAtacar()`	`getPosicion().distancia(link.getPosicion()) < 80 && linkEstaEnfrente()`
`# alSerGolpeado()`	Tope 0 y `programar(800, ...)` para volver a sin límite
`+ getPuntos()`	5000

```
    public class MasterStalfos extends Jefe {
        private ComportamientoAtaque espada;
        private ComportamientoAtaque contacto;
        private static final long DURACION_FASE = 2000;

        public MasterStalfos(Sprite sprite, int x, int y) {
            super(sprite, x, y, 5);
            setMovimiento(new MovimientoAleatorio(3, 1200));
            espada = new AtaqueEspada(2, 40);
            contacto = new AtaquePorContacto(1);
            faseEspada();                                  // empieza con la espada
        }

        private void faseEspada() {                        // Strategy en tiempo de ejecución
            setAtaque(espada);
            programar(DURACION_FASE, this::faseContacto);
        }

        private void faseContacto() {
            setAtaque(contacto);
            programar(DURACION_FASE, this::faseEspada);
        }

        public boolean quiereAtacar() {
```

```
            Link link = miJuego.getJugador();
            return getPosicion().distancia(link.getPosicion()) < 80 && linkEstaEnfrente();
        }

        protected void alSerGolpeado() {
            setDefensa(new DefensaLimitada(0, DefensaLimitada.SIEMPRE));
            programar(800, () -> setDefensa(new DefensaLimitada(DefensaLimitada.SIN_LIMITE, DefensaLimitada.SIEMPRE)));
        }

        public int getPuntos() { return 5000; }
    }
```

Consideraciones (defensa 6):

- Tiene un solo `ataque`: en la fase espada, tocarlo no lastima, y en la fase contacto no da espadazos.
- Las dos fases y la invulnerabilidad usan `programar`, así que las acciones las ejecuta `HiloMundo` en la vuelta del jefe: `setAtaque` no compite con otro hilo.
- Los 2 s de cada fase son un valor de ejemplo **(a confirmar)**.

StoneHinox (nivel 2, `H`):

Atributo	Para qué
`- embestida : ComportamientoAtaque`	`AtaquePorContacto(2)`: 1 corazón al tocar, solo mientras embiste (defensa 6)
`- lanzarRocas : ComportamientoAtaque`	`AtaquePorProyectil(this, 3, 2000)`
`- avalancha : ComportamientoAtaque`	`AtaqueAvalancha(this)`
`- caminar : ComportamientoMovimiento`	`MovimientoAleatorio(2, 1500)`
`- azar : Random`	Elige el siguiente ataque

Método	Qué hace
`+ StoneHinox(sprite, x, y)`	`super(sprite, x, y, 10)`, crea las estrategias, empieza caminando con `lanzarRocas`, tope 0 y `programar(3000, this::siguienteAtaque)`
`- siguienteAtaque()`	Elige al azar uno de los tres y lo prepara (ver código)
`+ embestidaTerminada()`	Queda vulnerable 1 s y después pasa al siguiente ataque
`+ ataqueTerminado()`	Lo llama `AtaqueAvalancha.alTerminar`: `siguienteAtaque()`
`+ generarProyectil()`	Roca grande hacia Link
`+ getPuntos()`	5000

```
    public class StoneHinox extends Jefe implements GeneradorProyectil {
        private ComportamientoAtaque embestida;
        private ComportamientoAtaque lanzarRocas;
        private ComportamientoAtaque avalancha;
        private ComportamientoMovimiento caminar;
        private Random azar;

        public StoneHinox(Sprite sprite, int x, int y) {
            super(sprite, x, y, 10);
            azar = new Random();
            embestida = new AtaquePorContacto(2);            // defensa 6: 1 corazón al tocar
            lanzarRocas = new AtaquePorProyectil(this, 3, 2000);
            avalancha = new AtaqueAvalancha(this);
            caminar = new MovimientoAleatorio(2, 1500);
            setMovimiento(caminar);
            setAtaque(lanzarRocas);
            setDefensa(new DefensaLimitada(0, DefensaLimitada.SIEMPRE));   // solo vulnerable tras embestir
            programar(3000, this::siguienteAtaque);
        }

        private void siguienteAtaque() {
            switch (azar.nextInt(3)) {
                case 0:
                    setMovimiento(new MovimientoInducido(miJuego.getJugador().getPosicion(), 6));
                    setAtaque(embestida);
                    programar(1500, this::embestidaTerminada);
                    break;
                case 1:
                    setMovimiento(caminar);
                    setAtaque(lanzarRocas);
                    programar(3000, this::siguienteAtaque);
                    break;
                default:
                    Posicion c = getSector().getCentro();            // cambio 4: conoce su sector
                    Posicion objetivo = new Posicion(c.getX() - getAncho() / 2, c.getY() - getAlto() / 2);
                    setMovimiento(new MovimientoInducido(objetivo, 3));
                    setAtaque(avalancha);                             // espera a llegar al centro
                    break;
            }
        }

        public void embestidaTerminada() {
```

```
                setMovimiento(caminar);
                setDefensa(new DefensaLimitada(DefensaLimitada.SIN_LIMITE, DefensaLimitada.SIEMPRE));
                programar(1000, () -> {
                    setDefensa(new DefensaLimitada(0, DefensaLimitada.SIEMPRE));
                    siguienteAtaque();
                });
            }

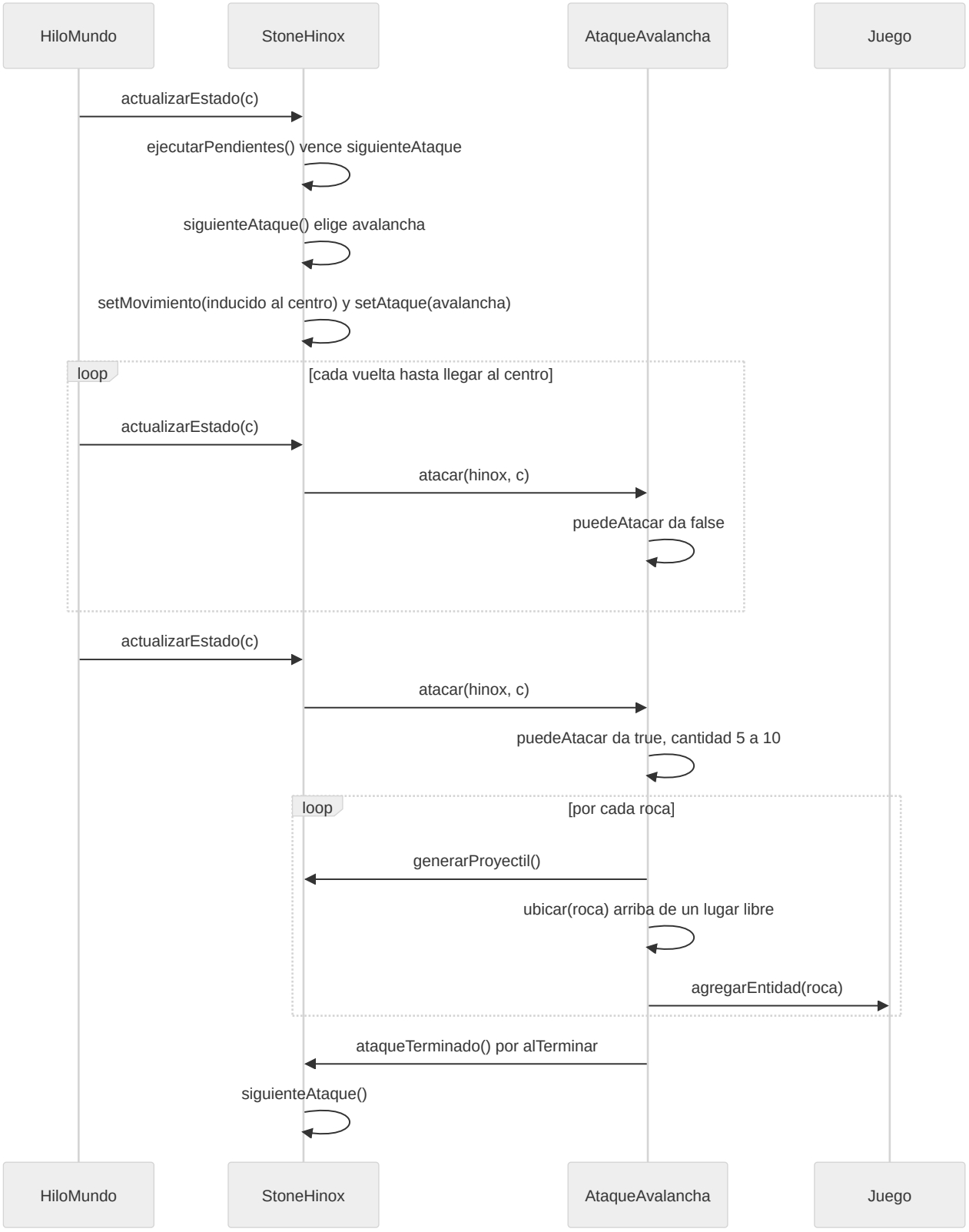
            public void ataqueTerminado() { siguienteAtaque(); }

            public Proyectil generarProyectil() {
                return miJuego.getFabricaEntidades().crearRocaGrande(getX(), getY(), direccionHaciaJugador());
            }

            public int getPuntos() { return 5000; }
        }
```

> **Nota sobre el centro.** El objetivo de la avalancha sale de `getSector().getCentro()` (cambio 4: StoneHinox conoce su sector). Como `getCentro()` es el **punto** central y la posición de una entidad es su esquina superior izquierda, se apunta a `centro - (ancho / 2, alto / 2)` para que quede centrado. `AtaqueAvalancha.puedeAtacar` compara contra ese mismo punto.

Diagrama de secuencia: StoneHinox encadena ataques:



Consideraciones:

- **Las tres rocas de "lanzar rocas" salen juntas**, una encima de la otra, porque `AtaquePorProyectil.atacar` las crea en el mismo llamado. Se ven como una sola. Si se quiere que salgan en abanico, `ubicar` puede correr cada una unos píxeles al costado; si se quiere que salgan **seguidas**, StoneHinox puede usar `cantidad = 1` con intervalo corto y cambiar de ataque a los 3 s. **(a confirmar)**
- La **embestida** apunta a donde **estaba** Link al empezar, no lo persigue. El daño lo hace `AtaquePorContacto(2)` al tocar (2 medios) y lo resuelve `HiloMundo`, como con cualquier enemigo (defensa 6). Con `lanzarRocas` o `avalancha` puestos, tocarlo no lastima: lastiman sus rocas.
- StoneHinox solo recibe daño durante el segundo posterior a la embestida (enunciado: "único momento en que recibe daño").
- `Jefe*` no se mueve a otro sector: los jefes están en sectores cerrados por rocas.

---

15. Comportamientos (patrón Strategy)

Tres interfaces y siete clases. Cada entidad animada tiene una estrategia de cada tipo y la puede cambiar con `setMovimiento`, `setAtaque` y `setDefensa`. Así, por ejemplo, la trampa pasa de quieta a abalanzarse sin `if` sobre su tipo.

15.1 Las tres interfaces

Interfaz	Método	Quién lo llama
`ComportamientoMovimiento`	`+ mover(e : EntidadAnimada, c : AdministradorColisiones)`	`EntidadAnimada.actualizarEstado`
`ComportamientoAtaque`	`+ atacar(atacante : EntidadAnimada, c : AdministradorColisiones)`	`EntidadAnimada.actualizarEstado`
	`+ ataqueAlContactar(atacante : EntidadAnimada, tocado : EntidadAnimada)` (defensa 6)	`Enemigo.colisionarCon(Link)` (Visitor)
`ComportamientoDefensa`	`+ recibirDanio(e : EntidadAnimada, danio : int, origen : Direccion) : boolean`	`EntidadAnimada.recibirDanio`. Devuelve si el daño pasó

Relaciones: las agrega `EntidadAnimada*` (◇ `#movimiento 1`, `#ataque 1`, `#defensa 1`). Reciben el `AdministradorColisiones` por parámetro (dependencia, no se dibuja).

15.2 Las tres estrategias de movimiento

Clase	Atributos	`mover(e, c)`	La usan
`MovimientoAleatorio`	`- azar`, `# velocidad`, `# intervaloCambio`, `# ultimoCambio`	Cada `intervaloCambio` ms elige una dirección al azar; `c.mover(e, dir, velocidad)`; si quedó bloqueado, elige otra	Moblin, Octorok, MasterStalfos, StoneHinox caminando
`MovimientoInducido`	`# direccion`, `# objetivo`, `# velocidad`	Con objetivo: `e.avanzarHacia(objetivo, velocidad)`. Con dirección: `c.mover(e, direccion, velocidad)`. Con las dos en `null`: quieta	Link (teclado), proyectiles, trampa, erizo, embestida, ir al centro
`MovimientoTeletransporte`	`# intervalo`, `# ultimo`	Cada `intervalo` ms: `e.setPosicion(c.getSector().posicionLibreAlAzar(...))` y `e.teletransportarse()`	Wizzrobe

```
        public class MovimientoAleatorio implements ComportamientoMovimiento {
            private Random azar;
            protected int velocidad;
            protected long intervaloCambio;
            protected long ultimoCambio;

            public MovimientoAleatorio(int velocidad, long intervaloCambio) {
                this.azar = new Random();
                this.velocidad = velocidad;
                this.intervaloCambio = intervaloCambio;
            }

            public void mover(EntidadAnimada e, AdministradorColisiones c) {
                long ahora = System.currentTimeMillis();
                if (ahora - ultimoCambio >= intervaloCambio) {
                    e.setDireccion(Direccion.values()[azar.nextInt(4)]);
                    ultimoCambio = ahora;
                }
                if (!c.mover(e, e.getDireccion(), velocidad))
                    e.setDireccion(Direccion.values()[azar.nextInt(4)]);
            }
        }

        public class MovimientoInducido implements ComportamientoMovimiento {
            protected volatile Direccion direccion;      // la escribe el teclado (EDT) en el caso de Link
            protected Posicion objetivo;
            protected int velocidad;

            public MovimientoInducido(Direccion d, int velocidad) {
                this.direccion = d;
                this.velocidad = velocidad;
            }

            public MovimientoInducido(Posicion objetivo, int velocidad) {
                this.objetivo = objetivo;
                this.velocidad = velocidad;
            }

            public void setDireccion(Direccion d) { direccion = d; }
            public Direccion getDireccion() { return direccion; }

            public void mover(EntidadAnimada e, AdministradorColisiones c) {
                if (objetivo != null) e.avanzarHacia(objetivo, velocidad);
                else if (direccion != null) c.mover(e, direccion, velocidad);
            }
        }

        public class MovimientoTeletransporte implements ComportamientoMovimiento {
            protected long intervalo;
            protected long ultimo;
```

```
    public MovimientoTeletransporte(long intervalo) { this.intervalo = intervalo; }

    public void mover(EntidadAnimada e, AdministradorColisiones c) {
        long ahora = System.currentTimeMillis();
        if (ahora - ultimo >= intervalo) {
            e.setPosicion(c.getSector().posicionLibreAlAzar(e.getAncho(), e.getAlto()));
            e.teletransportarse();
            ultimo = ahora;
        }
    }
}
```

Consideraciones:

- **Toda decisión de bloqueo pasa por `c.mover`.** Las estrategias eligen la dirección; si pueden avanzar lo decide el administrador. La única excepción a propósito es `avanzarHacia` (rocas que caen, embestida, ir al centro).
- La primera vez, `ultimoCambio` y `ultimo` valen 0, así que el Moblin cambia de dirección y el Wizzrobe salta en la primera vuelta. Es lo esperable.
- La teletransportación del Wizzrobe no pasa por `puedeMoverA` porque `posicionLibreAlAzar` ya devuelve un lugar libre.

15.3 Las cuatro estrategias de ataque

Clase	Atributos	`atacar(a, c)`	La usan
`AtaquePorContacto`	`# danio` (defensa 6)	**No hace nada**. Defensa 6: `ataqueAlContactar` aplica `tocado.recibirDanio(danio, atacante.getDireccion())`	Moblin, erizo, trampa, embestida, MasterStalfos en su fase de contacto, por defecto (con 0)
`AtaqueEspada`	`# danio`, `# alcance`	Si `a.quiereAtacar()`: `golpear(a, danio, alcance, c)` y `a.ataqueRealizado()`. `ataqueAlContactar` vacío	MasterStalfos en su fase espada
`AtaqueRecargado` (hereda de `AtaqueEspada`)	`- TIEMPO_CARGA = 3000`, `- inicioCarga`	Si `a.soltoElAtaque()`: golpe normal o doble (daño y alcance ×2) si `estaCargado()`, y `a.ataqueRealizado()`	Link
`AtaquePorProyectil`	`# generadorProyectil`, `# cantidad`, `# intervalo`, `# ultimoAtaque`	Template Method: si `puedeAtacar(a)`, genera `cantidadDeProyectiles()` proyectiles, los `ubicar` y los agrega; después `alTerminar(a)`	Octorok, Wizzrobe, StoneHinox (lanzar rocas)
`AtaqueAvalancha` (hereda de `AtaquePorProyectil`)	`- azar`	Redefine los cuatro pasos (tabla de abajo)	StoneHinox

Paso	`AtaquePorProyectil`	`AtaqueAvalancha`
`# puedeAtacar(a)`	`pasoElIntervalo() && a.linkEstaEnfrente()`	Llegó al centro: distancia al centro `< 4`
`# cantidadDeProyectiles()`	`cantidad`	`5 + azar.nextInt(6)` (5 a 10)
`# ubicar(p, a, c)`	Centra el proyectil sobre el atacante	Elige `c.getSector().posicionLibreAlAzar(...)`, pone la roca arriba de todo (`y = 0`) y `p.setDestino(lugar)`
`# alTerminar(a)`	Nada	`a.ataqueTerminado()`
`# pasoElIntervalo()`	`ahora - ultimoAtaque >= intervalo`	(heredado)

```
public interface ComportamientoAtaque {
    void atacar(EntidadAnimada atacante, AdministradorColisiones c);
    void ataqueAlContactar(EntidadAnimada atacante, EntidadAnimada tocado);   // defensa 6
}

public class AtaquePorContacto implements ComportamientoAtaque {
    protected int danio;                                  // daño al tocar, en medios corazones

    public AtaquePorContacto(int danio) { this.danio = danio; }

    public void atacar(EntidadAnimada atacante, AdministradorColisiones c) { }   // nada por turno

    public void ataqueAlContactar(EntidadAnimada atacante, EntidadAnimada tocado) {
        tocado.recibirDanio(danio, atacante.getDireccion());
    }
}

public class AtaqueEspada implements ComportamientoAtaque {
    protected int danio;
    protected int alcance;

    public AtaqueEspada(int danio, int alcance) {
        this.danio = danio;
        this.alcance = alcance;
    }

    public void atacar(EntidadAnimada atacante, AdministradorColisiones c) {
        if (atacante.quiereAtacar()) {
            golpear(atacante, danio, alcance, c);
            atacante.ataqueRealizado();
        }
```

```
        }

        public void ataqueAlContactar(EntidadAnimada atacante, EntidadAnimada tocado) { }   // defensa 6: la espada no lastima al tocar

        protected void golpear(EntidadAnimada a, int danio, int alcance, AdministradorColisiones c) {
            c.golpearEn(a, alcance, danio);
        }
    }

    public class AtaqueRecargado extends AtaqueEspada {
        private static final long TIEMPO_CARGA = 3000;
        private long inicioCarga = -1;

        public AtaqueRecargado(int danio, int alcance) { super(danio, alcance); }

        public void cargar() { inicioCarga = System.currentTimeMillis(); }

        public void atacar(EntidadAnimada atacante, AdministradorColisiones c) {
            if (!atacante.soltoElAtaque()) return;
            int factor = estaCargado() ? 2 : 1;
            golpear(atacante, danio * factor, alcance * factor, c);
            inicioCarga = -1;
            atacante.ataqueRealizado();
        }

        public boolean estaCargado() {
            return inicioCarga >= 0 && System.currentTimeMillis() - inicioCarga >= TIEMPO_CARGA;
        }
    }

    public class AtaquePorProyectil implements ComportamientoAtaque {
        protected GeneradorProyectil generadorProyectil;
        protected int cantidad;
        protected long intervalo;
        protected long ultimoAtaque;

        public AtaquePorProyectil(GeneradorProyectil g, int cantidad, long intervalo) {
            this.generadorProyectil = g;
            this.cantidad = cantidad;
            this.intervalo = intervalo;
        }

        public void atacar(EntidadAnimada atacante, AdministradorColisiones c) {   // Template Method
            if (!puedeAtacar(atacante)) return;
            int n = cantidadDeProyectiles();                  // se pide una sola vez (ver consideraciones)
            for (int i = 0; i < n; i++) {
                Proyectil p = generadorProyectil.generarProyectil();
                ubicar(p, atacante, c);
                atacante.getJuego().agregarEntidad(p);
            }
            ultimoAtaque = System.currentTimeMillis();
            alTerminar(atacante);
        }

        public void ataqueAlContactar(EntidadAnimada atacante, EntidadAnimada tocado) { }   // defensa 6: no lastima al tocar

        protected void ubicar(Proyectil p, EntidadAnimada a, AdministradorColisiones c) {
            int x = a.getX() + (a.getAncho() - p.getAncho()) / 2;
            int y = a.getY() + (a.getAlto() - p.getAlto()) / 2;
            p.setPosicion(new Posicion(x, y));
        }

        protected void alTerminar(EntidadAnimada atacante) { }
        protected boolean puedeAtacar(EntidadAnimada a) { return pasoElIntervalo() && a.linkEstaEnfrente(); }
        protected int cantidadDeProyectiles() { return cantidad; }
        protected boolean pasoElIntervalo() { return System.currentTimeMillis() - ultimoAtaque >= intervalo; }
    }

    public class AtaqueAvalancha extends AtaquePorProyectil {
        private Random azar;

        public AtaqueAvalancha(GeneradorProyectil g) {
            super(g, 0, 0);
            azar = new Random();
        }

        protected void ubicar(Proyectil roca, EntidadAnimada a, AdministradorColisiones c) {
            Posicion lugar = c.getSector().posicionLibreAlAzar(roca.getAncho(), roca.getAlto());
            roca.setPosicion(new Posicion(lugar.getX(), 0));
            roca.setDireccion(Direccion.ABAJO);
            roca.setDestino(lugar);
        }

        protected void alTerminar(EntidadAnimada atacante) { atacante.ataqueTerminado(); }

        protected boolean puedeAtacar(EntidadAnimada a) {
            Posicion centro = a.getSector().getCentro();
```

```
            Posicion objetivo = new Posicion(centro.getX() - a.getAncho() / 2, centro.getY() - a.getAlto() / 2);
            return a.getPosicion().distancia(objetivo) < 4;
        }

        protected int cantidadDeProyectiles() { return 5 + azar.nextInt(6); }
    }
```

Consideraciones:

- ⚠️ `AtaqueEspada` **no tiene enfriamiento**: si MasterStalfos tiene a Link enfrente, golpea en cada vuelta (cada 30 ms). Lo frena la invulnerabilidad de 1 s de Link, así que en la práctica pega una vez por segundo.
- En `AtaqueRecargado`, el golpe sale **al soltar** la tecla. Un toque rápido es un golpe normal casi inmediato.
- El `ubicar` del padre recibe `c` aunque no lo use: es la firma del extendido, y la avalancha sí lo necesita.
- ⚠️ `atacar` guarda `cantidadDeProyectiles()` en `n` **antes** del `for`. En `AtaqueAvalancha` ese método devuelve un número al azar distinto cada vez: si se llamara en la condición del `for`, la cantidad cambiaría mientras se recorre.

15.4 Las dos estrategias de defensa

Clase	Atributos	`recibirDanio(e, danio, origen)`	Configuraciones
`DefensaLimitada`	`# danioMaximo`, `# golpesRestantes`, `+ SIN_LIMITE = Integer.MAX_VALUE`, `+ SIEMPRE = -1`	Deja pasar `min(danio, danioMaximo)`; si es 0 devuelve `false`; si no, `e.aplicarDanio(real)`. Si tenía golpes contados y se terminan, queda sin límite	Sin límite (normal), tope 0 (invulnerable), `(1, 3)` (bellota)
`DefensaPorEscudo`	`# soloDeFrente`, `# sinEscudo`	Si bloquea (todo, o solo lo que viene de frente), devuelve `false`; si no, delega en `sinEscudo`	`(false)`: Link con escudo. `(true)`: Moblin con escudo

```
    public class DefensaLimitada implements ComportamientoDefensa {
        public static final int SIN_LIMITE = Integer.MAX_VALUE;
        public static final int SIEMPRE = -1;
        protected int danioMaximo;
        protected int golpesRestantes;

        public DefensaLimitada(int danioMaximo, int golpes) {
            this.danioMaximo = danioMaximo;
            this.golpesRestantes = golpes;
        }

        public boolean recibirDanio(EntidadAnimada e, int danio, Direccion origen) {
            int real = Math.min(danio, danioMaximo);
            if (real == 0) return false;
            e.aplicarDanio(real);
            if (golpesRestantes != SIEMPRE && --golpesRestantes == 0) {
                danioMaximo = SIN_LIMITE;
                golpesRestantes = SIEMPRE;
            }
            return true;
        }

        public int getGolpesRestantes() { return golpesRestantes; }
    }

    public class DefensaPorEscudo implements ComportamientoDefensa {
        protected boolean soloDeFrente;
        protected ComportamientoDefensa sinEscudo;

        public DefensaPorEscudo(boolean soloDeFrente) {
            this.soloDeFrente = soloDeFrente;
            this.sinEscudo = new DefensaLimitada(DefensaLimitada.SIN_LIMITE, DefensaLimitada.SIEMPRE);
        }

        public boolean recibirDanio(EntidadAnimada e, int danio, Direccion origen) {
            boolean deFrente = origen == e.getDireccion().opuesta();
            if (!soloDeFrente || deFrente) return false;      // bloqueado
            return sinEscudo.recibirDanio(e, danio, origen);
        }
    }
```

Qué es `origen`: la dirección en la que **venía** el golpe (la dirección del atacante). Si Link mira a la `DERECHA` y le pega a un Moblin con escudo que mira a la `IZQUIERDA`, `origen = DERECHA` y `e.getDireccion().opuesta() = DERECHA`: viene de frente y el escudo lo bloquea. Si Link le pega por la espalda, el Moblin mira a la `DERECHA`, la opuesta es `IZQUIERDA` y no coincide: el golpe pasa.

Configuraciones de defensa por entidad:

Entidad	Defensa
Moblin, Octorok, Erizo, Link normal	`DefensaLimitada(SIN_LIMITE, SIEMPRE)`
Trampa, proyectiles	`DefensaLimitada(0, SIEMPRE)`
Wizzrobe, MasterStalfos, StoneHinox	Alternan entre sin límite y tope 0
Link 1 s después de un golpe	`DefensaLimitada(0, SIEMPRE)` y vuelve a `defensaBase`
Link después de la bellota	`defensaBase = DefensaLimitada(1, 3)`

Entidad	Defensa
Link con el escudo	`DefensaPorEscudo(false)`
MoblinConEspadaYEscudo	`DefensaPorEscudo(true)`

---

16. Mapa y niveles

16.1 Sector

Utilidad: una pantalla del mapa, de 768 × 528 píxeles, con **una sola lista** de todas sus entidades (fijas y animadas, Link incluido cuando está ahí).

Elemento	Qué es	Para qué
`# entidades : List<Entidad>`	Atributo	Todas las entidades del sector. → `#entidades 0..*`
`+ ANCHO : int = 768`, `+ ALTO : int = 528`	Constantes `static final`	Tamaño de todos los sectores
`# fila : int`, `# columna : int`	Atributos	Dónde está en la grilla de la `Escena`
`+ Sector(fila, columna)`	Constructor	Lo crea `GeneradorNivel`
`+ agregar(e)`	Comando	`entidades.add(e)` y `e.setSector(this)` (cambio 4)
`+ quitar(e)`	Comando	Saca una entidad que **dejó de existir**. La llama `Entidad.eliminar()`
`+ retirar(e)`	Comando	Saca una entidad que **sigue existiendo** y se va a otro lado: Link al cambiar de sector o al reiniciar el nivel
`+ actualizarEntidades(c)`	Comando	Una vuelta de `HiloMundo`: `actualizarPorMundo(c)` a cada entidad no eliminada
`+ getEntidades() : List<Entidad>`	Consulta	Devuelve una **copia** de la lista
`+ posicionLibreAlAzar(ancho, alto) : Posicion`	Consulta	Un lugar donde entra un rectángulo de ese tamaño sin pisar a nadie. Lo usan los fragmentos, el Wizzrobe y la avalancha
`+ getCentro() : Posicion`	Consulta	`(ANCHO / 2, ALTO / 2)` = `(384, 264)`
`+ getFila()`, `+ getColumna()`	Consultas	Para `Escena.getVecino`

Relaciones: es parte de `Escena` (◆ `#sectores 1..*`); referencia a sus entidades (→ `#entidades 0..*`) y las entidades lo referencian a él (→ `#miSector 1`, cambio 4); lo referencia `AdministradorColisiones` (`#sector 1`).

```
        public class Sector {
            public static final int ANCHO = 768;
            public static final int ALTO = 528;
            protected List<Entidad> entidades;
            protected int fila;
            protected int columna;

            public Sector(int fila, int columna) {
                this.fila = fila;
                this.columna = columna;
                this.entidades = new ArrayList<>();
            }

            public synchronized void agregar(Entidad e) {
                entidades.add(e);
                e.setSector(this);
            }

            public synchronized void quitar(Entidad e) { entidades.remove(e); }
            public synchronized void retirar(Entidad e) { entidades.remove(e); }

            public void actualizarEntidades(AdministradorColisiones c) {    // SIN synchronized (sección 22)
                for (Entidad e : getEntidades())
                    if (!e.estaEliminada()) e.actualizarPorMundo(c);
            }

            public synchronized List<Entidad> getEntidades() { return new ArrayList<>(entidades); }

            public Posicion posicionLibreAlAzar(int ancho, int alto) {
                Random azar = new Random();
                List<Entidad> todas = getEntidades();
                for (int intento = 0; intento < 200; intento++) {
                    int x = azar.nextInt(ANCHO - ancho + 1);
                    int y = azar.nextInt(ALTO - alto + 1);
                    Hitbox lugar = new Hitbox(x, y, ancho, alto);
                    boolean libre = true;
                    for (Entidad e : todas)
                        if (!e.estaEliminada() && e.getHitbox().intersecta(lugar)) libre = false;
                    if (libre) return new Posicion(x, y);
                }
                return getCentro();
            }

            public Posicion getCentro() { return new Posicion(ANCHO / 2, ALTO / 2); }
```

```
        public int getFila() { return fila; }
        public int getColumna() { return columna; }
    }
```

Consideraciones:

- `quitar` y `retirar` **hacen lo mismo con la lista**, pero significan cosas distintas y los llaman desde lugares distintos. Se dejan los dos, como en el diagrama, porque así el código dice qué está pasando.
- ⚠️ `actualizarEntidades` **no es** `synchronized`**.** Si lo fuera, `HiloMundo` tendría la llave del sector mientras mueve a un enemigo, y ese enemigo puede terminar llamando cosas que piden otras llaves (`Juego`, Link). Con `HiloJugador` haciendo el camino inverso, el juego se traba (abrazo mortal). Recorrer la copia de `getEntidades()` alcanza para que no falle. Detalle en la sección 22.
- `posicionLibreAlAzar` usa coordenadas cualquiera. Si se prefiere que todo quede alineado a la grilla, se puede multiplicar por 48: `x = 48 * azar.nextInt(16)`. **(propuesta)**

16.2 Escena

Utilidad: la grilla de sectores de un nivel (el mapa completo). Sabe cuál es el vecino de un sector en cada dirección.

Elemento	Qué hace
`# sectores : Sector[][]`	La grilla. ◆ `#sectores 1..*`
`# filas : int`, `# columnas : int`	Tamaño de la grilla (línea `MAPA` del archivo)
`+ Escena(filas, columnas)`	Crea la matriz vacía
`+ agregarSector(fila, columna, s)`	`sectores[fila][columna] = s`
`+ getSector(fila, columna)`	
`+ getVecino(s, d)`	El sector de al lado de `s` en la dirección `d`, o `null` si no hay
`+ getFilas()`, `+ getColumnas()`	

```
    public Sector getVecino(Sector s, Direccion d) {
        int f = s.getFila() + d.getDy();
        int c = s.getColumna() + d.getDx();
        if (f < 0 || f >= filas || c < 0 || c >= columnas) return null;
        return sectores[f][c];
    }
```

Ejemplo: en el nivel 1 (grilla de 2 × 3), `getVecino(sector(0,0), DERECHA)` es el sector `(0,1)` y `getVecino(sector(0,0), ARRIBA)` es `null`.

16.3 Nivel

Utilidad: un nivel armado: su mapa, sus jefes y dónde empieza Link.

Elemento	Qué hace	Relación
`# mapa : Escena`	El mapa	◆ `#mapa 1`
`# jefes : List<Jefe>`	Los jefes del nivel	→ `#jefes 1..*`
`# filaInicial`, `# columnaInicial : int`	Sector donde empieza Link	
`# posicionInicial : Posicion`	Píxel donde empieza Link dentro de ese sector	→ `#posicionInicial 1`
`+ Nivel(mapa, jefes, fila, columna, inicio)`	Lo crea `GeneradorNivel`	
`+ getMapa()`, `getPosicionInicial()`, `getJefes()`		
`+ getSectorInicial()`	`mapa.getSector(filaInicial, columnaInicial)`	
`+ jefesDerrotados() : boolean`	`true` si todos los jefes están eliminados	

```
    public boolean jefesDerrotados() {
        for (Jefe j : jefes)
            if (!j.estaEliminada()) return false;
        return true;
    }
```

16.4 GeneradorNivel y el formato del archivo

Utilidad: lee `nivel<n>.txt` y arma el `Nivel` con todas sus entidades. Es el **único** lugar donde se usan celdas (`CELDA = 48`). Requisito esencial de la defensa 6: "cargar al menos un nivel completo desde un archivo" y entregar `Formateo-de-niveles.pdf` junto a esta clase.

Elemento	Qué hace
`# carpetaNiveles : String`	Dónde están los archivos, por ejemplo `"/niveles/"`
`- CELDA : int = 48`	Píxeles por celda
`+ GeneradorNivel(carpetaNiveles)`	
`+ generar(numero, fabrica) : Nivel`	Lee el archivo, crea escena, sectores, entidades y fragmentos, y devuelve el nivel
`- leerArchivo(numero) : List<String>`	Las líneas no vacías del archivo
`- crearEntidad(simbolo, x, y, fabrica) : Entidad`	Traduce un carácter a una entidad con la fábrica (o `null` para `.`)

Elemento	Qué hace
`- colocarFragmentosDeCorazon(e, fabrica)`	4 fragmentos, en 4 sectores distintos al azar

Relaciones: lo referencia `Juego` (`#generadorNivel 1`). Crea `Nivel` y usa `FabricaEntidades` por parámetro (dependencias no dibujadas).

Formato (resumen; el completo está en `Defensa 3/Formateo-de-niveles.md`):

```
MAPA <filas> <columnas>
CELDAS <ancho> <alto> <tamaño>
INICIO <fila> <columna> <x> <y>
SECTOR <fila> <columna>
<alto líneas de ancho caracteres>
SECTOR ...
```

Símbolo	Crea	Símbolo	Crea
`R`	`Roca`	`O`	`Octorok`
`A`	`Agua`	`T`	`TrampaDeCuchillas`
`P`	`Pasto`	`W`	`Wizzrobe`
`V`	`CaidasAlVacio`	`S`	`MasterStalfos` (jefe)
`Q`	`SuelosQuebradizo`	`H`	`StoneHinox` (jefe)
`M`	`Moblin`	`$`	`Rupia`
`N`	`MoblinConEspadaYEscudo`	`C`	`Corazon`
`E`	`ErizoDeMar`	`X`	`Espada`
`.`	Nada (vacío o paso al vecino si está en el borde)		

```java
public class GeneradorNivel {
    private static final int CELDA = 48;
    protected String carpetaNiveles;

    public GeneradorNivel(String carpetaNiveles) { this.carpetaNiveles = carpetaNiveles; }

    public Nivel generar(int numero, FabricaEntidades fabrica) {
        List<String> lineas = leerArchivo(numero);
        String[] mapa = lineas.get(0).split(" ");
        String[] celdas = lineas.get(1).split(" ");
        String[] inicio = lineas.get(2).split(" ");
        int ancho = Integer.parseInt(celdas[1]);
        int alto = Integer.parseInt(celdas[2]);

        Escena escena = new Escena(Integer.parseInt(mapa[1]), Integer.parseInt(mapa[2]));
        List<Jefe> jefes = new ArrayList<>();
        int i = 3;
        while (i < lineas.size()) {
            String[] cabecera = lineas.get(i).split(" ");
            Sector sector = new Sector(Integer.parseInt(cabecera[1]), Integer.parseInt(cabecera[2]));
            escena.agregarSector(sector.getFila(), sector.getColumna(), sector);
            for (int r = 0; r < alto; r++) {
                String fila = lineas.get(i + 1 + r);
                for (int k = 0; k < ancho; k++) {
                    char s = fila.charAt(k);
                    int x = k * CELDA;
                    int y = r * CELDA;
                    if (s == 'S' || s == 'H') {
                        Jefe j = (s == 'S') ? fabrica.crearMasterStalfos(x, y) : fabrica.crearStoneHinox(x, y);
                        jefes.add(j);
                        sector.agregar(j);
                    } else {
                        Entidad e = crearEntidad(s, x, y, fabrica);
                        if (e != null) sector.agregar(e);
                    }
                }
            }
            i += 1 + alto;
        }
        colocarFragmentosDeCorazon(escena, fabrica);
        Posicion posicion = new Posicion(Integer.parseInt(inicio[3]) * CELDA, Integer.parseInt(inicio[4]) * CELDA);
        return new Nivel(escena, jefes, Integer.parseInt(inicio[1]), Integer.parseInt(inicio[2]), posicion);
    }

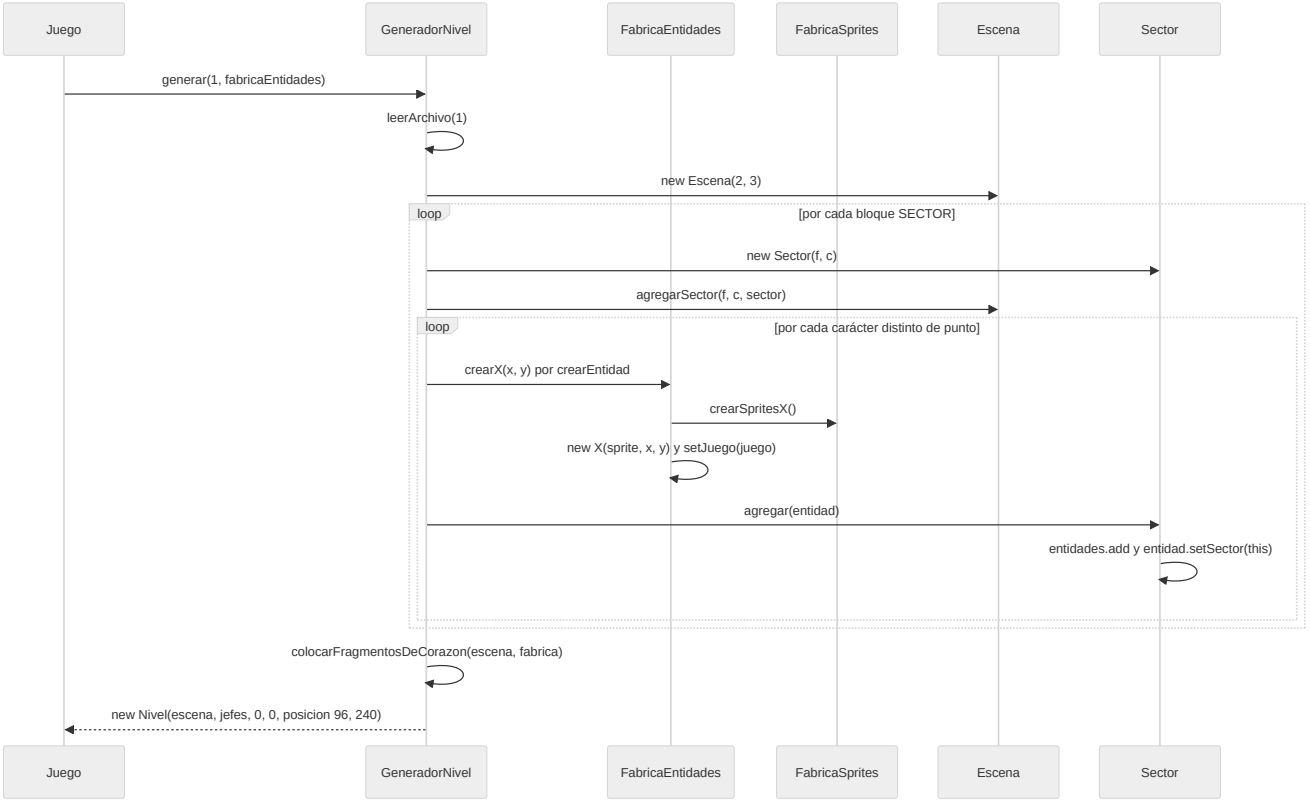
    private List<String> leerArchivo(int numero) {
        String ruta = carpetaNiveles + "nivel" + numero + ".txt";
        List<String> lineas = new ArrayList<>();
        try (BufferedReader lector = new BufferedReader(
                new InputStreamReader(getClass().getResourceAsStream(ruta), StandardCharsets.UTF_8))) {
            String linea;
            while ((linea = lector.readLine()) != null)
                if (!linea.isBlank()) lineas.add(linea);
        } catch (IOException | NullPointerException ex) {
            throw new IllegalStateException("No se pudo leer " + ruta, ex);
```

```
            }
            return lineas;
        }

        private Entidad crearEntidad(char simbolo, int x, int y, FabricaEntidades fabrica) {
            switch (simbolo) {
                case 'R': return fabrica.crearRoca(x, y);
                case 'A': return fabrica.crearAgua(x, y);
                case 'P': return fabrica.crearPasto(x, y);
                case 'V': return fabrica.crearCaidasAlVacio(x, y);
                case 'Q': return fabrica.crearSuelosQuebradizo(x, y);
                case 'M': return fabrica.crearMoblin(x, y);
                case 'N': return fabrica.crearMoblinConEspadaYEscudo(x, y);
                case 'E': return fabrica.crearErizoDeMar(x, y);
                case 'O': return fabrica.crearOctorok(x, y);
                case 'T': return fabrica.crearTrampaDeCuchillas(x, y);
                case 'W': return fabrica.crearWizzrobe(x, y);
                case '$': return fabrica.crearRupia(x, y);
                case 'C': return fabrica.crearCorazon(x, y);
                case 'X': return fabrica.crearEspada(x, y);
                case '.': return null;
                default: throw new IllegalArgumentException("Símbolo desconocido: " + simbolo);
            }
        }

        private void colocarFragmentosDeCorazon(Escena e, FabricaEntidades fabrica) {
            List<Sector> sectores = new ArrayList<>();
            for (int f = 0; f < e.getFilas(); f++)
                for (int c = 0; c < e.getColumnas(); c++)
                    if (e.getSector(f, c) != null) sectores.add(e.getSector(f, c));
            Collections.shuffle(sectores);
            for (int k = 0; k < 4 && k < sectores.size(); k++) {
                Posicion p = sectores.get(k).posicionLibreAlAzar(CELDA, CELDA);
                sectores.get(k).agregar(fabrica.crearFragmentoDeCorazon(p.getX(), p.getY()));
            }
        }
    }
```

Diagrama de secuencia: carga del nivel 1:



Consideraciones:

- **El archivo va como recurso** (dentro del classpath, en una carpeta `niveles`) y se lee con `getResourceAsStream`, así funciona igual desde el IDE y desde un `.jar`.
- Los imports son de `java.io`, `java.nio.charset` y `java.util`. Nada de librerías externas.
- **El `switch` sobre el carácter no viola la regla del `instanceof`:** traduce texto a objetos, no pregunta el tipo de un objeto ya creado.
- **Los jefes se tratan antes de `crearEntidad`** porque `crearEntidad` devuelve `Entidad` y la lista es de `Jefe`: así no hace falta *cast*.
- **Errores del archivo:** un símbolo desconocido o una línea corta tiran una excepción con un mensaje claro. Es mejor que el juego no arranque a que arranque con un mapa roto.

- ⚠️ El `Formateo-de-niveles.pdf` tiene que estar **en el mismo paquete** que esta clase (lo pide la defensa 6).

16.5 AdministradorColisiones

Utilidad: todo lo espacial durante el movimiento: si una entidad puede avanzar, con qué se choca después de moverse, a quién alcanza una espada y si salió de la pantalla. Siempre trabaja sobre el **sector visible**.

Elemento	Qué hace
`# sector : Sector`	El sector visible. → `#sector 1`
`+ AdministradorColisiones()`	Lo crea el constructor de `Juego` (◆ `#colisiones 1`)
`+ setSector(s)`, `+ getSector()`	Lo cambia `Juego.activarSector` / al cambiar de sector
`+ mover(e, d, pasos) : boolean`	Si `puedeMoverA`: `e.avanzar(d, pasos)` y, si quedó afuera, `e.alSalirDelSector(d)`. Si no: `e.alChocarConBloqueo()`. Devuelve si avanzó
`- puedeMoverA(e, d, pasos) : boolean`	¿El rectángulo corrido pisa alguna entidad con `bloqueaPaso()`?
`+ verificarColisiones(quien)`	A cada entidad que se superpone con `quien`: `aceptarColision(quien)`
`+ golpearEn(atacante, alcance, danio)`	A cada entidad en la zona `adelante(dir, alcance)`: `recibirGolpeDeEspada(atacante, danio)`
`+ estaFuera(h) : boolean`	`!h.estaDentroDe(Sector.ANCHO, Sector.ALTO)`

Relaciones: es parte de `Juego` (◆ `#colisiones 1`); lo referencian `HiloJugador` y `HiloMundo` (`#colisiones 1`); referencia al sector visible (`#sector 1`). Las estrategias y entidades lo reciben por parámetro.

```
        public class AdministradorColisiones {
            protected volatile Sector sector;          // lo cambia Juego desde otro hilo

            public AdministradorColisiones() { }

            public void setSector(Sector s) { sector = s; }
            public Sector getSector() { return sector; }

            public boolean mover(EntidadAnimada e, Direccion d, int pasos) {
                if (!puedeMoverA(e, d, pasos)) {
                    e.alChocarConBloqueo();
                    return false;
                }
                e.avanzar(d, pasos);
                if (estaFuera(e.getHitbox())) e.alSalirDelSector(d);
                return true;
            }

            private boolean puedeMoverA(EntidadAnimada e, Direccion d, int pasos) {
                Hitbox siguiente = e.getHitbox().desplazada(d, pasos);
                for (Entidad otra : sector.getEntidades())
                    if (otra != e && otra.bloqueaPaso() && otra.getHitbox().intersecta(siguiente)) return false;
                return true;
            }

            public void verificarColisiones(EntidadAnimada quien) {
                Sector s = sector;
                for (Entidad otra : s.getEntidades()) {
                    if (quien.estaEliminada() || quien.getSector() != s) return;   // murió o cambió de sector
                    if (otra != quien && !otra.estaEliminada() && otra.getHitbox().intersecta(quien.getHitbox()))
                        otra.aceptarColision(quien);
                }
            }

            public void golpearEn(EntidadAnimada atacante, int alcance, int danio) {
                Hitbox zona = atacante.getHitbox().adelante(atacante.getDireccion(), alcance);
                for (Entidad otra : sector.getEntidades())
                    if (otra != atacante && !otra.estaEliminada() && otra.getHitbox().intersecta(zona))
                        otra.recibirGolpeDeEspada(atacante, danio);
            }

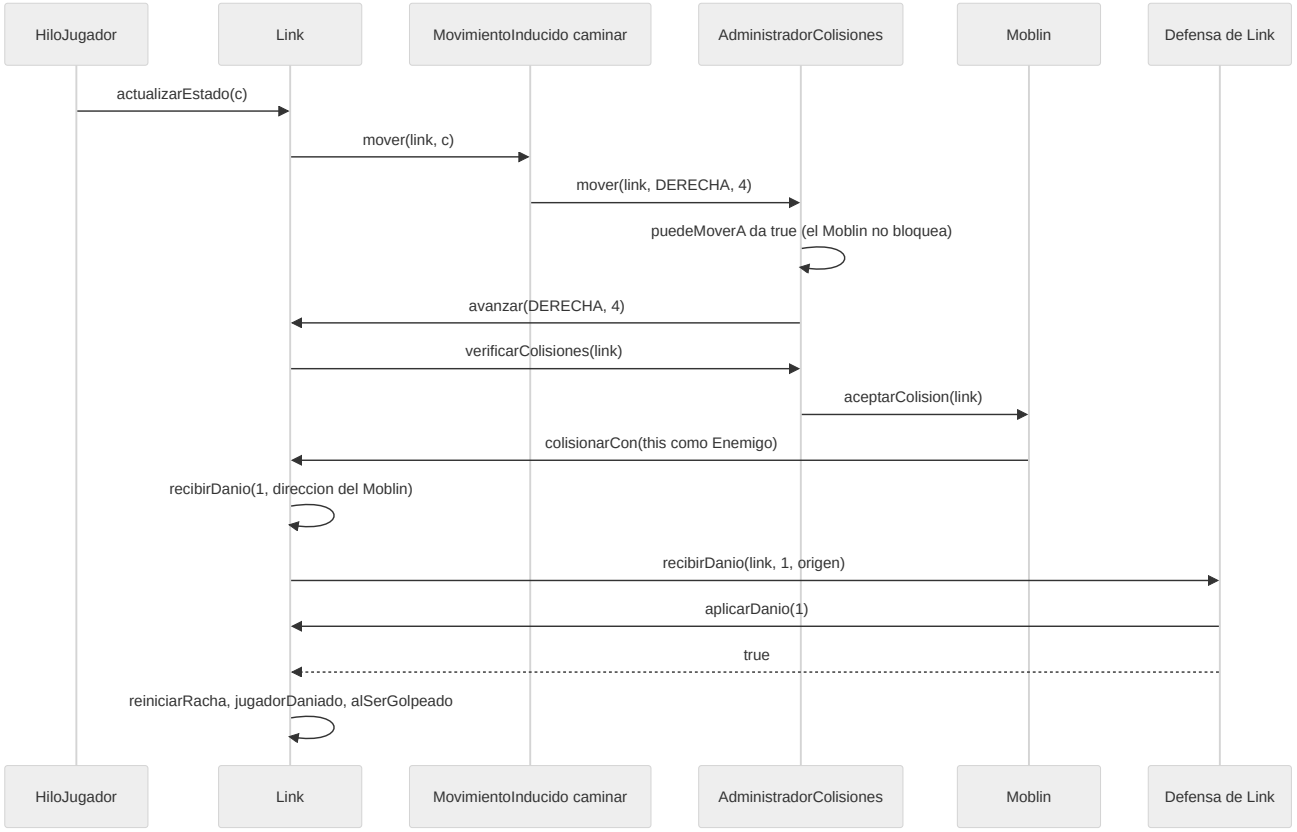
            public boolean estaFuera(Hitbox h) { return !h.estaDentroDe(Sector.ANCHO, Sector.ALTO); }
        }
```

Consideraciones:

- ⚠️ **Va en el paquete de las entidades** (ver sección 3): llama a dos ganchos `#`.
- ⚠️ **Ningún método es `synchronized`.** Recorren copias (`getEntidades()`), que es suficiente. Si `mover` fuera `synchronized`, se arma el abrazo mortal de la sección 22.
- **`verificarColisiones` corta si `quien` cambió de sector.** Pasa cuando Link cae al vacío o mata al último jefe en medio del recorrido: `Juego` recarga el nivel y Link ya está en otro sector. Sin el corte, el resto del recorrido compararía a Link (en su posición nueva) con entidades del sector viejo. La comparación usa `getSector()`, que existe gracias al cambio 4.
- **Cómo se llega a cada gancho:**
  - Moblin contra una roca: `puedeMoverA` da `false`, `alChocarConBloqueo()` vacío, y `MovimientoAleatorio` elige otra dirección.
  - Trampa contra una roca: `alChocarConBloqueo()` → `volver()`.

- Proyectil contra una roca: `alChocarConBloqueo()` → desaparece.
- Link cruza el borde: `alSalirDelSector(d)` → `Juego.cambiarDeSector(d)`.
- Proyectil cruza el borde: `alSalirDelSector(d)` → desaparece.

Diagrama de secuencia: Link camina y choca con un Moblin (todo en `HiloJugador`):



---

17. Fábricas (Abstract Factory)

17.1 FabricaEntidades

Utilidad: el único lugar donde se hace `new` de las entidades del juego. Cada `crearX(x, y)` pide el sprite a la fábrica de sprites del dominio elegido, crea la entidad y le asigna el juego.

Elemento	Qué hace
`# fabricaSprites : FabricaSprites`	El dominio gráfico. → `#fabricaSprites 1`
`# juego : Juego`	Para `e.setJuego(juego)` en cada entidad
`+ FabricaEntidades(juego, fabricaSprites)`	La crea `Juego.iniciar`
`+ crearLink(x, y)`	Link con el escudo en el inventario
`+ crearMoblin` … `crearStoneHinox`	Los 8 enemigos y jefes
`+ crearRupia` … `crearFragmentoDeCorazon`	Los 4 power-ups
`+ crearEspada`, `crearEscudo`	Los objetos
`+ crearRoca` … `crearSuelosQuebradizo`	Los 5 obstáculos
`+ crearProyectil(x, y, d)`	Proyectil común, daño 1
`+ crearRocaGrande(x, y, d)`	Proyectil de StoneHinox, daño 2

Relaciones: la referencia `Juego` (`#fabricaEntidades 1`); referencia a `FabricaSprites*`. Crea todas las entidades (dependencias no dibujadas).

```
        public class FabricaEntidades {
            protected FabricaSprites fabricaSprites;
            protected Juego juego;

            public FabricaEntidades(Juego juego, FabricaSprites fabricaSprites) {
                this.juego = juego;
                this.fabricaSprites = fabricaSprites;
            }

            public Link crearLink(int x, int y) {
                Link l = new Link(fabricaSprites.crearSpritesLink(), x, y);
                l.setJuego(juego);
                l.agregarAlInventario(crearEscudo(x, y));      // Link empieza con el escudo
```

```
            return l;
        }

        public Moblin crearMoblin(int x, int y) {
            Moblin m = new Moblin(fabricaSprites.crearSpritesMoblin(), x, y);
            m.setJuego(juego);
            return m;
        }

        public Proyectil crearProyectil(int x, int y, Direccion d) {
            Proyectil p = new Proyectil(fabricaSprites.crearSpritesProyectil(), x, y, d, 1);
            p.setJuego(juego);
            return p;
        }

        public Proyectil crearRocaGrande(int x, int y, Direccion d) {
            Proyectil p = new Proyectil(fabricaSprites.crearSpritesRocaGrande(), x, y, d, 2);
            p.setJuego(juego);
            return p;
        }

        // ... el resto sigue el mismo molde: new X(fabricaSprites.crearSpritesX(), x, y) y setJuego(juego)
    }
```

Consideraciones:

- Las 22 `crearX` tienen la misma forma. Es repetitivo pero claro, y cada una coincide 1 a 1 con el extendido. **No** agregar un método auxiliar privado sin dibujarlo.
- ⚠️ `setJuego` se llama **antes** de devolver la entidad. Si una entidad usa `miJuego` en su constructor, todavía es `null`: por eso ninguna lo usa ahí (StoneHinox programa su primer ataque para dentro de 3 s).

17.2 FabricaSprites*, FabricaSpritesOriginal y FabricaSpritesAlternativa

Utilidad: los dos dominios gráficos que pide el enunciado. Las clases del juego son las mismas; solo cambian las imágenes.

Elemento	Qué hace
`+ crearSpritesLink()` … `+ crearSpritesRocaGrande()` (22)	Cada una: `armar("nombre", estados...)`
`# armar(nombre, estados...) : Sprite`	Crea un `Sprite` y le agrega `carpeta() + nombre + "-" + estado + ".png"` por cada estado
`# carpeta() : String`	Abstracto. `"/imagenes/original/"` o `"/imagenes/alternativa/"`

Relaciones: abstracta; la heredan `FabricaSpritesOriginal` y `FabricaSpritesAlternativa` (cada una redefine solo `carpeta()`); la referencia `FabricaEntidades`; crea `Sprite` (dependencia «crea»). La elige la vista (`PanelPantallaPrincipal.elegirDominio`).

```
    public abstract class FabricaSprites {
        public Sprite crearSpritesLink() {
            return armar("link", "abajo", "arriba", "izquierda", "derecha",
                         "ataque-abajo", "ataque-arriba", "ataque-izquierda", "ataque-derecha");
        }
        public Sprite crearSpritesMoblin() { return armar("moblin", "abajo", "arriba", "izquierda", "derecha"); }
        public Sprite crearSpritesRoca() { return armar("roca", "normal"); }
        // ... una por entidad

        protected Sprite armar(String nombre, String... estados) {
            Sprite s = new Sprite();
            for (String estado : estados)
                s.agregar(estado, carpeta() + nombre + "-" + estado + ".png");
            return s;
        }

        protected abstract String carpeta();
    }

    public class FabricaSpritesOriginal extends FabricaSprites {
        public FabricaSpritesOriginal() { }
        protected String carpeta() { return "/imagenes/original/"; }
    }
```

Consideraciones:

- Es un Abstract Factory con un Template Method adentro: la abstracta arma todo y la concreta solo dice **de qué carpeta**. Por eso las dos concretas son de una línea.
- ⚠️ Los nombres de archivo tienen que coincidir **exactamente** (mayúsculas incluidas: en Linux y dentro de un `.jar` importan). Conviene una tabla con todos los archivos en el README de recursos.
- La fábrica está en la lógica pero no usa Swing: solo arma **textos** con rutas. Quien carga la imagen es `CeldaGrafica`.

---

18. Núcleo del juego

18.1 ControladorJuego (interfaz)

Utilidad: lo que la vista le puede pedir a la lógica. La realiza `Juego`; la referencian `GUI` (`#controladorJuego 1`) y `AnimadorTransicion` (`#controlador 1`).

Método	Quién lo llama	Qué hace en `Juego`
`+ iniciar(modo, dominio)`	`GUI.iniciarPartida`	Arma y arranca la partida
`+ registrarNombre(nombre)`	`GUI.pedirNombre`, cuando el jugador escribe su nombre	Guarda la entrada en el ranking del modo
`+ transicionTerminada()`	`AnimadorTransicion.run`, al terminar	Activa el sector nuevo y reanuda los hilos
`+ getRanking(m) : Ranking`	`GUI.mostrarPantallaRanking`	El ranking de ese modo
`+ getControlJugador() : ControlJugador`	`GUI.iniciarPartida`, después de `iniciar`	Devuelve a Link visto como `ControlJugador`

18.2 ControladorGrafico (interfaz)

Utilidad: lo que la lógica le puede pedir a la vista. La realiza `GUI`; la referencia `Juego` (`#controladorGrafico 1`).

Método	Cuándo lo llama `Juego`
`+ mostrarEntidad(e) : Observer`	Por cada entidad al activar un sector, y por cada entidad nueva (`agregarEntidad`)
`+ mostrarJugador(j) : Observer`	Al activar un sector
`+ limpiarEscena()`	Antes de mostrar un sector
`+ transicion(d)`	Al cambiar de sector
`+ actualizarTiempo(segundos)`	Cada segundo (`tic`)
`+ actualizarNivel(numero)`	Al cargar un nivel
`+ pedirNombre(puntaje)`	Al terminar, si el puntaje entra en el top 5
`+ mostrarRanking(r, nombreModo)`	Al terminar (después de registrar el nombre, o directamente si no entró)

⚠️ **Todos** se llaman desde hilos que no son el EDT (`HiloJugador`, `Reloj`, `AnimadorTransicion`). Por eso en `GUI` cada uno mete el cambio de Swing dentro de `SwingUtilities.invokeLater` (ver 20.1).

18.3 Juego

Utilidad: el árbitro de la partida. Arma todo al iniciar, carga niveles, activa sectores, recibe los avisos de las entidades (puntos, daño, jefes, caídas, cambio de sector), maneja los hilos y los rankings.

Realiza `ControladorJuego`.

Atributo	Tipo	Relación	Se crea en
`# controladorGrafico`	`ControladorGrafico`	→ `#controladorGrafico 1`	Constructor (parámetro)
`# jugador`	`Link`	→ `#jugador 1`	`iniciar` (con la fábrica)
`# nivel`	`Nivel`	◇ `#nivel 1`	`cargarNivel`
`# numeroNivel`	`int`		`iniciar` (1)
`# modoJuego`	`ModoJuego`	◇ `#modoJuego 1`	`iniciar` (parámetro)
`# rankings`	`Ranking[3]`	◆ `#rankings 3`	Constructor (los carga la persistencia)
`# reloj`	`Reloj`	◆ `#reloj 1`	`iniciar`
`# hiloJugador`	`HiloJugador`	◆ `#hiloJugador 1`	`iniciar`
`# hiloMundo`	`HiloMundo`	◆ `#hiloMundo 1`	`iniciar`
`# colisiones`	`AdministradorColisiones`	◆ `#colisiones 1`	Constructor
`# generadorNivel`	`GeneradorNivel`	→ `#generadorNivel 1`	Constructor
`# fabricaEntidades`	`FabricaEntidades`	→ `#fabricaEntidades 1`	`iniciar` (con el dominio elegido)
`# persistencia`	`PersistenciaRanking`	→ `#persistencia 1`	Constructor

Índice de los rankings (propuesta): `0` Clásico, `1` Contrarreloj, `2` NoHit. Archivos `ranking0.txt`, `ranking1.txt` y `ranking2.txt`, armados con el índice para no agregar una constante que no está en el extendido.

Avisos que recibe y desde qué hilo:

Método	Lo llama	Hilo	`synchronized`
`enemigoDerrotado(e)`	`Enemigo.morir()`	`HiloJugador` (espada de Link)	Sí
`jefeDerrotado(j)`	`Jefe.morir()`	`HiloJugador`	No (recarga el nivel)
`powerUpRecogido(p)`	`Link.powerUpRecogido`	`HiloJugador`	Sí
`jugadorDaniado()`	`Link.recibirDanio`	`HiloJugador` o `HiloMundo` (espada de MasterStalfos)	Sí
`jugadorSinVidas()`	`Link.morir`, `Link.caerAlVacio`	Los dos	No (llama `terminarPartida`, que sí)
`jugadorCayoAlVacio()`	`Link.caerAlVacio`	`HiloJugador`	No
`cambiarDeSector(d)`	`Link.alSalirDelSector`	`HiloJugador`	No

Método	Lo llama	Hilo	`synchronized`
`agregarEntidad(e)`	Ataques, drops, suelo roto	Los dos	No (lo protege `Sector.agregar`)
`tic(segundos)`	`Reloj.ciclo`	`Reloj`	Sí
`terminarPartida()`	Modos, `jugadorSinVidas`, `jefeDerrotado`	Cualquiera	Sí
`transicionTerminada()`	`AnimadorTransicion`	El de la transición	No
`iniciar`, `registrarNombre`, `getRanking`	`GUI`	EDT	No

```
        public class Juego implements ControladorJuego {
            protected ControladorGrafico controladorGrafico;
            protected Link jugador;
            protected Nivel nivel;
            protected int numeroNivel;
            protected ModoJuego modoJuego;
            protected Ranking[] rankings;
            protected Reloj reloj;
            protected HiloJugador hiloJugador;
            protected HiloMundo hiloMundo;
            protected AdministradorColisiones colisiones;
            protected GeneradorNivel generadorNivel;
            protected FabricaEntidades fabricaEntidades;
            protected PersistenciaRanking persistencia;

            public Juego(ControladorGrafico vista) {
                controladorGrafico = vista;
                colisiones = new AdministradorColisiones();
                generadorNivel = new GeneradorNivel("/niveles/");
                persistencia = new PersistenciaRanking();
                rankings = new Ranking[3];
                for (int i = 0; i < 3; i++) rankings[i] = persistencia.cargar("ranking" + i + ".txt");
            }

            // ---- ControladorJuego ----
            public void iniciar(ModoJuego modo, FabricaSprites dominio) {
                modoJuego = modo;
                numeroNivel = 1;
                fabricaEntidades = new FabricaEntidades(this, dominio);
                jugador = fabricaEntidades.crearLink(0, 0);
                reloj = new Reloj(this);
                hiloJugador = new HiloJugador(jugador, colisiones);
                hiloMundo = new HiloMundo(colisiones);
                cargarNivel();
                reloj.iniciar();
                hiloJugador.iniciar();
                hiloMundo.iniciar();
            }

            public void registrarNombre(String nombre) {
                modoJuego.registrarPuntaje(this, new EntradaRanking(nombre, jugador.getPuntaje()));
                controladorGrafico.mostrarRanking(getRanking(modoJuego), modoJuego.getNombre());
            }

            public void transicionTerminada() {
                activarSector(colisiones.getSector());
                hiloJugador.reanudar();
                hiloMundo.reanudar();
            }

            public Ranking getRanking(ModoJuego m) { return m.getRanking(this); }   // sin instanceof
            public ControlJugador getControlJugador() { return jugador; }

            // ---- Avisos de las entidades ----
            public synchronized void enemigoDerrotado(Enemigo e) {
                jugador.sumarPuntos(e.getPuntos());
                jugador.incrementarRacha();
                if (jugador.getRacha() == 4) {
                    jugador.reiniciarRacha();
                    aparecerBellota(e.getX(), e.getY());
                }
            }

            public void jefeDerrotado(Jefe j) {
                jugador.sumarPuntos(j.getPuntos());
                if (!nivel.jefesDerrotados()) return;
                if (modoJuego.esUltimoNivel(numeroNivel)) {
                    terminarPartida();
                } else {
                    numeroNivel++;
                    hiloMundo.pausar();
                    cargarNivel();
                    hiloMundo.reanudar();
                }
```

```
}

public synchronized void powerUpRecogido(PowerUP p) { jugador.sumarPuntos(p.getPuntos()); }
public synchronized void jugadorDaniado() { modoJuego.alRecibirDanio(this); }
public void jugadorSinVidas() { terminarPartida(); }

public void jugadorCayoAlVacio() {
    hiloMundo.pausar();
    cargarNivel();                      // reinicia el nivel actual
    hiloMundo.reanudar();
}

public void cambiarDeSector(Direccion d) {
    Sector actual = colisiones.getSector();
    Sector nuevo = nivel.getMapa().getVecino(actual, d);
    if (nuevo == null) {                            // no hay vecino: vuelve al borde
        jugador.entrarPorBordeOpuesto(d.opuesta());
        return;
    }
    hiloJugador.pausar();
    hiloMundo.pausar();
    actual.retirar(jugador);
    jugador.entrarPorBordeOpuesto(d);
    nuevo.agregar(jugador);                         // cambio 4: actualiza jugador.miSector
    colisiones.setSector(nuevo);
    controladorGrafico.transicion(d);               // al terminar llama transicionTerminada()
}

public void agregarEntidad(Entidad e) {
    colisiones.getSector().agregar(e);
    e.registrarObserver(controladorGrafico.mostrarEntidad(e));
}

public synchronized void terminarPartida() {
    if (!reloj.estaActivo()) return;                 // ya terminó (evita terminar dos veces)
    reloj.detener();
    hiloJugador.detener();
    hiloMundo.detener();
    int puntaje = jugador.getPuntaje();
    if (getRanking(modoJuego).entraEnTop5(puntaje)) controladorGrafico.pedirNombre(puntaje);
    else controladorGrafico.mostrarRanking(getRanking(modoJuego), modoJuego.getNombre());
}

public synchronized void tic(int segundos) {
    controladorGrafico.actualizarTiempo(segundos);
    modoJuego.alPasarTiempo(this, segundos);
}

// ---- Rankings (los llama cada modo desde registrarPuntaje) ----
public void agregarAlRankingClasico(EntradaRanking e) { agregar(0, e); }
public void agregarAlRankingContrarreloj(EntradaRanking e) { agregar(1, e); }
public void agregarAlRankingNoHit(EntradaRanking e) { agregar(2, e); }

private void agregar(int i, EntradaRanking e) {
    if (rankings[i].insertar(e)) persistencia.guardar(rankings[i], "ranking" + i + ".txt");
}

// ---- Niveles y sectores ----
private void cargarNivel() {
    Sector anterior = jugador.getSector();          // cambio 4: Link sabe dónde está
    if (anterior != null) anterior.retirar(jugador);
    nivel = generadorNivel.generar(numeroNivel, fabricaEntidades);
    Sector inicio = nivel.getSectorInicial();
    jugador.setPosicion(nivel.getPosicionInicial());
    inicio.agregar(jugador);
    modoJuego.alIniciarNivel();
    controladorGrafico.actualizarNivel(numeroNivel);
    activarSector(inicio);
}

private void activarSector(Sector s) {
    colisiones.setSector(s);
    controladorGrafico.limpiarEscena();
    for (Entidad e : s.getEntidades())
        if (e != jugador) e.registrarObserver(controladorGrafico.mostrarEntidad(e));
    jugador.registrarObserver(controladorGrafico.mostrarJugador(jugador));   // Link arriba de todo
}

private void aparecerBellota(int x, int y) { agregarEntidad(fabricaEntidades.crearBellota(x, y)); }

// ---- Consultas ----
public Ranking getRankingClasico() { return rankings[0]; }
public Ranking getRankingContrarreloj() { return rankings[1]; }
public Ranking getRankingNoHit() { return rankings[2]; }
public Link getJugador() { return jugador; }
public FabricaEntidades getFabricaEntidades() { return fabricaEntidades; }
public int getNumeroNivel() { return numeroNivel; }
```

```
        public int getTiempo() { return reloj.getSegundos(); }
    }
```

Consideraciones:

- `e != jugador` en `activarSector` compara **referencias**, no tipos: no es `instanceof`. Sirve para mostrar a Link con su celda especial (`CeldaJugador`, que además actualiza el HUD).
- ⚠️ **Celdas viejas.** Cada vez que se activa un sector se registra una celda nueva en cada entidad. Las celdas del sector anterior siguen registradas como observadores (el extendido no tiene `quitarObserver`). Para que no hagan nada, `CeldaGrafica.actualizar()` ignora la actualización si ya no está en el panel (`getParent() == null`). La lista de observadores de Link crece una celda por cambio de sector; para una partida es poco y se acepta.
- ⚠️ **`getControlJugador()` antes de `iniciar` devuelve `null`**, porque Link se crea en `iniciar`. La `GUI` lo pide después.
- **Durante la transición, Link ya está en el borde del sector nuevo**, así que su celda salta en el panel viejo mientras se desliza. Si molesta, `PanelPantallaNivel.desplazar` puede esconder la celda de Link durante la animación. **(propuesta)**
- `pausar()` no espera a que el otro hilo termine su vuelta: `HiloMundo` puede estar terminando una vuelta en el sector viejo mientras `Juego` arma el nuevo. Lo peor que pasa es que un enemigo del sector viejo dé un paso más.
- El `Reloj` sigue corriendo durante la transición (el tiempo de la partida no se detiene). **(a confirmar)**
- ⚠️ **No llamar desde un método `synchronized` de `Juego` a `recibirDanio`, `aplicarDanio` ni `programar` de una entidad** (son `synchronized` en la entidad). Es la regla del orden de llaves de la sección 22.

18.4 ModoJuego*, Clasico, Contrarreloj y NoHit

Utilidad: las reglas que cambian según el modo elegido en el menú. `Juego` les avisa sucesos (empezó un nivel, Link recibió daño, pasó un segundo) y cada modo reacciona.

Elemento de `ModoJuego*`	Qué hace por defecto
`# cantidadNiveles : int`	2 en los tres modos
`# ModoJuego(cantidadNiveles)`	
`+ registrarPuntaje(j, e)`	Abstracto: lo manda al ranking de **su** modo
`+ alIniciarNivel()`	Nada
`+ alRecibirDanio(j)`	Nada
`+ alPasarTiempo(j, segundos)`	Nada
`+ esUltimoNivel(numero)`	`numero >= cantidadNiveles`
`+ getNombre()`	Abstracto
`+ getRanking(j)`	Abstracto: devuelve el ranking de su modo

Modo	Redefine	Efecto
`Clasico()`	`registrarPuntaje` → `j.agregarAlRankingClasico(e)`; `getNombre` → `"Clásico"`; `getRanking` → `j.getRankingClasico()`	Se juega hasta vencer al último jefe o quedarse sin vidas
`Contrarreloj(tiempoLimite)`	`alIniciarNivel` → `tiempoRestante = tiempoLimite`; `alPasarTiempo` → `tiempoRestante--` y, si llega a 0, `j.terminarPartida()`; `getTiempoLimite`, `getTiempoRestante`; ranking propio	Tiempo límite por nivel
`NoHit()`	`alRecibirDanio(j)` → `j.terminarPartida()`; ranking propio	El primer daño termina la partida

```
    public class Contrarreloj extends ModoJuego {
        protected int tiempoLimite;
        protected int tiempoRestante;

        public Contrarreloj(int tiempoLimite) {
            super(2);
            this.tiempoLimite = tiempoLimite;
        }

        public void registrarPuntaje(Juego j, EntradaRanking e) { j.agregarAlRankingContrarreloj(e); }
        public void alIniciarNivel() { tiempoRestante = tiempoLimite; }

        public void alPasarTiempo(Juego j, int segundos) {
            tiempoRestante--;
            if (tiempoRestante <= 0) j.terminarPartida();
        }

        public String getNombre() { return "Contrarreloj"; }
        public int getTiempoLimite() { return tiempoLimite; }
        public int getTiempoRestante() { return tiempoRestante; }
        public Ranking getRanking(Juego j) { return j.getRankingContrarreloj(); }
    }
```

Relaciones: `Juego` agrega el modo (◇ `#modoJuego 1`). Los crea la vista (`PanelPantallaPrincipal.elegirModo`), que conoce las tres clases concretas: es aceptable porque es la vista la que ofrece los botones.

Consideraciones:

- **`getRanking(j)` resuelve sin `instanceof` qué ranking mostrar:** `Juego.getRanking(m)` le pregunta al modo, y cada modo devuelve el suyo.

- **Tiempo en pantalla en Contrarreloj.** `tic` le pasa a la vista los segundos **transcurridos**. Si en Contrarreloj se quiere mostrar el tiempo **restante**, sin `instanceof` hay dos caminos: que el reloj cuente hacia atrás en ese modo, o que `ModoJuego` tenga una consulta común (por ejemplo `tiempoAMostrar(segundos)`), que habría que agregar al extendido. **(a confirmar)**

18.5 Ranking, EntradaRanking y PersistenciaRanking

Clase	Elementos	Qué hacen
`Ranking`	`- MAXIMO = 5`, `# entradas : List<EntradaRanking>` (◆ `#entradas 0..5`)	`insertar(e)`: si entra, la agrega ordenada de mayor a menor y deja 5; devuelve si entró. `entraEnTop5(puntaje)`, `getEntradas()`, `cantidad()`
`EntradaRanking`	`# nombre`, `# puntaje`	Un renglón del ranking. Inmutable
`PersistenciaRanking`	—	`guardar(r, archivo)` escribe un renglón `nombre;puntaje` por entrada. `cargar(archivo)` lee el archivo y devuelve el `Ranking` (vacío si el archivo no existe)

```
public class Ranking {
    private static final int MAXIMO = 5;
    protected List<EntradaRanking> entradas;

    public Ranking() { entradas = new ArrayList<>(); }

    public boolean insertar(EntradaRanking e) {
        if (!entraEnTop5(e.getPuntaje())) return false;
        entradas.add(e);
        entradas.sort((a, b) -> Integer.compare(b.getPuntaje(), a.getPuntaje()));
        if (entradas.size() > MAXIMO) entradas.remove(MAXIMO);
        return true;
    }

    public boolean entraEnTop5(int puntaje) {
        return entradas.size() < MAXIMO || puntaje > entradas.get(entradas.size() - 1).getPuntaje();
    }

    public List<EntradaRanking> getEntradas() { return new ArrayList<>(entradas); }
    public int cantidad() { return entradas.size(); }
}

public class PersistenciaRanking {
    public PersistenciaRanking() { }

    public void guardar(Ranking r, String archivo) {
        try (PrintWriter salida = new PrintWriter(new FileWriter(archivo, StandardCharsets.UTF_8))) {
            for (EntradaRanking e : r.getEntradas())
                salida.println(e.getNombre().replace(";", " ") + ";" + e.getPuntaje());
        } catch (IOException ex) {
            System.err.println("No se pudo guardar el ranking: " + ex.getMessage());
        }
    }

    public Ranking cargar(String archivo) {
        Ranking r = new Ranking();
        File f = new File(archivo);
        if (!f.exists()) return r;
        try (BufferedReader lector = new BufferedReader(new FileReader(f, StandardCharsets.UTF_8))) {
            String linea;
            while ((linea = lector.readLine()) != null) {
                String[] partes = linea.split(";");
                if (partes.length == 2) r.insertar(new EntradaRanking(partes[0], Integer.parseInt(partes[1].trim())));
            }
        } catch (IOException | NumberFormatException ex) {
            System.err.println("Ranking dañado, se empieza vacío: " + ex.getMessage());
        }
        return r;
    }
}
```

Consideraciones:

- ⚠️ El ranking se guarda **fuera** del `.jar` (en la carpeta desde donde se ejecuta el juego): los recursos de un `.jar` son de solo lectura.
- Un empate con el quinto **no** entra (`>`). Se puede cambiar a `>=` si el equipo prefiere que el último en llegar desplace al anterior.
- `PersistenciaRanking` no está en la vista ni usa Swing: es lógica de archivos con `java.io`.

---

19. Hilos

19.1 Runnable (de Java)

`java.lang.Runnable`, con `+ run()`. No se programa: se implementa. La realizan `HiloJuego*` y `AnimadorTransicion`. `AccionProgramada` **tiene** un `Runnable` pero no lo realiza.

19.2 HiloJuego* (abstracta, Template Method)

Utilidad: el ciclo "repetir, si no está en pausa hacer una vuelta, dormir", escrito una sola vez. Cada subclase dice qué es una vuelta (`ciclo()`) y cada cuánto (`periodo`).

Elemento	Qué hace
`# hilo : Thread`	El hilo de Java. Se crea en `iniciar()`
`# activo : boolean (volatile)`	`false` termina el ciclo
`# pausado : boolean (volatile)`	`true` saltea las vueltas (transición, recarga de nivel)
`# periodo : long`	Milisegundos entre vueltas
`# HiloJuego(periodo)`	
`+ iniciar()`	`activo = true`, `pausado = false`, `new Thread(this).start()`
`+ detener()`	`activo = false`
`+ pausar()`, `+ reanudar()`	Cambian `pausado`
`+ run()`	El ciclo. `final`: ninguna subclase lo cambia
`# ciclo()`	Abstracto: una vuelta
`+ estaActivo()`, `+ estaPausado()`	

```
public abstract class HiloJuego implements Runnable {
    protected Thread hilo;
    protected volatile boolean activo;
    protected volatile boolean pausado;
    protected long periodo;

    protected HiloJuego(long periodo) { this.periodo = periodo; }

    public void iniciar() {
        activo = true;
        pausado = false;
        hilo = new Thread(this);
        hilo.start();
    }

    public void detener() { activo = false; }
    public void pausar() { pausado = true; }
    public void reanudar() { pausado = false; }

    public final void run() {
        while (activo) {
            if (!pausado) ciclo();
            try {
                Thread.sleep(periodo);
            } catch (InterruptedException e) {
                activo = false;
            }
        }
    }

    protected abstract void ciclo();

    public boolean estaActivo() { return activo; }
    public boolean estaPausado() { return pausado; }
}
```

19.3 HiloJugador, HiloMundo y Reloj

Clase	Atributos	Constructor	`ciclo()`
`HiloJugador`	`# jugador : Link` (→ `#jugador 1`), `# colisiones` (→ `#colisiones 1`)	`super(30)`	`jugador.actualizarEstado(colisiones)`
`HiloMundo`	`# colisiones` (→ `#colisiones 1`)	`super(30)`	`Sector s = colisiones.getSector(); if (s != null) s.actualizarEntidades(colisiones);`
`Reloj`	`# segundos`, `# juego` (→ `#juego 1`)	`super(1000)`	`segundos++` y `juego.tic(segundos)`

`Reloj` además tiene `+ reiniciar()` (`segundos = 0`) y `+ getSegundos()`.

Consideraciones:

- ⚠️ **Una excepción dentro de `ciclo()` mata el hilo** y el juego queda congelado sin aviso. Mientras se programa, conviene que `run()` atrape `RuntimeException` alrededor de `ciclo()` y la imprima (`e.printStackTrace()`), para ver el error en la consola y que el juego siga. **(propuesta)**
- Solo el sector **visible** se actualiza: los enemigos de los demás sectores quedan quietos hasta que Link entre.
- Los tres hilos se crean en `iniciar` y se detienen en `terminarPartida`. Una partida nueva crea hilos nuevos (un `Thread` terminado no se puede volver a arrancar).

---

20. Vista

20.1 GUI

Utilidad: la fachada de la capa gráfica. Realiza `ControladorGrafico` (lo que le pide la lógica) y `ControladorDeVistas` (navegación entre pantallas). Tiene la ventana y los tres paneles, y procesa el teclado.

Atributo	Relación
`# ventana : Ventana`	◆ `#ventana 1`
`# panelPrincipal`, `# panelRanking`, `# panelNivel`	→ `#panelPrincipal 1`, `#panelRanking 1`, `#panelNivel 1`
`# controladorJuego : ControladorJuego`	→ `#controladorJuego 1`
`# controlJugador : ControlJugador`	→ `#controlJugador 1`

Método	Qué hace
`+ GUI()`	Crea la ventana y los tres paneles (pasándose como `ControladorDeVistas`) y registra el teclado
`+ setControladorJuego(c)`	Lo llama `main`
`+ mostrarPantallaPrincipal()`	`ventana.mostrarPanel(panelPrincipal)`
`+ mostrarPantallaRanking(m)`	Pide `controladorJuego.getRanking(m)`, lo carga en el panel y lo muestra
`+ iniciarPartida(modo, dominio)`	Muestra el panel del nivel, `controladorJuego.iniciar(...)` y guarda `controlJugador`
`+ mostrarEntidad(e) : Observer`	Crea la `CeldaGrafica`, la agrega al panel (en el EDT) y la devuelve
`+ mostrarJugador(j) : Observer`	Igual con `CeldaJugador`
`+ limpiarEscena()`, `transicion(d)`, `actualizarTiempo(s)`, `actualizarNivel(n)`	Delegan en el panel del nivel (en el EDT). `transicion` arranca un `AnimadorTransicion`
`+ pedirNombre(puntaje)`	Un `JOptionPane` y `controladorJuego.registrarNombre(nombre)`
`+ mostrarRanking(r, nombreModo)`	Carga el ranking en su panel y lo muestra
`# procesarTecla(codigo, presionada)`	Traduce la tecla a una acción de `ControlJugador`

```
public class GUI implements ControladorGrafico, ControladorDeVistas {
    protected Ventana ventana;
    protected PanelPantallaPrincipal panelPrincipal;
    protected PanelPantallaRanking panelRanking;
    protected PanelPantallaNivel panelNivel;
    protected ControladorJuego controladorJuego;
    protected ControlJugador controlJugador;

    public GUI() {
        ventana = new Ventana("The Legend of Zelda");
        panelPrincipal = new PanelPantallaPrincipal(this);
        panelRanking = new PanelPantallaRanking(this);
        panelNivel = new PanelPantallaNivel(this);
        KeyboardFocusManager.getCurrentKeyboardFocusManager().addKeyEventDispatcher(e -> {
            if (e.getID() == KeyEvent.KEY_PRESSED) procesarTecla(e.getKeyCode(), true);
            else if (e.getID() == KeyEvent.KEY_RELEASED) procesarTecla(e.getKeyCode(), false);
            return false;
        });
    }

    public void setControladorJuego(ControladorJuego c) { controladorJuego = c; }

    // ---- ControladorDeVistas (los llaman los paneles, en el EDT) ----
    public void mostrarPantallaPrincipal() { ventana.mostrarPanel(panelPrincipal); }

    public void mostrarPantallaRanking(ModoJuego m) {
        panelRanking.mostrarRanking(controladorJuego.getRanking(m), m.getNombre());
        ventana.mostrarPanel(panelRanking);
    }

    public void iniciarPartida(ModoJuego modo, FabricaSprites dominio) {
        ventana.mostrarPanel(panelNivel);
        controladorJuego.iniciar(modo, dominio);
        controlJugador = controladorJuego.getControlJugador();   // después de iniciar
    }

    // ---- ControladorGrafico (los llama la lógica, desde otros hilos) ----
    public Observer mostrarEntidad(EntidadLogica e) {
        CeldaGrafica celda = new CeldaGrafica(e, panelNivel);
        SwingUtilities.invokeLater(() -> panelNivel.agregarCelda(celda));
        return celda;
    }

    public Observer mostrarJugador(EntidadJugador j) {
        CeldaJugador celda = new CeldaJugador(j, panelNivel);
        SwingUtilities.invokeLater(() -> panelNivel.agregarCelda(celda));
        return celda;
    }

    public void limpiarEscena() { SwingUtilities.invokeLater(panelNivel::limpiar); }
```

```
    public void transicion(Direccion d) {
        new Thread(new AnimadorTransicion(panelNivel, d, controladorJuego)).start();
    }

    public void actualizarTiempo(int segundos) {
        SwingUtilities.invokeLater(() -> panelNivel.actualizarTiempo(segundos));
    }

    public void actualizarNivel(int numero) {
        SwingUtilities.invokeLater(() -> panelNivel.actualizarNivel(numero));
    }

    public void pedirNombre(int puntaje) {
        SwingUtilities.invokeLater(() -> {
            String nombre = JOptionPane.showInputDialog(ventana, "Puntaje: " + puntaje + ". Tu nombre:");
            controladorJuego.registrarNombre(nombre == null || nombre.isBlank() ? "Anónimo" : nombre);
        });
    }

    public void mostrarRanking(Ranking r, String nombreModo) {
        SwingUtilities.invokeLater(() -> {
            panelRanking.mostrarRanking(r, nombreModo);
            ventana.mostrarPanel(panelRanking);
        });
    }

    protected void procesarTecla(int codigo, boolean presionada) {
        if (controlJugador == null) return;                    // no hay partida
        switch (codigo) {
            case KeyEvent.VK_UP:
                if (presionada) controlJugador.mover(Direccion.ARRIBA); else controlJugador.detenerMovimiento();
                break;
            case KeyEvent.VK_DOWN:
                if (presionada) controlJugador.mover(Direccion.ABAJO); else controlJugador.detenerMovimiento();
                break;
            case KeyEvent.VK_LEFT:
                if (presionada) controlJugador.mover(Direccion.IZQUIERDA); else controlJugador.detenerMovimiento();
                break;
            case KeyEvent.VK_RIGHT:
                if (presionada) controlJugador.mover(Direccion.DERECHA); else controlJugador.detenerMovimiento();
                break;
            case KeyEvent.VK_A: if (presionada) controlJugador.atacar(); else controlJugador.soltarAtaque(); break;
            case KeyEvent.VK_S: if (presionada) controlJugador.usarEscudo(); else controlJugador.soltarEscudo(); break;
            case KeyEvent.VK_D: if (presionada) controlJugador.cambiarObjeto(); break;
            default: break;
        }
    }
}
```

> **Por qué los cuatro casos se repiten:** un método auxiliar privado (por ejemplo `moverODetener(d, presionada)`) lo haría más corto, pero no está en el extendido. Si el equipo lo quiere, se agrega primero al diagrama.

Consideraciones:

- ⚠️ **Constructor de `CeldaGrafica`.** El extendido dice `CeldaGrafica(e)`, pero la celda tiene el atributo `# panelNivel` y no hay forma de asignarlo (no hay *setter*). El código de arriba usa `CeldaGrafica(e, panelNivel)`, como ya tiene `CeldaJugador(j, p)`. **Hay que actualizar el constructor en el extendido. (a confirmar)**
- **Soltar una flecha** llama `detenerMovimiento()` aunque el jugador tenga otra flecha apretada; Link se frena hasta que el sistema repita la otra tecla (unos 500 ms). Para evitarlo hace falta recordar la última flecha, un atributo que no está en el diagrama. Se acepta así. **(a confirmar)**
- **Teclas:** A ataca, S escudo, D cambia de objeto, flechas para moverse. Ajustar si el enunciado fija otras.
- El `KeyEventDispatcher` evita el problema clásico del foco (un `KeyListener` en un panel deja de andar si el foco se va a un botón). Es Swing estándar.
- `iniciarPartida` corre en el EDT y `Juego.iniciar` lee el archivo del nivel ahí mismo. Tarda muy poco; si se notara, se puede pasar a un hilo.

20.2 Ventana

Utilidad: el `JFrame` de la aplicación. `+ Ventana(titulo)` arma la ventana (tamaño 768 × 528 más el HUD, `EXIT_ON_CLOSE`, centrada); `+ mostrarPanel(p : JPanel)` cambia el panel visible: `setContentPane(p); revalidate(); repaint();`. Es parte de `GUI` (◆ `#ventana 1`).

Consideración: en el reducido `Ventana` no tiene flecha a `JFrame`, aunque lo es. Programarla como `class Ventana extends JFrame` y, si el ayudante lo pide, dibujar la herencia como se hizo con `JPanel` y `JLabel`. **(a confirmar)**

20.3 PanelVista* y los tres paneles

`PanelVista*` (abstracta, hereda de `JPanel`): `# controladorVistas` (→ `#controladorVistas 1`), `# PanelVista(c)` (guarda el controlador y llama `armarComponentes()`), `# armarComponentes()` abstracto.

Panel	Atributos	Métodos
`PanelPantallaPrincipal`	`- dominio : FabricaSprites`, `- modo :`	`elegirDominio(d)` y `elegirModo(m)` (los llaman los botones), `jugar()` → `controladorVistas.iniciarPartida(modo, dominio)`, `verRanking()` →

Panel	Atributos	Métodos
	`ModoJuego`	`controladorVistas.mostrarPantallaRanking(modo)`, `armarComponentes()`: botones de dominio (Original / Alternativo), de modo (Clásico / Contrarreloj / NoHit), Jugar y Ranking
`PanelPantallaRanking`	`- tabla : JTable`, `- titulo : JLabel`	`mostrarRanking(r, nombreModo)`: llena la tabla con las entradas, `volver()` → `controladorVistas.mostrarPantallaPrincipal()`, `armarComponentes()`
`PanelPantallaNivel`	`# celdas : List<CeldaGrafica>` (◆ `#celdas 0..*`) y seis `JLabel` del HUD	`agregarCelda(c)`, `quitarCelda(c)`, `limpiar()`, `actualizarHUD(j)`, `actualizarTiempo(s)`, `actualizarNivel(n)`, `desplazar(d)`, `armarComponentes()`

```
public class PanelPantallaNivel extends PanelVista {
    protected List<CeldaGrafica> celdas;
    private JLabel etiquetaNivel;
    private JLabel etiquetaPuntaje;
    private JLabel etiquetaTiempo;
    private JLabel etiquetaVidas;
    private JLabel etiquetaRupias;
    private JLabel etiquetaObjeto;

    public PanelPantallaNivel(ControladorDeVistas c) { super(c); }

    protected void armarComponentes() {
        celdas = new ArrayList<>();
        setLayout(null);                                    // las celdas se ubican con setBounds
        setPreferredSize(new Dimension(Sector.ANCHO, Sector.ALTO + 40));
        // crear las seis etiquetas del HUD y ubicarlas en la franja de abajo (y = 528)
    }

    public void agregarCelda(CeldaGrafica c) {              // siempre en el EDT
        celdas.add(c);
        add(c, 0);                                          // índice 0: queda dibujada arriba de las anteriores
        c.actualizar();
        repaint();
    }

    public void quitarCelda(CeldaGrafica c) {
        celdas.remove(c);
        remove(c);
        repaint();
    }

    public void limpiar() {
        for (CeldaGrafica c : new ArrayList<>(celdas)) quitarCelda(c);
    }

    public void actualizarHUD(EntidadJugador j) {
        etiquetaVidas.setText("Vidas: " + j.getVidas());
        etiquetaPuntaje.setText("Puntos: " + j.getPuntaje());
        etiquetaRupias.setText("Rupias: " + j.getRupias());
        etiquetaObjeto.setText("Objeto: " + j.getNombreObjetoEquipado());
        // corazones: j.getCorazones() / 2.0 de j.getCorazonesMaximos() / 2
    }

    public void desplazar(Direccion d) {                    // lo llama AnimadorTransicion, fuera del EDT
        for (int paso = 1; paso <= 30; paso++) {
            int dx = -d.getDx() * getWidth() * paso / 30;
            int dy = -d.getDy() * getHeight() * paso / 30;
            SwingUtilities.invokeLater(() -> {
                for (CeldaGrafica c : celdas)
                    c.setLocation(c.entidadObservada.getX() + dx, c.entidadObservada.getY() + dy);
            });
            try {
                Thread.sleep(15);                           // el sleep va fuera del invokeLater
            } catch (InterruptedException e) {
                return;
            }
        }
    }
}
```

> **Nota sobre `desplazar`.** Cada celda se corre tomando la posición lógica de su entidad (`entidadObservada`, que el panel ve porque está en el mismo paquete) más el desplazamiento del paso. Las entidades del sector viejo están quietas porque los hilos están en pausa. Link ya está en el sector nuevo, así que su celda puede verse corrida durante la animación; si molesta, se esconde con `setVisible(false)` al empezar y vuelve al activarse el sector. Usa `getWidth()` y `getHeight()` del panel en lugar de `Sector.ANCHO`, para que la vista no dependa de una clase de la lógica. **(propuesta)**

Consideraciones:

- ⚠️ `add(c, 0)`: en Swing, el componente con índice **más bajo** se dibuja **arriba**. Si se usara `add(c)` (al final), Link, que se agrega último, quedaría **debajo** del pasto.

- El panel usa `setLayout(null)`: cada celda se ubica con `setBounds(x, y, ancho, alto)` en píxeles, los mismos de la lógica. Por eso la lógica trabaja en píxeles.
- `PanelVista*` llama a `armarComponentes()` desde su constructor: en las subclases, los atributos se inicializan **dentro** de `armarComponentes()` (no en la declaración), porque ese código corre antes que los inicializadores de la subclase. ⚠️ Es un error común de Java.

20.4 ControladorDeVistas (interfaz)

`mostrarPantallaPrincipal()`, `mostrarPantallaRanking(m)`, `iniciarPartida(modo, dominio)`. La realiza `GUI`; la referencia cada panel (`#controladorVistas 1`). Así los paneles cambian de pantalla sin conocer a `GUI` ni a `Juego`.

20.5 Observer, CeldaGrafica y CeldaJugador

`Observer` (interfaz): `+ actualizar()`. La realiza `CeldaGrafica`; la referencia `Entidad*` (`#observadores 0..*`).

`CeldaGrafica` (hereda de `JLabel`, realiza `Observer`): la imagen de **una** entidad en el panel. Es el "mapeo 1 a 1" entre entidad lógica y gráfica que pide la defensa 6.

Elemento	Qué hace
`# entidadObservada : EntidadLogica`	→ `#entidadObservada 1`
`# panelNivel : PanelPantallaNivel`	Dónde está (para sacarse al eliminarse la entidad)
`+ CeldaGrafica(e)` (ver la consideración de 20.1)	Guarda la entidad y el panel
`+ actualizar()`	En el EDT: si la entidad fue eliminada, `panelNivel.quitarCelda(this)`; si no, cambia ícono y posición

`CeldaJugador` (hereda de `CeldaGrafica`): `# jugadorObservado : EntidadJugador` (→ `#jugadorObservado 1`), `+ CeldaJugador(j, p)`, `+ actualizar()`: lo mismo que la celda común y además `panelNivel.actualizarHUD(jugadorObservado)`.

```
        public class CeldaGrafica extends JLabel implements Observer {
            protected EntidadLogica entidadObservada;
            protected PanelPantallaNivel panelNivel;

            public CeldaGrafica(EntidadLogica e, PanelPantallaNivel p) {
                entidadObservada = e;
                panelNivel = p;
            }

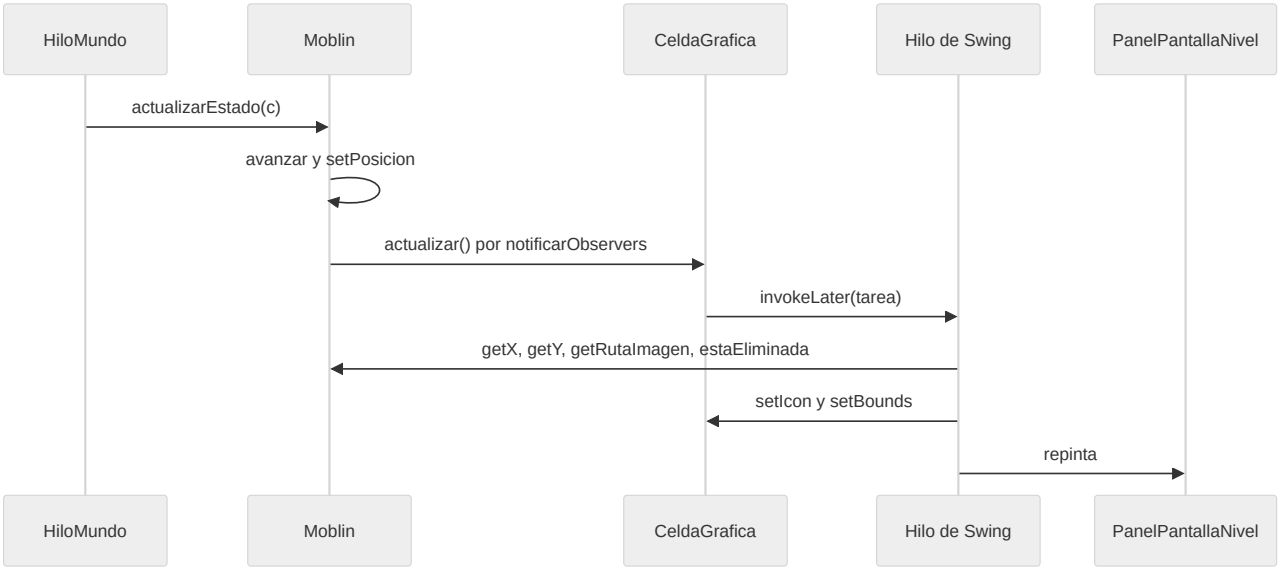
            public void actualizar() {                          // lo llaman los hilos de la lógica
                SwingUtilities.invokeLater(() -> {
                    if (getParent() == null) return;                // celda vieja (otro sector): ignorar
                    if (entidadObservada.estaEliminada()) {
                        panelNivel.quitarCelda(this);
                        return;
                    }
                    ImageIcon icono = new ImageIcon(getClass().getResource(entidadObservada.getRutaImagen()));
                    setIcon(icono);
                    setBounds(entidadObservada.getX(), entidadObservada.getY(), icono.getIconWidth(), icono.getIconHeight());
                });
            }
        }

        public class CeldaJugador extends CeldaGrafica {
            protected EntidadJugador jugadorObservado;

            public CeldaJugador(EntidadJugador j, PanelPantallaNivel p) {
                super(j, p);
                jugadorObservado = j;
            }

            public void actualizar() {
                super.actualizar();
                SwingUtilities.invokeLater(() -> panelNivel.actualizarHUD(jugadorObservado));
            }
        }
```

Diagrama de secuencia: Observer, de la lógica a la pantalla:



Consideraciones:

- `agregarCelda` y `actualizar` **llegan en orden:** los dos pasan por `invokeLater`, que respeta el orden de llegada. Por eso la primera `actualizar()` siempre encuentra la celda ya dentro del panel. La comprobación de "celda vieja" solo descarta las celdas de sectores anteriores (las que `limpiar()` sacó).
- **Imágenes:** `new ImageIcon(URL)` usa el caché de imágenes de Java (`Toolkit`), así que crear el ícono en cada actualización no vuelve a leer el archivo. No hace falta un caché propio (que sería un atributo nuevo).
- ⚠️ Si `getResource` devuelve `null` (ruta mal escrita), `new ImageIcon(null)` tira excepción. Mientras se programa, conviene revisar que existan todas las rutas que arma `FabricaSprites`.
- La celda lee la entidad **en el EDT**, en el momento de dibujar. Puede leer una posición un poco más nueva que la del aviso: no importa, siempre muestra lo último.

20.6 AnimadorTransicion

Utilidad: la animación de deslizar la pantalla al cambiar de sector, en un hilo propio de la vista.

Elemento	Qué hace
`# panel : PanelPantallaNivel`	→ `#panel 1`
`# direccion : Direccion`	Hacia dónde se desliza
`# controlador : ControladorJuego`	→ `#controlador 1`: a quién avisar
`+ AnimadorTransicion(p, d, c)`	
`+ run()`	`panel.desplazar(direccion)` y `controlador.transicionTerminada()`

Relaciones: realiza `Runnable`. La crea `GUI.transicion(d)` (dependencia no dibujada).

---

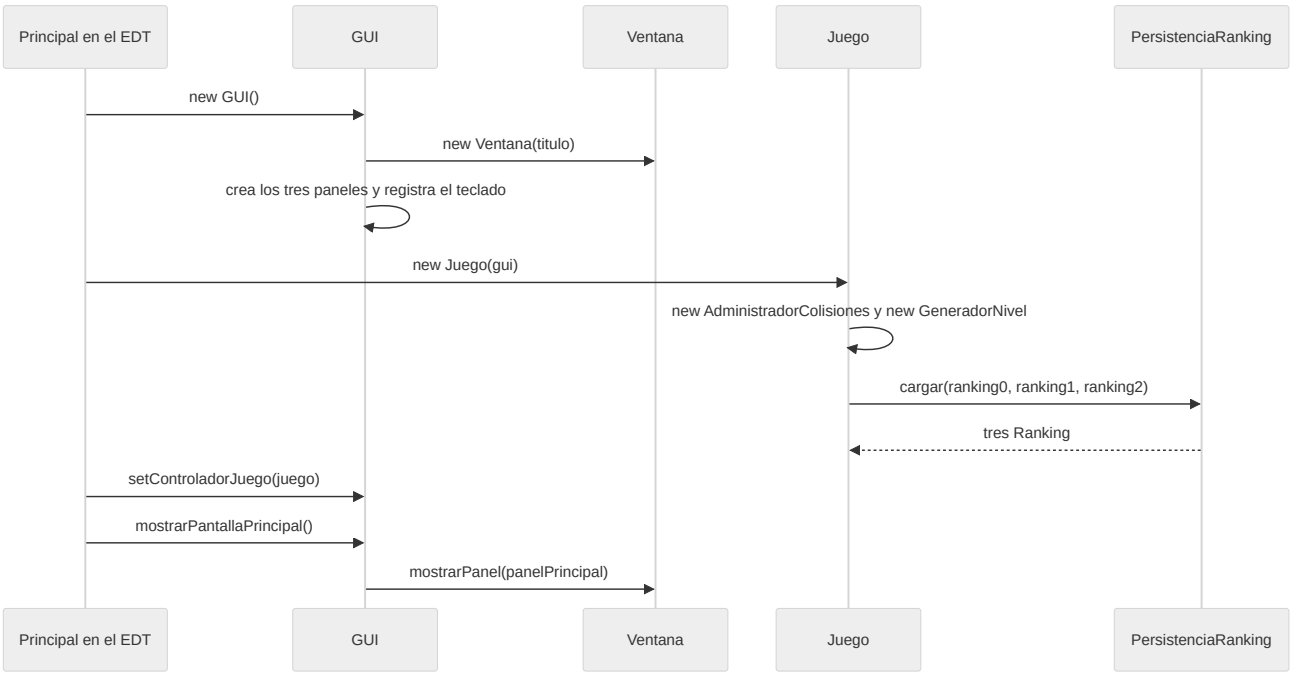
21. Flujos completos (diagramas de secuencia)

Cada flujo indica en qué hilo corre. Los nombres de los participantes son objetos del juego.

21.1 Arranque de la aplicación

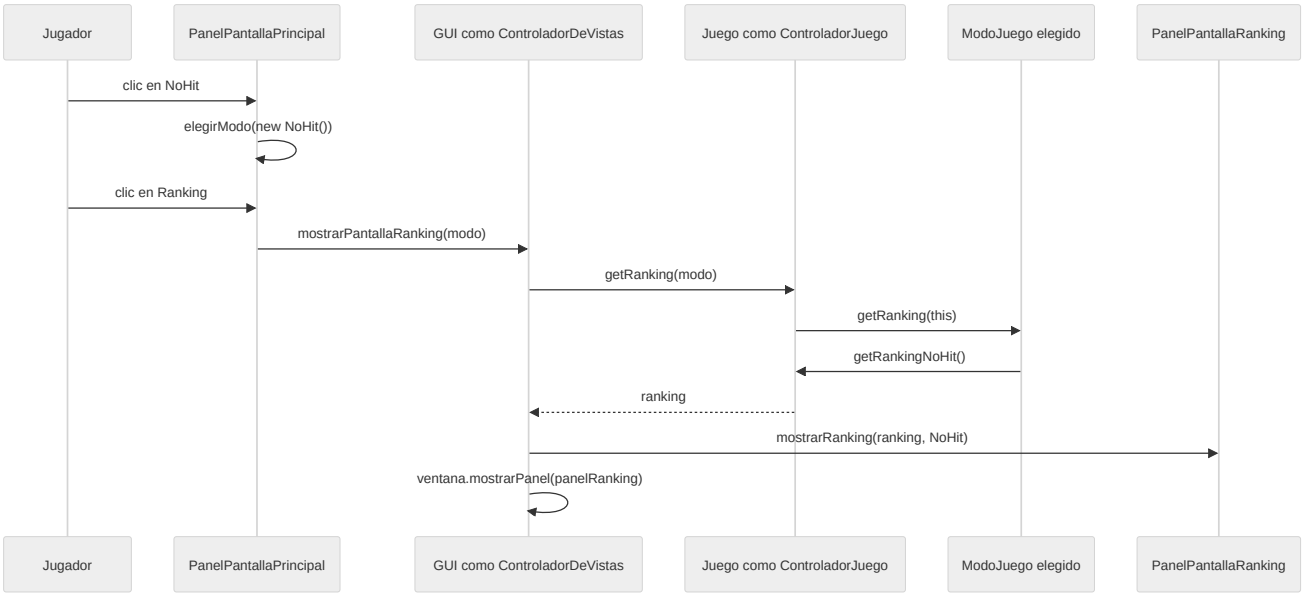
La clase con `main` no está en el diagrama. Arma la vista, la lógica y las conecta:

```
        public class Principal {
            public static void main(String[] args) {
                SwingUtilities.invokeLater(() -> {          // Swing se arma en su hilo
                    GUI gui = new GUI();
                    Juego juego = new Juego(gui);           // Juego conoce a la vista como ControladorGrafico
                    gui.setControladorJuego(juego);         // la vista conoce a Juego como ControladorJuego
                    gui.mostrarPantallaPrincipal();
                });
            }
        }
```



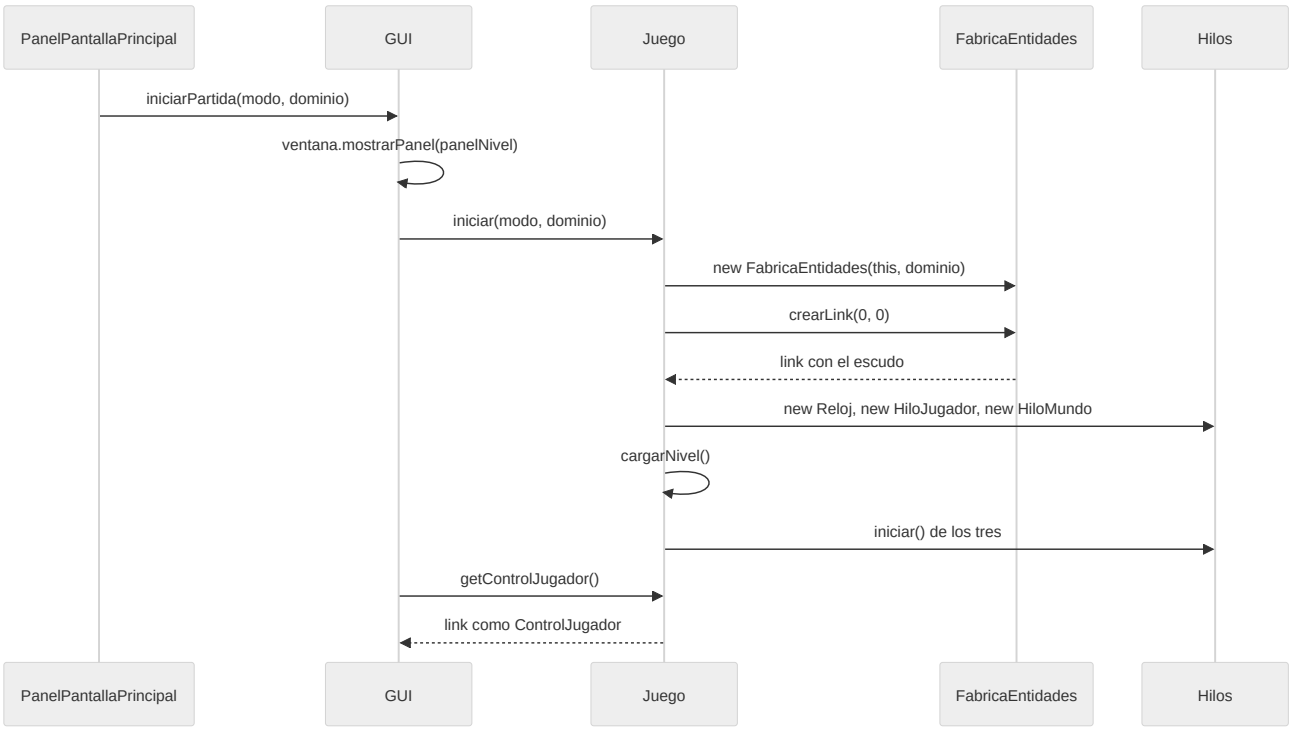
21.2 Del menú al ranking

En el EDT. El jugador elige un modo y aprieta "Ranking":



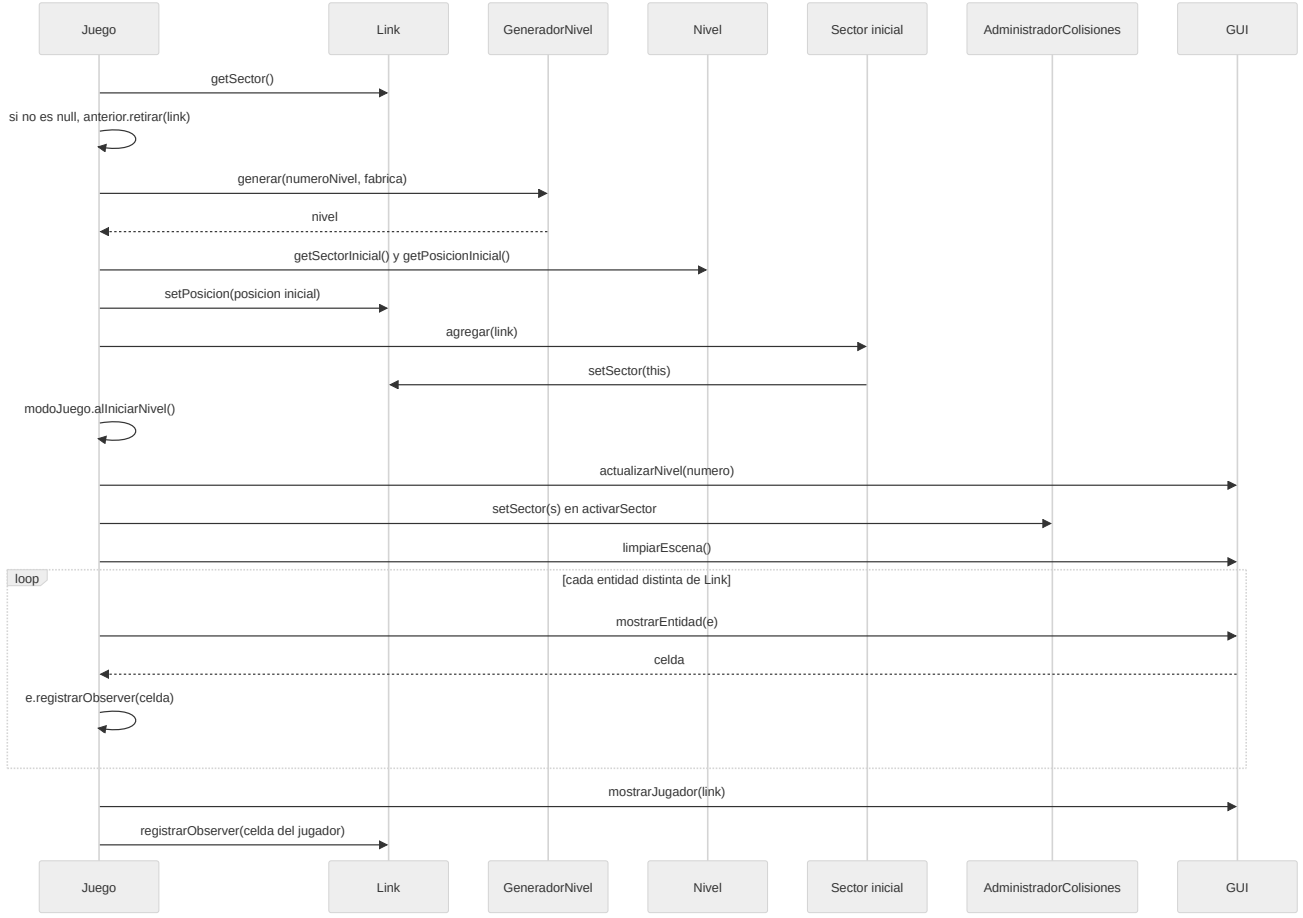
21.3 Iniciar una partida

En el EDT hasta que arrancan los hilos:



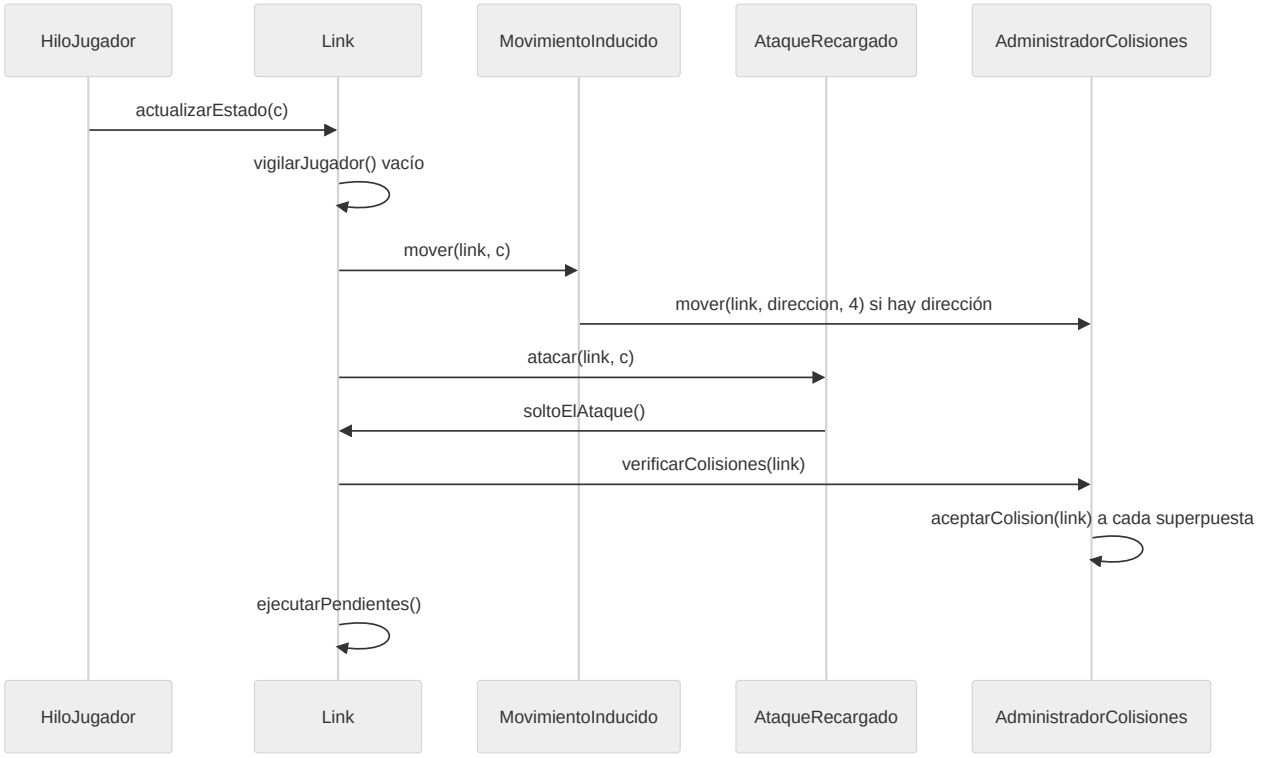
21.4 Cargar un nivel y activar el sector inicial

`cargarNivel()` (privado de `Juego`). Corre en el EDT al iniciar, y en `HiloJugador` al caer al vacío o pasar de nivel:



21.5 Una vuelta de HiloJugador

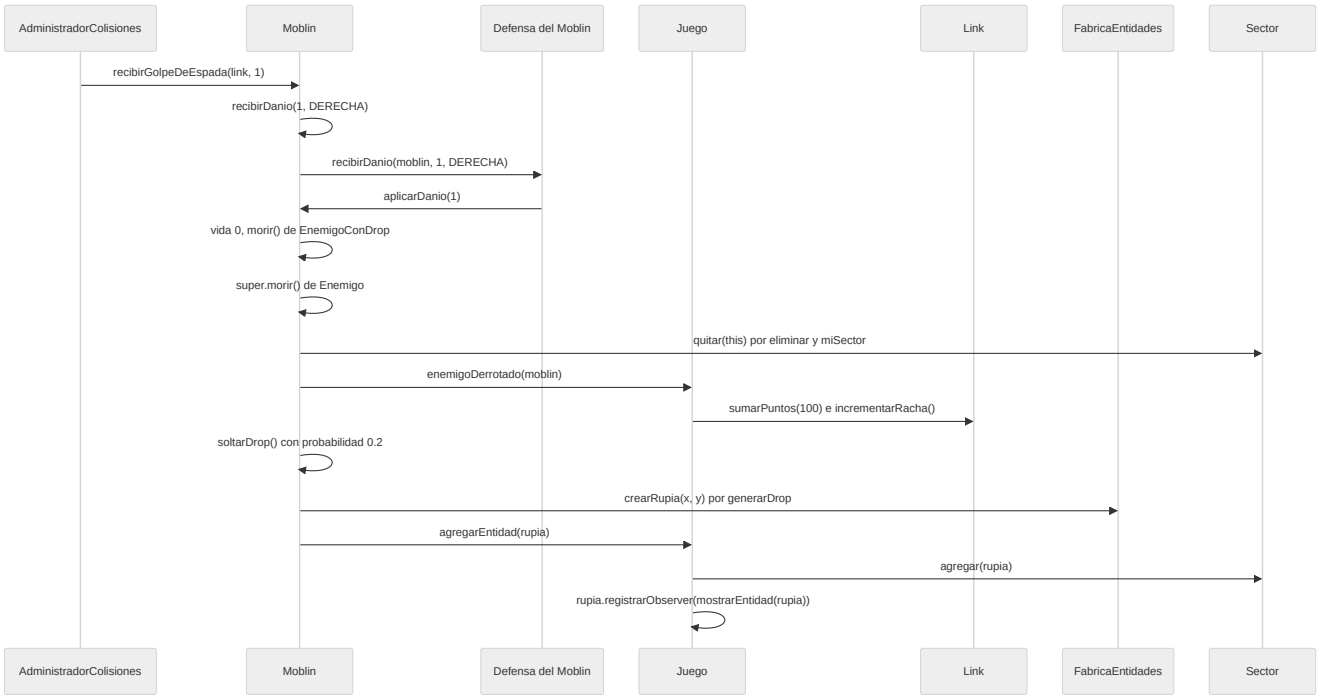
Cada 30 ms, aunque el jugador no toque nada:



La vuelta de `HiloMundo` está en 12.1 (un Moblin).

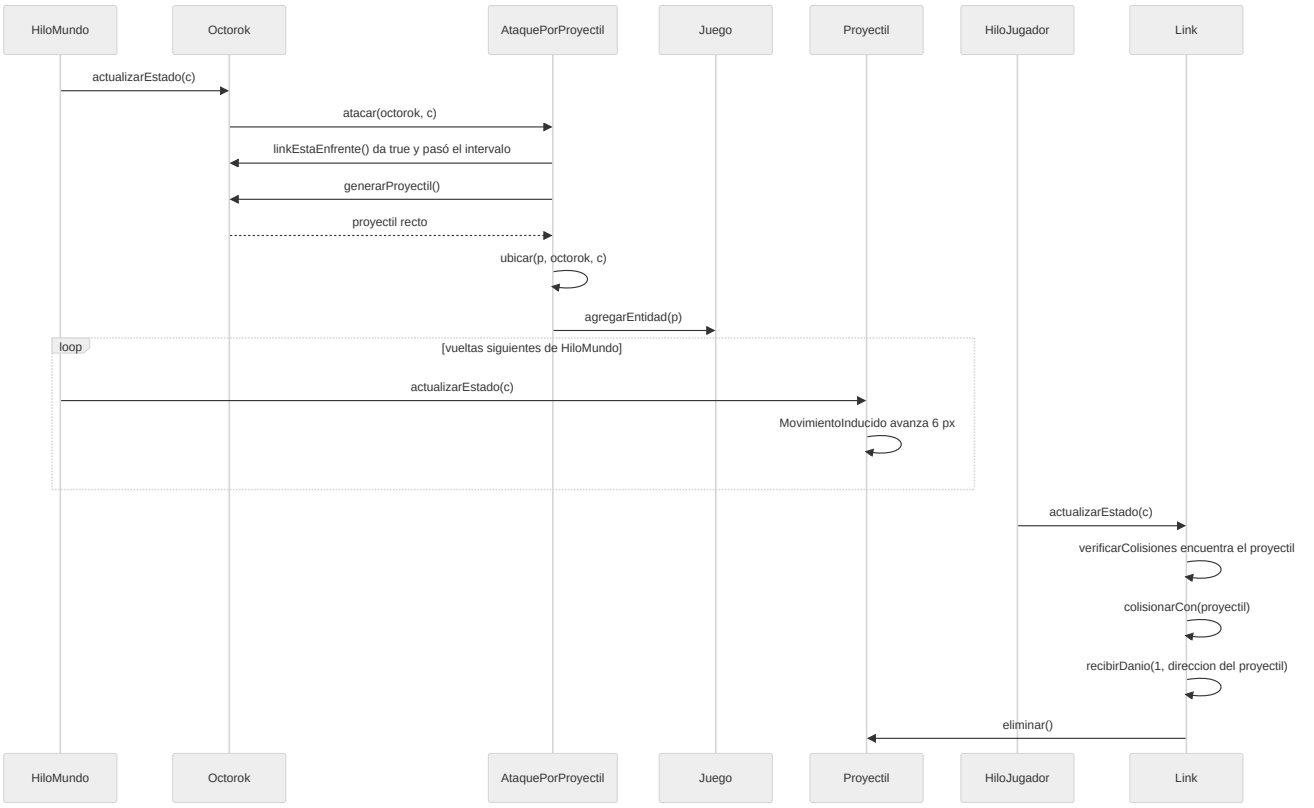
21.6 Link mata a un Moblin y aparece un drop

En `HiloJugador`, dentro del ataque de Link:



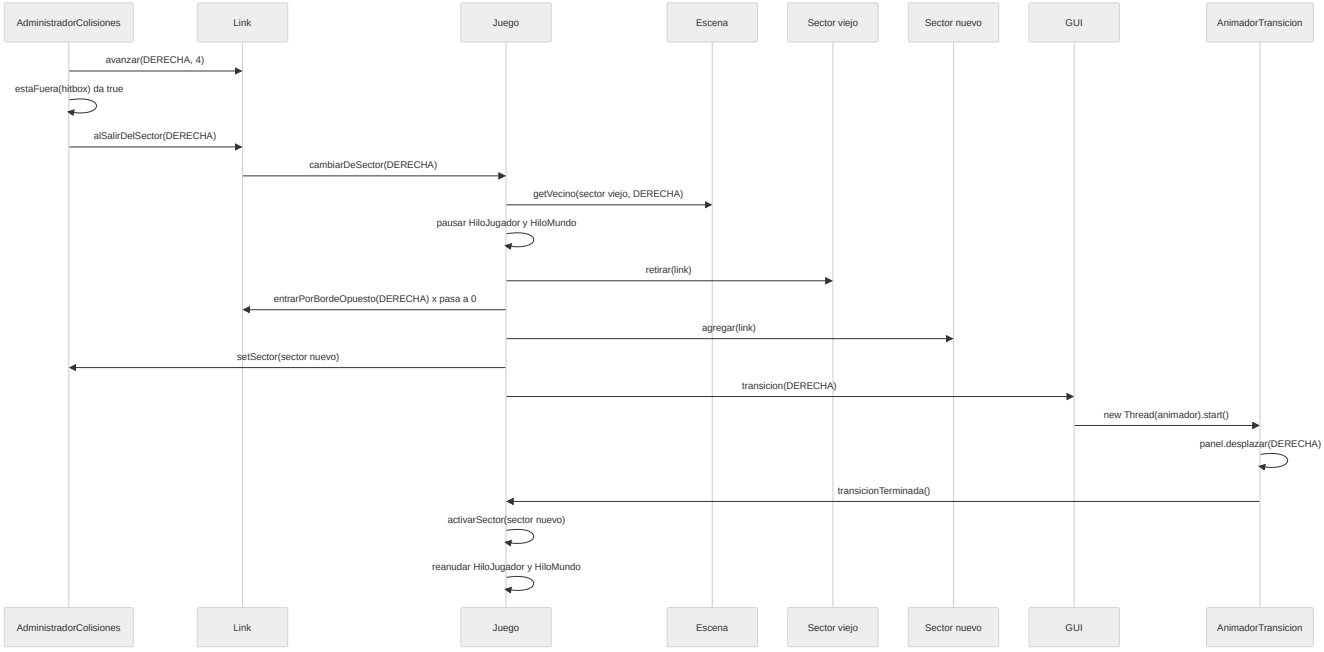
21.7 El Octorok dispara y la roca alcanza a Link quieto

El disparo y el vuelo en `HiloMundo`; el impacto en `HiloJugador`:

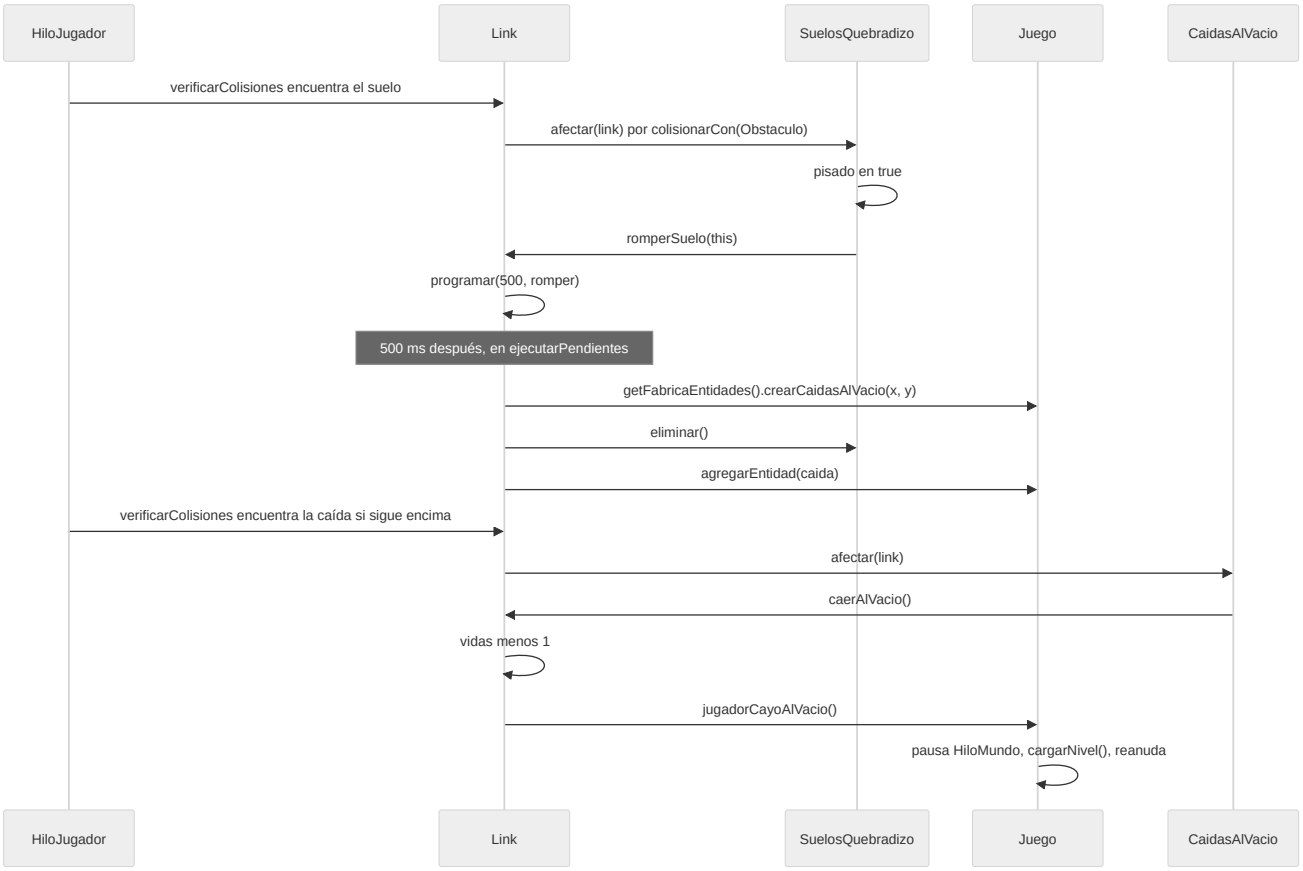


21.8 Cambio de sector

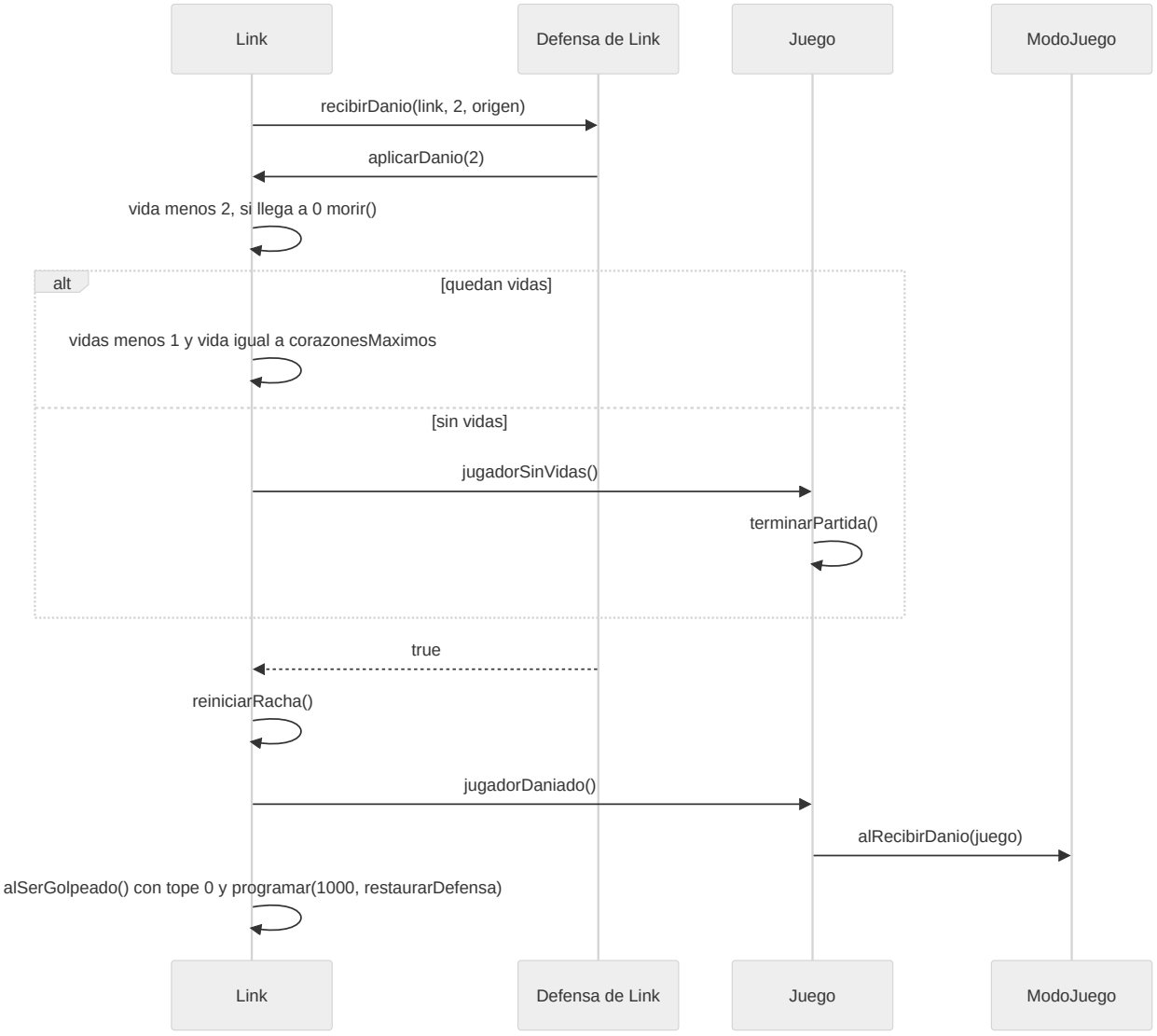
Empieza en `HiloJugador`, sigue en el hilo de la transición y termina otra vez en los dos hilos de juego:



21.9 Suelo quebradizo y caída al vacío

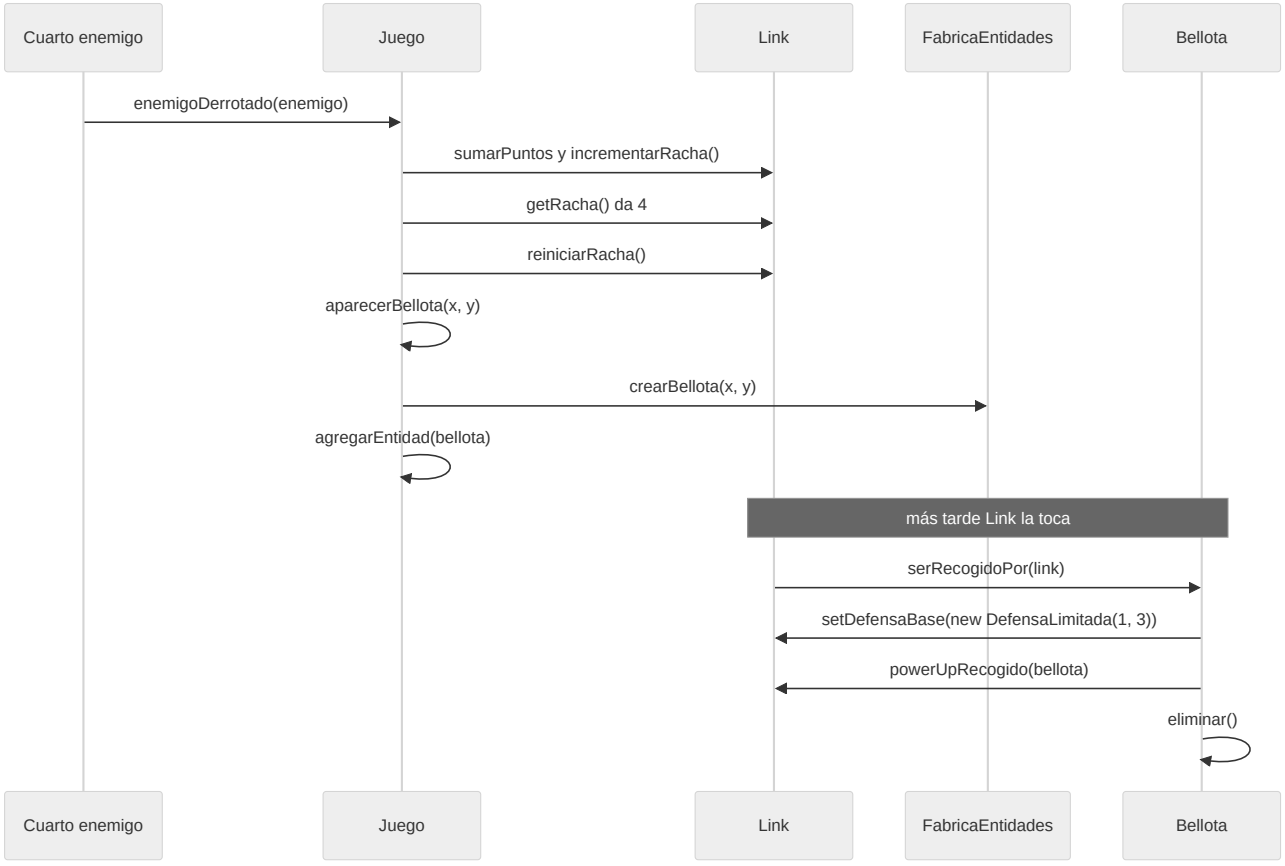


21.10 Daño, vidas y fin por vidas

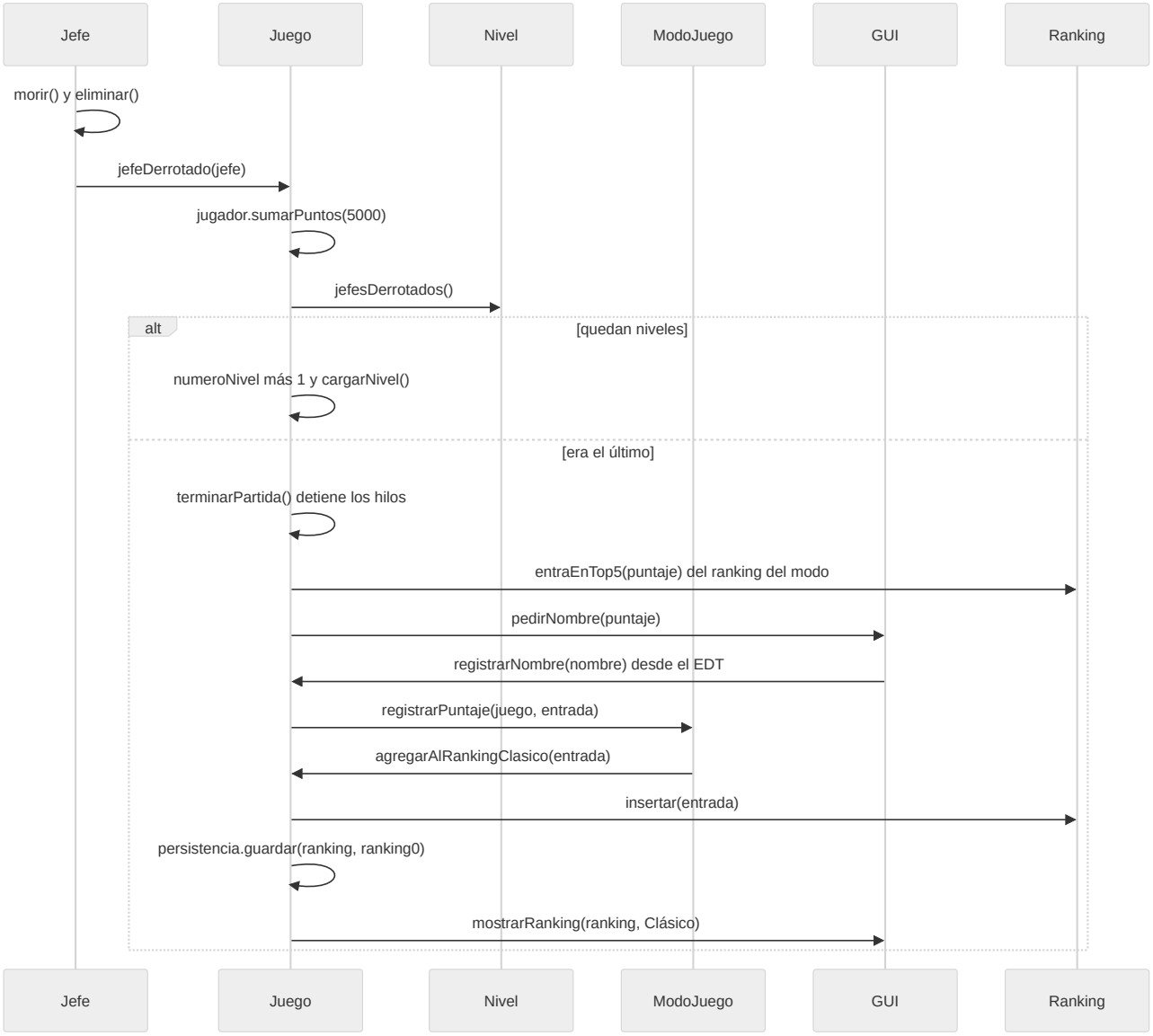


21.11 Aparece una bellota

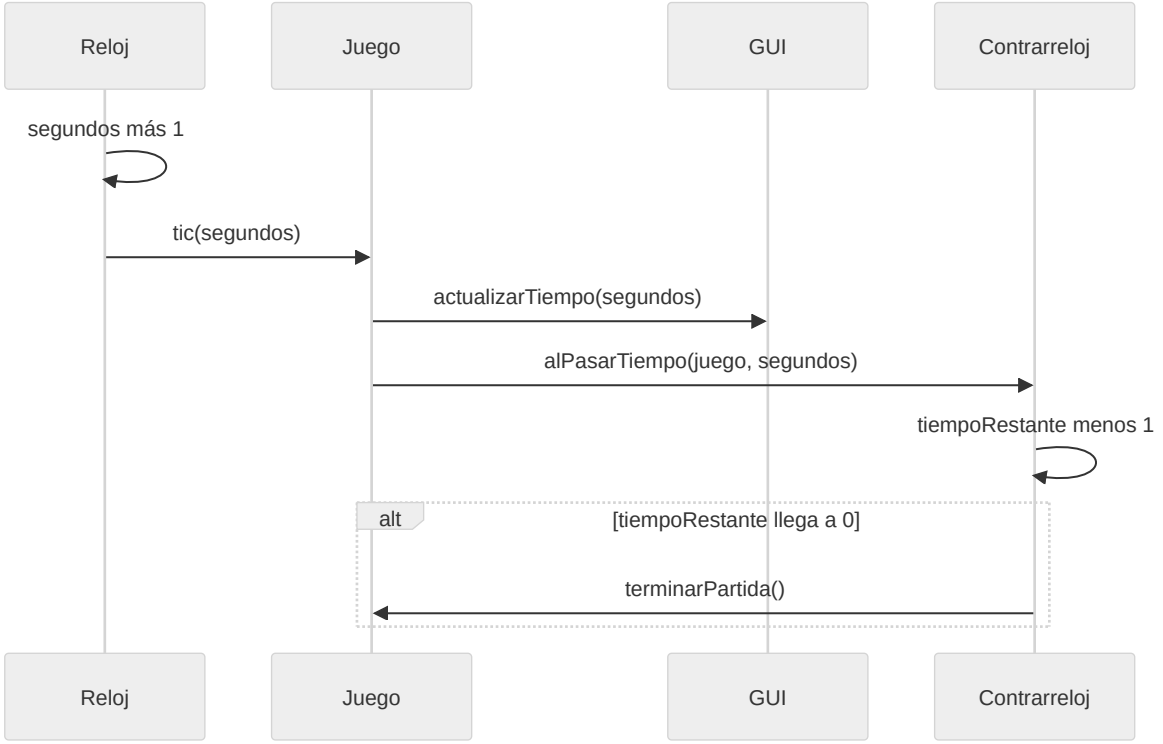
Link derrota 4 enemigos seguidos sin recibir daño:



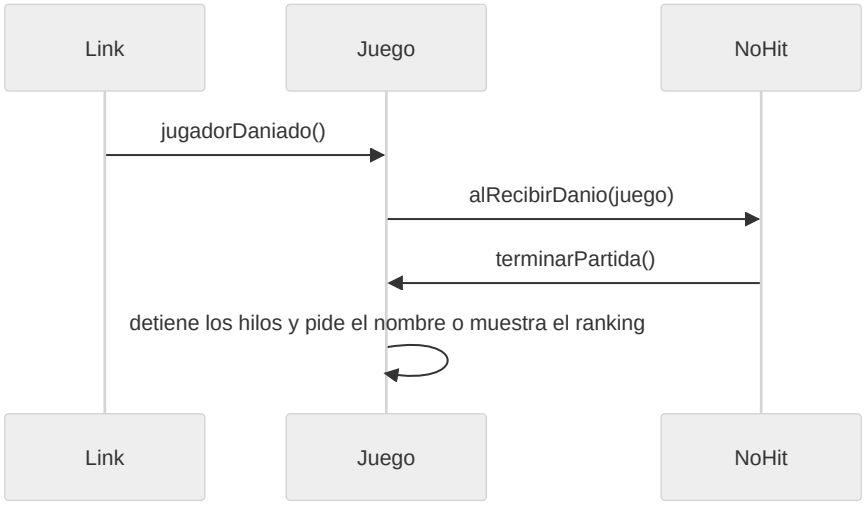
21.12 Jefe derrotado, fin de nivel y ranking



21.13 Contrarreloj: un segundo



21.14 NoHit: el primer daño termina la partida



En Clásico y Contrarreloj `alRecibirDanio` es el gancho vacío de `ModoJuego*`, así que no pasa nada.

---

22. Concurrencia: qué se sincroniza y por qué

22.1 Los hilos y qué toca cada uno

Hilo	Qué hace	Toca
EDT (Swing)	Teclado, botones, dibujar	Intenciones de Link (`volatile`), componentes Swing
`HiloJugador`	Link: moverse, atacar, chocar, pendientes	Link, entidades del sector visible, `Juego`
`HiloMundo`	Todas las demás entidades del sector visible	Enemigos, proyectiles, a veces Link (espada del jefe), `Juego`
`Reloj`	Un `tic` por segundo	`Juego`, modo
`AnimadorTransicion`	Deslizar la pantalla y avisar	Panel (vía `invokeLater`), `Juego.transicionTerminada`

22.2 La regla de las colisiones

> **Defensa 6: el choque Link–enemigo lo resuelve `HiloMundo`.** Los demás choques en los que participa Link (proyectiles, obstáculos, power-ups, objetos) los sigue resolviendo `HiloJugador`.

Se cumple sin `instanceof`. Cuando `HiloMundo` detecta un Moblin encima de Link, termina en `moblin.colisionarCon(link)`, que llama a `ataque.ataqueAlContactar(moblin, link)` y la estrategia aplica el daño. Cuando `HiloJugador` detecta lo mismo, termina en `link.colisionarCon(moblin)`, que tiene implementación vacía. Un solo daño por choque.

`recibirDanio` de `EntidadAnimada*` no cambia: lo que cambia es que ahora lo llama `HiloMundo`. Como es `synchronized` (también la versión de Link), la vida y la defensa no se pisan aunque Link se esté moviendo en su hilo. Y como `verificarColisiones` no es `synchronized`, el enemigo no tiene la llave del sector cuando llama a `recibirDanio`: se respeta el orden de 22.4.

22.3 Qué se protege

Dato compartido	Protección
Lista de `Sector`	`synchronized` en `agregar`, `quitar`, `retirar` y `getEntidades` (que devuelve una copia). **No** en `actualizarEntidades`
Vida y defensa de una entidad	`synchronized` en `recibirDanio` y `aplicarDanio`
`pendientes`	`synchronized` en `programar` y `sacarVencidas`. Las acciones se ejecutan **fuera** del lock
Puntaje y estado de la partida	`synchronized` en `Juego.enemigoDerrotado`, `powerUpRecogido`, `jugadorDaniado`, `tic` y `terminarPartida`
`activo`, `pausado`, intenciones de Link, `MovimientoInducido.direccion`, `AdministradorColisiones.sector`	`volatile`
Observadores	`CopyOnWriteArrayList`
Swing	Solo desde el EDT, con `SwingUtilities.invokeLater`

22.4 ⚠️ El abrazo mortal que hay que evitar

Esta guía **corrige** lo que dicen los explicativos sobre sincronizar `actualizarEntidades`, `mover` y `verificarColisiones`. Si se sincronizan, el juego se puede trabar así:

1. `HiloMundo` entra a `sector.actualizarEntidades(c)` (`synchronized`) y **tiene la llave del sector**. Dentro, mueve a MasterStalfos, cuya espada llama `link.recibirDanio(...)` (o un enemigo que toca a Link, defensa 6), que necesita **la llave de Link**.
2. Al mismo tiempo, `HiloJugador` está en `link.recibirDanio(...)` (**tiene la llave de Link**) porque tocó a un enemigo, y adentro Link muere, cae o recoge algo que termina en `sector.quitar(...)` o `sector.agregar(...)`, que necesita **la llave del sector**.
3. Cada hilo espera la llave que tiene el otro. Ninguno avanza nunca.

Las tres reglas que lo evitan:

1. **Orden de llaves: Entidad → `Juego` → `Sector`.** Un hilo puede pedir la llave de `Juego` teniendo la de una entidad, y la del sector teniendo cualquiera, pero **nunca al revés**.
2. **Nunca ejecutar código de entidades con la llave del sector tomada.** Por eso `actualizarEntidades` recorre una copia **sin** `synchronized`, y `AdministradorColisiones` no tiene métodos `synchronized`.
3. **Dentro de un método `synchronized` de `Juego`, no llamar a `recibirDanio`, `aplicarDanio` ni `programar`** de ninguna entidad. Los métodos de Link que sí llama `Juego` (`sumarPuntos`, `incrementarRacha`, `reiniciarRacha`) **no** son `synchronized`.

Cadena	¿Respeta el orden?
`Link.recibirDanio` (Link) → `Juego.jugadorDaniado` (Juego) → `NoHit` → `terminarPartida` (Juego)	Sí
`Moblin.recibirDanio` (Moblin) → `morir` → `eliminar` → `Sector.quitar` (Sector)	Sí
`Juego.enemigoDerrotado` (Juego) → `aparecerBellota` → `Sector.agregar` (Sector)	Sí
`Sector.actualizarEntidades` **sincronizado** (Sector) → `link.recibirDanio` (Link)	**No: abrazo mortal**

22.5 Lo que corre en el EDT

- `procesarTecla` y los métodos de `ControlJugador` que llama: solo escriben atributos `volatile`. No pueden ser `synchronized` (si el EDT espera una llave, la ventana se congela).
- `GUI.iniciarPartida` y `Juego.iniciar`: arman la partida antes de que existan los hilos, así que no compiten con nadie.
- `pedirNombre` → `registrarNombre`: la partida ya terminó y los hilos están detenidos.

---

23. Correspondencia 1 a 1 con el extendido

La defensa 6 evalúa que el código coincida con el extendido. Esta tabla junta lo que en esta guía **no** coincide exactamente, para decidir una vez y actualizar el diagrama si hace falta.

Clase	Qué	Por qué	Qué hacer
`CeldaGrafica`	Constructor `(e, p)` en lugar de `(e)`	Hay que asignar `# panelNivel`	Cambiar el constructor en el extendido
`Direccion`	`dx` y `dy` como atributos del enumerado	Comodidad	Dibujarlos, o escribir `getDx()`/`getDy()` con `switch`
`TrampaDeCuchillas`, `MoblinConEspadaYEscudo`	Redefinen `# vigilarJugador()`	Es donde se usa	Agregar el método en las dos cajas
`Proyectil`	Detectar la llegada de las rocas de la avalancha	Nadie llama `alChocarConBloqueo()` con `avanzarHacia`	Elegir una opción de 14.1 y dibujarla si agrega un método
`Ventana`	`extends JFrame`	Es el `JFrame`	Dibujar la herencia si el ayudante la pide
Visibilidad	`alChocarConBloqueo()` y `alSalirDelSector(d)` son `#`	Los llama `AdministradorColisiones`	Ponerlo en el paquete de las entidades (sección 3) o pasarlos a `+`
`volatile`, `synchronized`, `final`	No se dibujan	Son detalles de Java	No hace falta dibujarlos
Constantes	`ARCHIVOS` de los rankings no existe	Se arma con el índice (`"ranking" + i + ".txt"`)	Nada
`Juego`	`getControlJugador()` devuelve `null` antes de `iniciar`	Link se crea en `iniciar`	Nada; la `GUI` lo pide después

Diferencias con los explicativos (manda el extendido):

En el explicativo	En el extendido (y en esta guía)
`getRankings()`	`getRanking(m : ModoJuego)` y los tres `getRankingX()`
`Proyectil.colisionarCon(Link)` con código	No existe: queda el gancho vacío heredado. (Defensa 6: `Enemigo.colisionarCon(Link)` sí existe y delega en `ataque.ataqueAlContactar`)
`iniciar()` / `detener()` en entidades, `p.desaparecer()` llamado desde Link	No existen / `desaparecer()` es privado: Link usa `p.eliminar()`
`Wizzrobe.recibirDanio` redefinido	`Wizzrobe.alSerGolpeado()`
`StoneHinox.iniciar()`	El constructor programa el primer ataque
`actualizarEntidades`, `mover` y `verificarColisiones` sincronizados	Sin `synchronized` (22.4)

---

24. Decisiones abiertas y cosas que el diagrama no fija

#	Tema	Opciones	Propuesta de esta guía
1	Paquetes	Cualquier estructura que respete las capas	La de la sección 3, con `AdministradorColisiones` junto a las entidades
2	Teclas	Las que diga el enunciado	Flechas, A (espada), S (escudo), D (cambiar objeto)
3	Escudo	Se usa solo si está equipado, o siempre	Solo equipado
4	Perder una vida (sin caer)	Sigue en el lugar, o reinicia el nivel	Sigue en el lugar con los corazones llenos
5	Tiempo en Contrarreloj	Mostrar transcurrido o restante	Restante, con una consulta común en `ModoJuego` (hay que dibujarla)
6	Reloj durante la transición	Corre o se pausa	Corre
7	Rocas de "lanzar rocas"	Juntas, en abanico o seguidas	En abanico, corriéndolas en `ubicar`
8	Llegada de las rocas de la avalancha	Quedan quietas o desaparecen al llegar	Desaparecen (redefinir `avanzarHacia` en `Proyectil`)
9	Hitbox de las caídas	Completo o reducido	Reducido (24 × 24)
10	Empate en el ranking	Entra o no	No entra
11	Sonidos	El enunciado los pide en el juego final	Una clase de la vista (por ejemplo `Sonidos`) con `javax.sound.sampled`; la lógica no los conoce. Hay que agregarla al diagrama cuando se haga
12	Daño entre enemigos por la espada del jefe	Se permite o no	Se permite (no hay otros enemigos en su sector)
13	Valores numéricos (velocidades, tiempos)	Los de la sección 5	Ajustarlos jugando

---

25. Checklist antes de entregar

Diseño y reglas:

- ☐ Ningún `instanceof` ni `getClass()` para decidir (buscar en todo el proyecto).
- ☐ Ningún `import javax.swing` ni `java.awt` fuera del paquete de la vista.
- ☐ Ningún `javax.swing.Timer` ni `java.util.Timer`.
- ☐ Ninguna librería externa.
- ☐ Cada clase, atributo y método del extendido existe en el código con el mismo nombre, y no hay de más (sección 23).
- ☐ Cada flecha del reducido es un atributo, un `extends` o un `implements` (sección 6).

Funcionamiento (requisitos de la defensa 6):

- ☐ Se carga el nivel 1 completo desde el archivo, y `Formateo-de-niveles.pdf` está en el paquete del parser.
- ☐ Se ven Link, los enemigos y los obstáculos sobre el nivel.
- ☐ El HUD muestra vidas, corazones, puntaje, rupias y objeto, y se actualiza solo (Observer).
- ☐ (Opcional) Link se mueve con las flechas en al menos un sector.
- ☐ (Opcional) Link choca con rocas y con al menos un enemigo o power-up.

Calidad (Clean Code):

- ☐ Indentación pareja en todo el proyecto (mismo formateador para los 4).
- ☐ Cada método hace una sola cosa y su nombre lo dice.
- ☐ Sin código comentado ni `System.out` de prueba.
- ☐ El código fuente está en una carpeta al mismo nivel que la de los diagramas.

---

26. Glosario

Término	Significado
Celda (del archivo)	Un carácter del archivo de nivel; mide 48 × 48 píxeles. Solo existe al leer el archivo
Celda gráfica	`CeldaGrafica`: el `JLabel` que dibuja una entidad
Doble despacho (Visitor)	`a.aceptarColision(b)` → `b.colisionarCon(a)`: Java elige el método según los dos tipos, sin `instanceof`
EDT	*Event Dispatch Thread*: el único hilo que puede tocar Swing
Gancho	Método vacío en una superclase para que las subclases lo redefinan si lo necesitan
Hitbox	Rectángulo en píxeles que ocupa una entidad
Medio corazón	Unidad de vida de Link y del daño hacia Link (`1`)
Sector	Una pantalla del mapa (768 × 528)
Sector visible	El que tiene `AdministradorColisiones`; el único que actualiza `HiloMundo`
Strategy	Comportamientos intercambiables (`setMovimiento`, `setAtaque`, `setDefensa`)
Template Method	Un método fijo que llama pasos redefinibles (`HiloJuego.run` → `ciclo`, `AtaquePorProyectil.atacar`)
Vuelta	Una ejecución de `ciclo()` de un hilo (cada 30 ms en los hilos de juego)

Término	Significado
Abrazo mortal	Dos hilos que esperan, cada uno, una llave que tiene el otro: el programa se congela